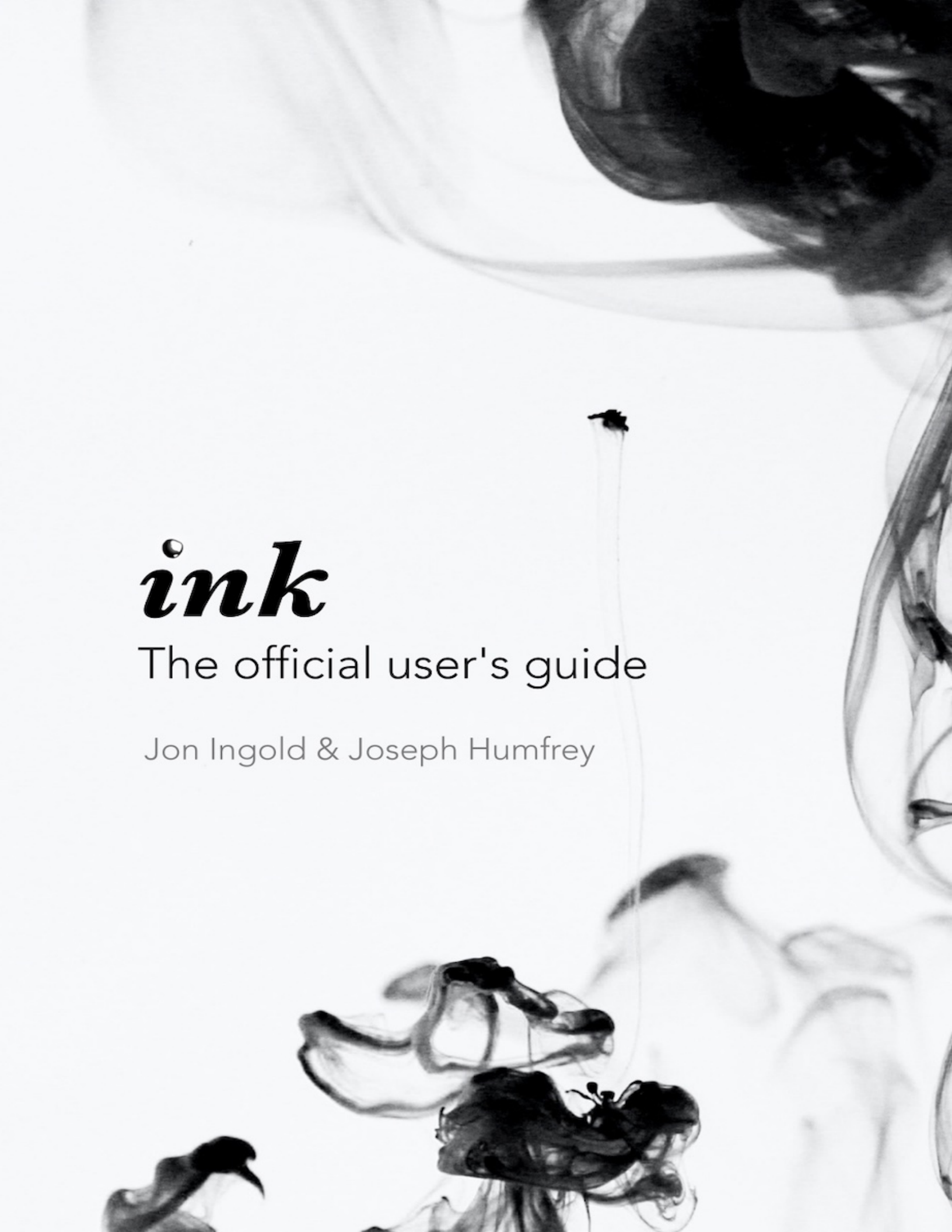


The background of the cover is a light, off-white color. It features several abstract ink splatters and blotches in various shades of grey and black. A thin, vertical line of ink extends from a small dark spot near the center towards the bottom. The overall aesthetic is artistic and minimalist.

ink

The official user's guide

Jon Ingold & Joseph Humfrey

inkle

Version 1.0.14

Ink Version 1.1

The text of this document is open source under an MIT licence. The text, including all source code examples may be freely copied, adapted and used.

The authors would like to recognise the invaluable contributions of multiple people to this document.

This formatted eBook edition (c) inkle Ltd. This copy belongs to cunningdave@gmail.com

ink

The official user's guide

Jon Ingold

&

Joseph Humfrey

*Writing
with Ink*

The Basics

ink is a scripting language built around the idea of marking up pure-text with flow in order to produce interactive scripts.

ink offers several features to enable non-technical writers to branch often, and play out the consequences of those branches, in both minor and major ways, with minimum fuss and complexity.

At its most basic, it can be used to write a Choose Your Own-style story, or a branching dialogue tree. But its real strength is in writing dialogues with lots of options and lots of recombination of the flow. It can be used to create entirely text-based games, or plugged into game engines and used to power the narrative components of more complex or more graphical games. ink has been used to make free to play mobile games, open-world 3D adventure games, customer chat-bots, consumer surveys, interactive novels, visual novels and more besides. Here at inkle we've used ink on every game we've shipped, and we occasionally reference these titles as examples – most notably *80 Days*, *Heaven's Vault* and *Overboard!* There are several other notable ink releases out there, but these are the ones we know well.

An ink script aims to be clean and logically ordered, so that branching dialogue can be tested “by eye”. The flow is described in a declarative fashion where possible.

It's also designed with editing and redrafting in mind; so skim-reading a flow should be fast, and moving it around convenient. And it starts from a core idea: the text comes first.

1. Writing Content

Software

You can write ink using any text editor, but the best software to use is inky: a text editor with the ink compiler built-in, which allows you to test your game as you write, and will help you locate and fix errors in your script quickly.

Games can be exported directly from inky as playable web-pages. With knowledge of JavaScript, these can be expanded into full games. ink can also be used with the Unity game engine using the ink-unity integration.

Links to all these resources can be found on the ink site:

<https://www.inklestudios.com/ink/>

There is also a community-developed integration for the Godot engine, and an (at time of writing) work-in-progress integration for Unreal.

The simplest ink script

The most basic ink script is just text in a `.ink` file.

```
Hello, world!
```

On running, this will output the content, and then stop.

Putting text onto separate lines produces separate lines of output content. The script:

```
Hello, world!  
Hello?
```

produces output that looks the same.

2. Choices

Input is offered to the player via text choices. A text choice is indicated by an `*` character.

If no other flow instructions are given, once made, the choice will flow into the next line of text.

```
Hello world!
*   Hello back!
    Nice to hear from you!
```

This produces the following “game”:

```
Hello world
1: Hello back!

> 1
Hello back!
Nice to hear from you.
```

Suppressing choice text

The example above demonstrates one of ink’s hardwired assumptions: that you’re going to be writing dialogue lines – that’s why the choice text is repeated in the response. But of course, you might not want to present your choices as verbatim lines of dialogue (and they might be actions, UI commands, and so forth.)

This is easy to do in ink: if the choice text is given in square brackets, then the text of the choice will not be printed into response. The ink

```
Hello world!
*   [Hello back!]
    Nice to hear from you!
```

produces

```
Hello world
1: Hello back!
```



```
> 1
Nice to hear from you.
```

Advanced: mixing choice and output text

The truth is a little more subtle: the square brackets *divide up* the option content. What's before is printed in both choice and output; what's inside only in the choice; and what's after, only in output. That is to say, the square brackets provide an alternative way for the line to end.

```
Hello world!
*   Hello [back!] right back to you!
```

produces:

```
Hello world
1: Hello back!
> 1
Hello right back to you!
```

This is most useful when writing “prose” dialogue choices, where the dialogue option is given a line of speech, which then turns into a full paragraph. (This format was used extensively in the game *80 Days*, but we’ve found it generally useful elsewhere!)

```
"What's that?" my master asked.
*   "I am somewhat tired[.]," I repeated.
    "Really," he responded. "How deleterious."
```

produces:

```
"What's that?" my master asked.
1. "I am somewhat tired."
> 1
"I am somewhat tired," I repeated.
"Really," he responded. "How deleterious."
```

Multiple choices

Anyway, to make our choices really choices, we need to provide *alternatives*. We do this in ink simply by listing them (the language syntax evolved from writing dialogue using bullet-points in a word processor; the concept is “what if that *just worked?*”)

```
"What's that?" my master asked.  
*   "I am somewhat tired[.]," I repeated.  
    "Really," he responded. "How deleterious."  
*   "Nothing, Monsieur!"[] I replied.  
    "Very good, then."  
*   "I said, this journey is appalling[.] and I want no  
more of it."  
    "Ah," he replied, not unkindly. "You are feeling  
frustrated."
```

This produces the following interactive moment:

```
"What's that?" my master asked.  
  
1: "I am somewhat tired."  
2: "Nothing, Monsieur!"  
3: "I said, this journey is appalling."  
  
> 3  
"I said, this journey is appalling and I want no more of  
it."  
"Ah," he replied, not unkindly. "You are feeling  
frustrated."
```

With the above syntax we can write a single set of choices – a short-lived thrill. For a real game, we'll need to move the flow onwards based on what the player chose. To do that, we'll need to take a moment to introduce a bit more structure.

3. Knots

Pieces of content are called knots

To allow the game to branch we need to mark-up sections of content with names (as an old-fashioned gamebook does with its ‘Paragraph 18’, and the like.)

Each section is called a “knot”, and knots are the fundamental structural unit of ink content.

Writing a knot

The start of a knot is indicated by two or more equals signs, as follows.

```
=== top_knot ===
```

(The equals signs on the end are optional; and the name needs to be a single word with no spaces. Programmers often use underscores in place of spaces.)

The start of a knot is its header; the content that follows will be inside that knot.

```
=== back_in_london ===  
We arrived into London at 9.45pm exactly.
```

There’s no marker for the end of a knot: a knot ends whenever the next one starts (or when the source file itself ends.)

4. Diverts

Knots divert to knots

You can tell the story to move from one knot to another using `->`, a “divert arrow”. Diverts happen immediately without any user input.

```
=== back_in_london ===  
  We arrived into London at 9.45pm exactly.  
  -> hurry_home  
  
=== hurry_home ===  
  We hurried home to Savile Row as fast as we could.
```

The first divert

When you start an ink file, content before the first knot will be run automatically. But knots themselves won't be. So if you start using knots, you'll need to tell the game where to go as part of the opening content, by diverting.

So the simplest knotty script that actually *does* anything is this:

```
-> top_knot  
  
=== top_knot ===  
  Hello world!
```

-> END

Once you start using knots, ink will start looking out for loose ends in the story. It produces a warning on compilation and/or run-time when it thinks it's found a place in the story where the story runs out of places to go. For instance, the “hello world” script above produces this on compilation:

```
WARNING: Apparent loose end exists where the flow runs  
out. Do you need a '-> END' statement, choice or divert?  
on line 3 of tests/test.ink
```

and this on running:

```
Runtime error in tests/test.ink line 3: ran out of
content. Do you need a '-> DONE' or '-> END'?
```

To remove the warning and the error we need to tell the compiler “the story is meant to stop here” and for that we use the `-> END` divert. (It’s not really a divert, it just looks like one, but in practice no one cares.)

The following plays and compiles without error:

```
-> top_knot

=== top_knot ===
  Hello world!
-> END
```

`-> END` is a marker for both the writer and the compiler.

Diverts continue the story

Diverts are what link one knot to another, and make your story flow. They’re invisible to the player and are intended to be seamless. They can even happen mid-sentence!

```
=== hurry_home ===
  We hurried home to Savile Row -> as_fast_as_we_could

=== as_fast_as_we_could ===
  as fast as we could.
```

produces:

```
We hurried home to Savile Row as fast as we could.
```

Glue

The above behaviour can also be achieved without the divert being on the same line, but we have to tell the compiler to do it. By default, ink inserts a line break every time a new line of content is found. But content can insist on not having a line-break after it, or before it, by using `<>`, or “glue”.

```
=== hurry_home ===  
  We hurried home <>  
  -> to_savile_row  
  
=== to_savile_row ===  
  to Savile Row  
  -> as_fast_as_we_could  
  
=== as_fast_as_we_could ===  
  <> as fast as we could.
```

produces:

```
We hurried home to Savile Row as fast as we could.
```

The glue marker is invisible to the player, and you can't use too much of it: multiple glues next to each other have no additional effect. (Note there's no way to “negate” a glue; once a line is sticky, it'll stick.)

5. Branching The Flow

Basic branching

Combining knots, options and diverts gives us the basic structure of a choose-your-own game.

```
=== paragraph_1 ===  
    You stand by the wall of Analand, sword in hand.  
  
    * [Open the gate]    -> paragraph_2  
    * [Smash down the gate]    -> paragraph_3  
    * [Turn back and go home]    -> paragraph_4  
  
=== paragraph_2 ===  
    You open the gate, and step out onto the path.  
    ...
```

Using diverts, the writer can branch the flow, and join it back up again, without showing the player that the flow has re-joined.

```
=== back_in_london ===  
    We arrived into London at 9.45pm exactly.  
  
    *   "There is not a moment to lose!"[] I declared.  
        -> hurry_outside  
  
    *   "Monsieur, let us savour this moment!"[] I declared.  
        My master clouted me firmly around the head and  
        dragged me out of the door.  
        -> dragged_outside  
  
    *   [We hurried home] -> hurry_outside  
  
=== hurry_outside ===  
    We hurried home to Savile Row -> as_fast_as_we_could  
  
=== dragged_outside ===  
    He insisted that we hurried home to Savile Row
```

```
-> as_fast_as_we_could
```

```
=== as_fast_as_we_could ===  
<> as fast as we could.
```

The story flow

Knots and diverts combine to create the basic story flow of the game. This flow is “flat” – there’s no call-stack, and diverts aren’t “returned” from.

In most ink scripts, the story flow starts at the top, bounces around in a spaghetti-like mess, and eventually, hopefully, reaches the `-> END`.

The very loose structure means writers can get on and write, branching and rejoining without worrying about the structure that they’re creating as they go. There’s no boiler-plate to creating new branches or diversions, and no need to track any state.

Advanced: Loops

You absolutely can use diverts to create looped content, and ink has several features to exploit this, including ways to make the content vary itself, and ways to control how often options can be chosen.

See the sections on [Variable Text](#) and [Conditional Choices](#) for more information.

Oh, and by the way, the following is legal and not a great idea:

```
=== round ===  
and  
-> round
```

If ink does get stuck in an infinite loop, you’ll need to crash the game to get out of it. In inky, however, the compiler is still watching for text changes to recompile even while it’s locked in a loop, which means if you can break your script, you can break the loop. The fastest way to do that is to type a `~` on a blank line. As soon as the compiler finds it, it’ll crash for reasons we’ll come on to later, and then you can go in and remove your infinite loop.

6. Includes and Stitches

As stories get longer, they become more confusing to keep organised without some additional structure. The classic solution to this for interactive designers is the flow-chart, but here at *inkle* we don't really believe in them: they make even simple things look complicated, and they make complicated things look absolutely appalling. *inkle* is very much about flat text scripts, with section labelling and good filenames being essential for keeping things organised.

Stitches divide knots

Knots can be divided up into sub-sections called “stitches”. These are marked using a single equals sign.

```
=== the_orient_express ===  
  = in_first_class  
  ...  
  = in_third_class  
  ...  
  = in_the_guards_van  
  ...  
  = missed_the_train  
  ...
```

One could use a knot for a scene, for instance, and stitches for the events within the scene.

Stitches have unique names

A stitch can be diverted to using its “address”, given as `knot.stitch`.

```
* [Travel in third class]  
  -> the_orient_express.in_third_class  
  
* [Travel in the guard's van]  
  -> the_orient_express.in_the_guards_van
```

The first stitch is the default

Diverting to a knot which contains stitches will divert to the first stitch in the knot. So:

```
* [Travel in first class]
  "First class, Monsieur. Where else?"
  -> the_orient_express
```

is the same as:

```
* [Travel in first class]
  "First class, Monsieur. Where else?"
  -> the_orient_express.in_first_class
```

(...unless we didn't put first-class first inside the knot. How unseemly!)

You can also include content at the top of a knot outside of any stitch. But you'll need to remember to divert out of it – the engine *won't* automatically enter the first stitch once it's worked its way through the header content.

```
=== the_orient_express ===
  We boarded the train, but where?
  * [First class] -> in_first_class
  * [Second class] -> in_second_class
  * [Third class] -> in_third_class

= in_first_class
  ...
= in_second_class
  ...
= in_third_class
  ...
```

Local diverts

From inside a knot, you don't need to use the full address for a stitch. When ink encounters a divert, it'll look in the most local context first for somewhere to go.

```
-> the_orient_express

=== the_orient_express ===
= in_first_class
  I settled my master.
```

```
*    [Move to third class]
    -> in_third_class

= in_third_class
    I put myself in third.
```

This means that while stitches and knots can't share names, several knots can contain the same stitch name. (So both the *Orient Express* and the *SS Mongolia* can have a first class.)

The compiler will warn you if ambiguous names are used.

Including multiple script files

You can also split your content across multiple files, using an `INCLUDE` statement.

```
INCLUDE newspaper.ink
INCLUDE cities/vienna.ink
INCLUDE journeys/orient_express.ink
```

Include statements should always go at the top of a file, and not inside knots, and in general it's sensible to put all your includes in your top-level “main” ink file, rather than hiding them away in sub-files. When you create an include file using the button for it in inky, this is exactly what it'll do.

There are no rules about file structure in ink. Include files exist purely for the human's benefit: since everything is global in ink, ink doesn't care about the structure you use. (Note to coders: that means you'll never want to include the same include file twice.)

7. Varying Choices

Choices can only be used once

By default, every choice in the game can only be chosen once, but if you don't have loops in your story, you'll never notice this behaviour.

If you do use loops, however, you'll quickly notice your options disappearing...

```
=== find_help ===  
  You search desperately for a friendly face in the  
  crowd.  
  *   The woman in the hat[?] pushes you roughly aside. -  
> find_help  
  *   The man with the briefcase[?] looks disgusted as  
  you stumble past him. -> find_help
```

produces:

```
You search desperately for a friendly face in the crowd.  
  
1: The woman in the hat?  
2: The man with the briefcase?  
  
> 1  
The woman in the hat pushes you roughly aside.  
You search desperately for a friendly face in the crowd.  
  
1: The man with the briefcase?  
  
>
```

... and on the next loop you'll have no options left.

Fallback choices

The above example stops where it does because the next choice ends up in an “out of content” run-time error. There are no choices left to offer, and no content for the player to read!

```
> 1
The man with the briefcase looks disgusted as you stumble
past him.
You search desperately for a friendly face in the crowd.

Runtime error in tests/test.ink line 6: ran out of
content. Do you need a '-> DONE' or '-> END'?
```

We can resolve this with a “fallback choice”. Fallback choices are never displayed to the player, but are ‘chosen’ by the game if no other options exist.

A fallback choice is simply written as a choice without any choice text:

```
*    -> out_of_options
```

And, in a slight abuse of syntax, we can give a fallback choice some content:

```
*    ->
    Mulder never could properly explain how he got out
of that burning box car.
    -> season_3
```

Example of a fallback choice

Adding this into the previous example gives us:

```
=== find_help ===
    You search desperately for a friendly face in the
crowd.
    *    The woman in the hat[?] pushes you roughly aside.
-> find_help
    *    The man with the briefcase[?] looks disgusted as
you stumble past him. -> find_help
    *    ->
        But it is too late: you collapse onto the station
platform. This is the end.
        -> END
```

and produces:

You search desperately for a friendly face in the crowd.

1: The woman in the hat?

2: The man with the briefcase?

> 1

The woman in the hat pushes you roughly aside.

You search desperately for a friendly face in the crowd.

1: The man with the briefcase?

> 1

The man with the briefcase looks disgusted as you stumble past him.

You search desperately for a friendly face in the crowd.

But it is too late: you collapse onto the station platform. This is the end.

Sticky choices

The ‘once-only’ behaviour of a choice is not always what we want, of course, so we have a second kind of choice: the “sticky” choice. A sticky choice is simply one that doesn't get used up, and is marked by a + bullet. (A “splatted” asterisk.)

```
=== homers_couch ===
+   [Eat another donut]
    You eat another donut. -> homers_couch
*   [Get off the couch]
    You struggle up off the couch to go and compose
    epic poetry.
    -> END
```

Fallback choices can be sticky too.

```
=== conversation_loop
*   [Talk about the weather] ->
chat_weather
*   [Talk about the children] ->
chat_children
*   [Talk about the impermanence of all things] ->
```

```
chat_philosophy
*   [Talk about minor skin conditions]           ->
chat_skin_conditions
+   -> sit_in_silence_again
```

Conditional choices

In an ink story, what has happened, has happened, and past events should affect future ones. In practice, this means we need to be able to turn off choices that don't fit the current playthrough, and turn on new ones that do. ink has quite a broad suite of logic available to the author to use, but the very simplest test there is asks, “has the player seen a particular piece of content?”

Every knot/stitch in the game has a unique address (so it can be diverted to), and we can use that same address directly to test if that piece of content has been seen in this playthrough.

```
*   { not visit_paris }           [Go to Paris]
-> visit_paris
+   { visit_paris }               [Return to Paris]
-> visit_paris
*   { visit_paris.met_estelle } [ Phone Mme Estelle ]
-> phone_estelle
```

Note that the test `knot_name` is true if *any* stitch inside that knot has been seen. Note also that conditionals don't override the once-only behaviour of options, so you'll still need sticky options for repeatable choices.

Advanced: multiple conditions

You can use several logical tests on an option; if you do, *all* the tests must all be passed for the option to appear. Conditionals like this can be placed on multiple lines if you like (there can sometimes be quite a lot of them).

```
*   { not visit_paris } [Go to Paris]  -> visit_paris
+   { visit_paris }
    { not bored_of_paris }
    [Return to Paris]  -> visit_paris
```

Logical operators: AND and OR

The above “multiple conditions” are really just conditions combined with the usual programming AND operator. ink supports `and` (also written as `&&`) and `or` (also written as `||`) in the usual way, as well as bracketing.

```
* { not (visit_paris or visit_rome) && (visit_london ||  
visit_new_york) }  
  [ Wait. Go where? I'm confused. ]  
  -> visit_someplace
```

For non-programmers `X` and `Y` means both `X` and `Y` must be true; `X or Y` means either or both; we don't have a `xor`, nor a `xnor` (though it's no one's fault if they `xnor`).

You can also use the standard `!` for `not`, though it'll sometimes confuse the compiler, which thinks `{!text}` is a once-only list with one entry. We recommend using `not` because negated Boolean tests are never that exciting!!!

You'll encounter lots of examples of these operators in the examples in the rest of this book.

Advanced: knot/stitch labels are actually read counts

The test:

```
* {seen_clue} [Accuse Mr Jefferson]
```

is actually testing `seen_clue` as an *integer* and asking “are you not zero?” A knot or stitch label is actually a variable containing the number of times the content at the address has been seen by the player in this game.

If it's non-zero, it'll return true in a test like the one above, but you can also be more specific as well:

```
* {seen_clue > 3} [Flat-out arrest red-handed Mr  
Jefferson]
```

Advanced: more logic

For more on logic and conditionality, see the section on [variables and logic](#).

8. Variable Text

Text can vary

So far, all the content we've seen has been static, fixed pieces of text. But content can also vary at the moment of being printed.

Sequences, cycles, and other alternatives

The simplest variations of text are provided blocks of possible *alternatives*, selected using one of several possible rules. Alternatives like this are written inside `{...}` curly brackets, with elements separated by `|` symbols (vertical divider lines – these appear on most keyboards despite being typographically useless. UNTIL NOW. Also, in code.)

Types of alternatives

Sequences:

A sequence (or a “stopping block”) is a set of alternatives that tracks how many times it's been seen, and each time, shows the next element along. When it runs out of new content it continues to show the final element.

This is the default in ink, so it requires no markup beyond the braces.

```
The radio hissed into life. {"Three!"|"Two!"|"One!"|There  
was the white noise racket of an explosion.|But it was  
just static.}
```

```
{I bought a coffee with my five-pound note.|I bought a  
second coffee for my friend.|I didn't have enough money  
to buy any more coffee.}
```

Cycles (marked with a `&`):

Cycles are like sequences, but instead of stopping on the last element, they loop their content.

```
It was  
{&Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunda  
y} today.
```

Once-only (marked with a `!`):

Once-only alternatives are like sequences, but when they run out of new content to display, they display nothing. (You can think of a once-only replacement as a

sequence with a blank last entry.)

```
He told me a joke. {!I laughed politely.|I smiled.|I  
grimaced.|I promised myself to not react again.}
```

Shuffles (marked with a ~):

Shuffles produce randomised output. In cases of two alternatives, they choose randomly each time.

```
I tossed the coin. {~Heads|Tails}.
```

In longer shuffles, they operate a shuffle algorithm, working through the full list in random order before resetting and doing so again.

```
My favourite color today is {~blue|red|green|orange}.
```

Note that a shuffle like this *can* produce, say, two consecutive orange days, should orange day fall at the end of one shuffle and the start of a next. But, thankfully, it can't do three.

Features of Alternatives

Alternatives can contain blank elements. The following does nothing for several turns, and then scares the player silly.

```
I took a step forward. {!|||Then the lights went out. -> eek}
```

Alternatives can be nested, with the only limit being your ability to work out what on earth is going on.

```
The Ratbear {&{wastes no time and |}swipes|scratches}  
{&at you|into your {&leg|arm|cheek}}.
```

Alternatives can include divert statements, so they aren't just superficial!

```
I {waited.|waited some more.|snoozed.|woke up and waited  
more.|gave up and left. -> leave_post_office}
```

Examples

Alternatives can be used inside loops to create the appearance of intelligent, state-tracking gameplay without particular effort.

Here's a one-knot version of whack-a-mole. Note we use once-only options, and a fallback, to ensure the mole doesn't move around, and the game will always end.

```
=== whack_a_mole ===
  {I heft the hammer.|{~Missed!|Nothing!|No good. Where
is he?|Ah-ha! Got him! -> END}}
  The {&mole|{&nasty|blasted|foul} {&creature|rodent}} is
{in here somewhere|hiding somewhere|still at
large|laughing at me|still unwhacked|doomed}. <>
  {!I'll show him!|But this time he won't escape!}
  * [{&Hit|Smash|Try} top-left] -> whack_a_mole
  * [{&Whallop|Splat|Whack} top-right] -> whack_a_mole
  * [{&Blast|Hammer} middle] -> whack_a_mole
  * [{&Clobber|Bosh} bottom-left] -> whack_a_mole
  * [{&Nail|Thump} bottom-right] -> whack_a_mole
  * ->
    Then you collapse from hunger. The mole has
defeated you!
    -> END
```

produces the following fun-filled rodent-splatting adventure:

```
I heft the hammer.
The mole is in here somewhere. I'll show him!

1: Hit top-left
2: Whallop top-right
3: Blast middle
4: Clobber bottom-left
5: Nail bottom-right

> 1
Missed!
The nasty creature is hiding somewhere. But this time he
won't escape!

1: Splat top-right
2: Hammer middle
3: Bosh bottom-left
```

```
4: Thump bottom-right
```

```
> 4
```

```
Nothing!
```

```
The mole is still at large.
```

```
1: Whack top-right
```

```
2: Blast middle
```

```
3: Clobber bottom-left
```

```
> 2
```

```
Ah-ha! Got him!
```

And here's a bit of unasked-for lifestyle advice. Note the sticky choice – the lure of the television will never fade:

```
=== turn_on_television ===
```

```
    I turned on the television {for the first time|for the
second time|again|once more}, but there was {nothing good
on, so I turned it off again|still nothing worth
watching|even less to hold my interest than
before|nothing but rubbish|a program about sharks and I
don't like sharks|nothing on}.
```

```
    +   [Try it again]  -> turn_on_television
```

```
    *   [Go outside instead]  -> go_outside_instead
```

```
=== go_outside_instead ===
```

```
    -> END
```

Advanced: Alternatives in choices

Alternatives can also be used inside choice text:

```
    +   "Hello, {&Master|Monsieur Fogg|you|brown-eyes}!"[]
I declared.
```

But there's a caveat; you can't start an option's text with a `{`, as it'll look like a conditional. But then, the caveat has a caveat: if you escape a whitespace `\` before your `{`, then ink will recognise it as text. So

```
    + \ {&Hello|Hi|Wotcha}!
```

will work as expected...

... only it won't, because choice text is *re-evaluated* when it is included in the output, meaning the sequence won't give you the same text as the choice. So using a sequence in a choice text usually doesn't do what you want it to do.

Sneak Preview: Multiline alternatives

ink has another format for making alternatives of varying content blocks that span several lines of text. See the section on [multiline blocks](#) for details.

Conditional text

Text can also vary depending on logical tests, just as options can. The format is `{test: text if true }` or `{test: text if true | text if false }`.

```
{met_blofeld: "I saw him. Only for a moment." }
```

or

```
"His real name was { met_blofeld.learned_his_name :  
Franz|a secret}."
```

They can be nested, the same as other alternatives, so that:

```
{met_blofeld: "I saw him. Only for a moment. His real  
name was {met_blofeld.learned_his_name: Franz|kept a  
secret}." | "I missed him. Was he particularly evil?" }
```

can produce either:

```
"I saw him. Only for a moment. His real name was Franz."
```

or:

```
"I saw him. Only for a moment. His real name was kept a  
secret."
```

or:

```
"I missed him. Was he particularly evil?"
```

9. Game Queries and Functions

`ink` provides a few useful ‘game level’ queries about game state, for use in conditional logic. They're not quite parts of the language, but they're always available and they can't be edited by the author. In a sense, they're the “standard library functions” of the language.

The convention is to name these in capital letters.

CHOICE_COUNT()

`CHOICE_COUNT` returns the number of options created so far in the current chunk. So for instance:

```
* {false} Option A
* {true} Option B
* {CHOICE_COUNT() == 1} Option C
```

produces two options, B and C. This can be useful for limiting how many options a player gets on a turn.

TURNS()

This returns the number of game turns since the game began.

TURNS_SINCE(-> knot)

`TURNS_SINCE` returns the number of moves (formally, player inputs) since a particular knot/stitch was last visited.

A value of 0 means “was seen as part of the current chunk”. A value of -1 means “has never been seen”. Any other positive value means it has been seen that many turns ago.

```
* {TURNS_SINCE(-> sleeping.intro) > 10} You're
feeling tired.
* {TURNS_SINCE(-> laugh) == 0} You try to stop
laughing.
```

Note that the parameter passed to `TURNS_SINCE` is a “divert target”, not simply the knot address itself (because the knot address is a number - the read count - not a location in the story...)

Sneak preview: using `TURNS_SINCE` in a function

The `URNS_SINCE(->x) == 0` test is so useful it's often worth wrapping it up as a function.

```
=== function came_from(-> x)
  ~ return URNS_SINCE(x) == 0
```

The chapter on [functions](#) outlines the syntax here a bit more clearly but the above allows you to say things like:

```
* {came_from(-> nice_welcome)} 'I'm happy to be here!'
* {came_from(-> nasty_welcome)} 'Let's keep this
quick.'
```

... and have the game react to content the player saw *just now*.

SEED_RANDOM()

For testing purposes, it's often useful to fix the random number generator so ink will produce the same outcomes every time you play. You can do this by “seeding” the random number system.

```
~ SEED_RANDOM(235)
```

The number you pass to the seed function is arbitrary, but providing different seeds will result in different sequences of outcomes.

Note that seeding can also be useful in non-debug contexts too – in *Overboard!* we re-seed the random generator every time the player begins the card game, using the number of times the player has visited that location as a seed, to prevent people quitting and reloading their save to get better cards.

Advanced: more queries

You can make your own ‘game level’ queries yourself by using “external functions”, which cause ink to pass the query up into the game code and use the result given. There’s a little more syntax involved for these: see the section on chapter on [external functions](#) for more information.

10. Comments?

By default, all text in your file will appear in the output content, unless marked up as a conditional, divert or a knot-label. But there's also some mark-up for the writer to use to keep track of their story.

Comments and TODOs

The simplest mark-up is a comment. ink supports two kinds of comment. There's the kind used for someone reading the code, which the compiler ignores completely. It comes in two flavours, a single line-comment marked by `//`, and a comment sections between a `/*` and a `*/`:

```
"What do you make of this?" she asked.  
  
// Something unprintable...  
"I couldn't possibly comment," I replied.  
  
/*  
... or an unlimited block of text  
*/
```

Secondly, there's a kind of comment for reminding the author what they need to do. The compiler indexes these, and inky keeps a list in the top banner.

```
TODO: Write this section properly!
```

Tags

A third kind of mark-up is a tag. These are comments the game engine doesn't ignore – in fact, they're comments designed to be read and used by the engine as the game is played. But inky doesn't care about them, so you'll have to wait for [Running Your Ink](#) for [more information](#).

```
A line of normal game-text. #blue
```


Weave

So far, we've been building branched stories in the simplest way, with “options” that link to “pages” like a paper choose-your-own book. But this has a heavy overhead: it requires us to uniquely name every destination in the story and spread everything out, which can slow down writing and discourage minor branching. Often branching stories don't really want to branch at every choice. Sometimes a branch is a minor detour, sometimes it's purely one line of varying colour.

`ink` has a much more powerful syntax designed specifically for simplifying story flows which have an always-forwards direction (as most stories do, and most computer programs don't). This format is called “weave”, and it's built out of the basic content/option syntax with two new features: the humble gather mark, `-`, and the nesting of choices and gathers.

1. Gathers

Gather points gather the flow

Let's go back to the first multi-choice example at the start of this book.

```
"What's that?" my master asked.  
* "I am somewhat tired[.]," I repeated.  
  "Really," he responded. "How deleterious."  
* "Nothing, Monsieur!"[] I replied.  
* "I said, this journey is appalling[.]" and I want no  
more of it."  
  "Ah," he replied, not unkindly. "I see you are  
frustrated."
```

In a real game, all three of these options might well lead to the same conclusion – Monsieur Fogg leaves the room. We can do this using a *gather*, without the need to create any new knots or add any diverts.

```
"What's that?" my master asked.  
* "I am somewhat tired[.]," I repeated.  
  "Really," he responded. "How deleterious."  
* "Nothing, Monsieur!"[] I replied.  
  "Very good, then."  
* "I said, this journey is appalling[.]" and I want  
no more of it."  
  "Ah," he replied, not unkindly. "I see you are  
frustrated."  
  
- With that, Monsieur Fogg left the room.
```

That single `-` at the bottom tells ink this is a point that collects up all the flows above it, and it produces the following playthrough:

```
"What's that?" my master asked.  
  
1: "I am somewhat tired."  
2: "Nothing, Monsieur!"  
3: "I said, this journey is appalling."
```

```
> 1
"I am somewhat tired," I repeated.
"Really," he responded. "How deleterious."
With that, Monsieur Fogg left the room.
```

Options and gathers form chains of content

We can string these gather-and-branch sections together to make branchy sequences that always run forwards.

```
=== escape ===
  I ran through the forest, the dogs snapping at my
  heels.

  * I checked the jewels[] were still in my pocket,
  and the feel of them brought a spring to my step. <>

  * I did not pause for breath[] but kept on running.
  <>

  * I cheered with joy. <>

  - The road could not be much further! Mackie would
  have the engine running, and then I'd be safe.

  * I reached the road and looked about[]. And would
  you believe it?
  * I should interrupt to say Mackie is normally very
  reliable[]. He's never once let me down. Or rather, never
  once, previously to that night.

  - The road was empty. Mackie was nowhere to be seen.
```

This is the most basic kind of weave. The rest of this section details additional features that allow weaves to nest, contain side-tracks and diversions, divert within themselves, and use earlier choices to influence later ones.

The weave philosophy

Weaves are more than just a convenient encapsulation of branching flow; they're also a way to author more robust content. The `escape` example above has already

six possible routes through it, and a more complex sequence might have lots and lots more. Using normal diverts, one has to check the links by chasing the diverts from point to point and it's easy for errors to creep in.

With a weave, the flow is *guaranteed* to start at the top and “fall” to the bottom. Flow errors are impossible in a basic weave structure, and the output text can be easily skim read. That means there's no need to actually test all the branches in game to be sure they work as intended. (There may *continuity* errors, but these are an order of magnitude less serious than the flow collapsing or ending up in the wrong place.)

Weaves also allow for easy redrafting of choice-points; in particular, it's easy to break a sentence up and insert additional choices for variety or pacing reasons, without having to re-engineer any of the flow before or after.

2. Nested Flow

The weaves shown above are simple, dropdown structures. Whatever the player does, they take the same number of turns to get from top to bottom. However, sometimes certain choices warrant a bit more depth or complexity.

For that, we allow weaves to nest.

This section comes with a warning. Nested weaves are very powerful and very compact, but they can take a bit of getting used to!

Options can be nested

Consider the following scene:

```
- "Well, Poirot? Murder or suicide?"
  * "Murder!"
  * "Suicide!"
- Ms. Christie lowered her manuscript a moment. The rest
  of the writing group sat, open-mouthed.
```

The first choice presented is “Murder!” or “Suicide!”. If Poirot declares a suicide, there's no more to do, but in the case of murder, there's a follow-up question needed – whom does he suspect?

We can add new options via a set of nested sub-choices. We tell the script that these new choices are “part of” another choice by using two asterisks, instead of just one.

```
- "Well, Poirot? Murder or suicide?"
  * "Murder!"
  "And who did it?"
  * * "Detective-Inspector Japp!"
  * * "Captain Hastings!"
  * * "Myself!"
  * "Suicide!"
- Mrs. Christie lowered her manuscript a moment. The rest
  of the writing group sat, open-mouthed.
```

(Note that it's good style to also indent the lines to show the nesting, but the compiler doesn't mind.)

Should we want to add new sub-options to the other route, we do that in similar fashion.

```

- "Well, Poirot? Murder or suicide?"
  * "Murder!"
    "And who did it?"
    * * "Detective-Inspector Japp!"
    * * "Captain Hastings!"
    * * "Myself!"
  * "Suicide!"
    "Really, Poirot? Are you quite sure?"
    * * "Quite sure."
    * * "It is perfectly obvious."
- Mrs. Christie lowered her manuscript a moment. The
  rest of the writing group sat, open-mouthed.

```

Now, that initial choice of accusation will lead to specific follow-up questions – but either way, the flow will still come back together at the gather point, for Mrs. Christie's cameo appearance.

But what if we want a more extended sub-scene?

Gather points can be nested too

Sometimes, it's not a question of expanding the number of options, but having more than one additional beat of story. We can do this by nesting gather points as well as options.

```

- "Well, Poirot? Murder or suicide?"
  * "Murder!"
    "And who did it?"
    * * "Detective-Inspector Japp!"
    * * "Captain Hastings!"
    * * "Myself!"
    - - "You must be joking!"
    * * "Mon ami, I am deadly serious."
    * * "If only..."
  * "Suicide!"
    "Really, Poirot? Are you quite sure?"
    * * "Quite sure."
    * * "It is perfectly obvious."
    - - "Well, blimey."
- Mrs. Christie lowered her manuscript a moment. The
  rest of the writing group sat, open-mouthed.

```

If the player chooses the “murder” option, they’ll have two choices in a row on their sub-branch – a whole flat weave, just for them.

Advanced: What gathers do

Gathers are hopefully intuitive, but their behaviour is a little harder to put into words: in general, after an option has been taken, the story finds *the next gather down that isn’t on a more nested level*, and diverts to it.

The basic idea is this: options separate the paths of the story, and gathers bring them back together. (Hence the name, “weave”!)

You can nest as many levels as you like

Above, we used two levels of nesting; the main flow, and the sub-flow. But there’s no limit to how many levels deep you can go.

```
- "Tell us a tale, Captain!"
  * "Very well, you sea-dogs. Here's a tale..."
    * * "It was a dark and stormy night..."
      * * * "...and the crew were restless..."
        * * * * "... and they said to their Captain..."
          * * * * * "...Tell us a tale Captain!"
    * "No, it's past your bed-time."
  - To a man, the crew began to yawn.
```

After a while, this sub-nesting gets hard to read and manipulate, so it’s good practice to divert away to a new stitch if a side-choice goes unwieldy. But, in theory at least, you could write your entire story as a single weave.

Example: a conversation with nested nodes

Here's a longer example:

```
- I looked at Monsieur Fogg
  * ... and I could contain myself no longer[].
    'What is the purpose of our journey, Monsieur?'
    'A wager,' he replied.

    * * 'A wager!'[] I returned.
      He nodded.
      * * * 'But surely that is foolishness!'
      * * * 'A most serious matter then!'
      - - - He nodded again.
```

```

        * * *   'But can we win?'
                'That is what we will endeavour to find
out,' he answered.
        * * *   'A modest wager, I trust?'
                'Twenty thousand pounds,' he replied,
quite flatly.
        * * *   I asked nothing further of him then[.],
and after a final, polite cough, he offered nothing more
to me. <>

        * *       'Ah[.'],' I replied, uncertain what I
thought.

        - -       After that, <>

        *       ... but I said nothing[] and <>
        *       ... and was so overcome I stared at my shoes[], and
in this manner, <>
- we passed the day in silence.

```

with a couple of possible playthroughs. A short one:

```

I looked at Monsieur Fogg

1: ... and I could contain myself no longer
2: ... but I said nothing
3: ... and was so overcome I stared at my shoes

> 2
... but I said nothing and we passed the day in silence.

```

and a longer one:

```

I looked at Monsieur Fogg

1: ... and I could contain myself no longer
2: ... but I said nothing
3: ... and was so overcome I stared at my shoes

> 1

```


... and I could contain myself no longer.
'What is the purpose of our journey, Monsieur?'
'A wager,' he replied.

1: 'A wager!'

2: 'Ah.'

> 1

'A wager!' I returned.
He nodded.

1: 'But surely that is foolishness!'

2: 'A most serious matter then!'

> 2

'A most serious matter then!'
He nodded again.

1: 'But can we win?'

2: 'A modest wager, I trust?'

3: I asked nothing further of him then.

> 2

'A modest wager, I trust?'

'Twenty thousand pounds,' he replied, quite flatly.
After that, we passed the day in silence.

Hopefully, this demonstrates the philosophy laid out above: that weaves offer a compact way to offer a lot of branching, a lot of choices, but with the guarantee of getting from beginning to end!

3. Tracking a Weave

Sometimes, the weave structure is sufficient. When it's not, we need a bit more control.

Weaves are largely unaddressed

By default, lines of content in a weave don't have an address or label, which means they can't be diverted to and they can't be tested for. In the most basic weave structure, choices vary the path the player takes through the weave and what they see, but once the weave is finished those choices and that path are forgotten.

But should we want to remember what the player has seen, we can – we add in labels where they're needed using the `(label_name)` syntax.

Gathers and options can be labelled

Gather points at any nested level can be labelled using brackets.

```
- (top_bit)
```

```
- - (middle_bit)
```

Once labelled, gather points can be diverted to, or tested for in conditionals, just like knots and stitches. This means you can use previous decisions to alter later outcomes inside the weave, while still keeping all the advantages of a clear, reliable forward-flow.

Options can also be labelled, just like gather points, using brackets. Label brackets come before conditions in the line.

```
* (insult_the_guard) { bravery >= 5 } "Oi! Lug-head!"
```

These addresses can be used in conditional tests, which can be useful for creating options unlocked by other options.

```
=== meet_guard ===  
The guard frowns at you.  
  
* (greet) [Greet him] 'Greetings.'  
* (get_out) 'Get out of my way[.'],' you tell the  
guard.
```

```

- 'Hmm,' replies the guard.

  *    {greet} 'Having a nice day?' // only if you greeted
him
  *    'Hmm?'[] you reply.
  *    {get_out} [Shove him aside] // only if you
threatened him
      You shove him sharply.
      -> fight_guard          // we're off to have a fight
now
- 'Mff,' the guard replies, offering a paper bag.
'Toffee?'

```

Scope

Inside the same stitch, you can use a label name directly – it's the local address of a gather or option. From outside the block you need to provide a more detailed address name, either to a different stitch within the same knot:

```

=== knot ===
= stitch_one
- (gatherpoint) Some content.
= stitch_two
  *    {stitch_one.gatherpoint} Option

```

... or pointing into another knot:

```

=== knot_one ===
  *    (option_one) {knot_two.stitch_two.gather_two} Option

=== knot_two ===
= stitch_two

- (gather_two)
  *    {knot_one.option_one } Option

```

Advanced: all options can be labelled

In truth, all content in ink is a weave, even if there are no gathers in sight, and you can label *any* option in the game with a bracket label and then reference it using the addressing syntax.

```
=== fight_guard ===
...

= throw_something
    *   (rock) [Throw rock at guard] -> throw
    *   (sand) [Throw sand at guard] -> throw

= throw
    You hurl {throw_something.rock:a rock|a handful of
sand} at the guard.
```

Advanced: Loops in a weave

Labelling gathers allows us to neatly create loops inside weaves. Here's a standard pattern for asking questions of an NPC.

```
- (opts)
    *   'Can I get a uniform from somewhere?'[] you ask
the cheerful guard.
        'Sure. In the locker.' He grins. 'Don't think
it'll fit you, though.'
    *   'Tell me about the security system.'
        'It's ancient,' the guard assures you. 'Old as
coal.'
    *   'Are there dogs?'
        'Hundreds,' the guard answers, with a toothy grin.
'Hungry devils, too.'
        // We require the player to ask at least one
question here
    *   {loop} [Enough talking]
        -> done
- (loop)
    // loop a few times before the guard gets bored
    { -> opts | -> opts | }
    He scratches his head.
    'Well, can't stand around talking all day,' he
declares.
```

```
- (done)
  You thank the guard, and move away.
```

Advanced: diverting to options

Options can also be diverted to: the divert goes to the output of having chosen that choice *as though the choice had been chosen*. So the content printed will ignore square bracketed text, and if the option is once-only, it will be marked as used up.

```
- (opts)
  * [Pull a face]
    You pull a face, and the soldier comes at you! ->
shove
  * (shove) [Shove the guard aside]
    You shove the guard to one side, but he comes back
swinging.
  * {shove} [Grapple and fight] -> fight_the_guard
- -> opts
```

produces:

```
1: Pull a face
2: Shove the guard aside

> 1
You pull a face, and the soldier comes at you! You shove
the guard to one side, but he comes back swinging.

1: Grapple and fight
>
```

Advanced: Gathers directly after an option

The following is valid, and frequently useful.

```
* "Are you quite well, Monsieur?"[] I asked.
- - (quite_well) "Quite well," he replied.
* "How did you do at the crossword, Monsieur?"[] I
asked.
```

```
-> quite_well
*   I said nothing[] and neither did my Master.
-   We fell into companionable silence once more.
```

Note the second-level gather point `quite_well` directly below the first option: there's nothing to gather here, really, but it gives us a handy place to divert the second option to, so we can write fewer lines while still conveying Fogg's character.

Variables and Logic

So far we've written conditional text, and conditional choices, using tests based on what content the player has seen so far.

ink also supports variables, both temporary and global, for storing numerical and content data, and even story flow commands. The language is fully-featured in terms of logic, strongly-featured for mathematics, and contains a few additional structures to help keep the often complex logic of a branching story better organised.

That said, you can write a pretty good adaptive story without ever reading another page of this manual if you prefer. If you take that route, goodbye! May this book act as a good object on which to rest your coffee cup. If, however, you wish to press onwards, please note that things will become more a little more programmery from here.

1. Global Variables

The most powerful kind of variable, and arguably the most useful for a story, is a variable to store some unique property about the state of the game – anything from the amount of money in the protagonist's pocket, to a value representing the protagonist's state of mind.

This kind of variable is called “global” because it can be fully accessed from anywhere in the story – it can be both set, and read, at any time. (Traditionally, programming tries to avoid this kind of thing, as it allows one part of a program to mess with another, unrelated part. But a story is a story, and stories are all about consequences: what happens in Vegas rarely stays there.)

Defining global variables

Global variables can be defined anywhere, via a `VAR` statement. They should be given an initial value, which defines what type of variable they are - integer, floating point (decimal), content, or a story address.

```
VAR knowledge_of_the_cure = false
VAR players_name = "Emilia"
VAR number_of_infected_people = 521
VAR current_epilogue = -> they_all_die_of_the_plague
```

Using global variables

We can test global variables to control options, and provide conditional text, in a similar way to what we have previously seen.

```
=== the_train ===
    The train jolted and rattled. { mood > 0:I was feeling
positive enough, however, and did not mind the odd
bump|It was more than I could bear}.
    *   { not_knows_about_wager } 'But, Monsieur, why are
we travelling?'[] I asked.
    *   { knows_about_wager } I contemplated our strange
adventure[]. Would it be possible?
```

Advanced: storing diverts as variables

A “divert” statement is actually a type of value in itself, and can be stored, altered, and diverted to.


```
VAR current_epilogue = -> everybody_dies

=== continue_or_quit ===
  Give up now, or keep trying to save your Kingdom?
  * [Keep trying!]    -> more_hopeless_introspection
  * [Give up]         -> current_epilogue
```

Advanced: Global variables are externally visible

Global variables can be accessed, and altered, from the runtime as well from the story, so they provide a good way to for the game to snoop on what's going on in the story.

The ink layer can also be a good place to store wider gameplay variables: saving and loading is handled for you and the story itself can react to the current values.

Printing variables

The value of a variable can be printed out as content using an inline syntax similar to sequences and conditional text:

```
VAR friendly_name_of_player = "Jackie"
VAR age = 23

My name is Jean Passepartout, but my friend's call me
{friendly_name_of_player}. I'm {age} years old.
```

This can be useful in debugging as well as in the normal course of the story. (And for more complex printing based on logic and variables, see the chapter on [functions](#).)

Evaluating strings

It might be noticed that above we referred to variables as being able to contain “content”, rather than “strings”. That was deliberate, because a string defined in ink can contain ink – although it will always evaluate to a string. (Yikes!)

```
VAR a_colour = ""

~ a_colour = "{~red|blue|green|yellow}"

{a_colour}
```

... produces one of red, blue, green or yellow.

Note that once a piece of content like this is evaluated, its value is “sticky”. (The quantum state collapses.) So the following:

```
The goon hits you, and sparks fly before your eyes,  
{a_colour} and {a_colour}.
```

... won't produce a very interesting effect. (If you really want this to work, use a text function to print the colour!)

This is also why the initial value of a string can't contain logic, so:

```
VAR a_colour = "{~red|blue|green|yellow}"
```

is explicitly disallowed; it would be evaluated on the construction of the story, which probably isn't what you want.

2. Logic

Assignment

Obviously, global variables are not intended to be constant, so we need a syntax for altering them.

By default, any text in an ink script is printed out directly; so we use a markup symbol, `~`, to indicate when a line of content is actually intended to be doing some numerical work.

The following statements all assign values to variables, doing more or less work along the way:

```
=== set_some_variables ===  
~ knows_about_wager = true  
~ x = (x * x) - (y * y) + c  
~ y = 2 * x * y
```

Mathematics

ink can do maths, and perform basic mathematical tests. The following tests various numerical conditions:

```
{ x == 1.2 }  
{ x != 0 }  
{ x / 2 > 4 }  
{ y - 1 <= x * x }
```

Alongside the core mathematical operations (+, -, * and /), ink supports % (or mod), returning the remainder after integer division (so `12 mod 5 == 2`). There's also `MIN(a, b)` and `MAX(c, d)`, and `POW` for doing to-the-power-of:

```
{MIN(7, -3)} is -3.  
  
{POW(3, 2)} is 9.  
{POW(16, 0.5)} is 4. // power of 0.5 is square root
```

Operations can be nested using brackets in the normal way:

```
{ (MIN(POW(x, 2), POW(y, 3))) <= MAX(k mod 3, (4 * k) mod 5) }
```

If more complex operations are required, one can write functions (using recursion if necessary), or call out to external, game-code functions (for anything more advanced). There's a [section on this](#) coming up.

Increment and decrement

ink supports increment and decrement shorthand for addition and subtraction only.

```
~ x ++ // means ~ x = x + 1
~ x -- // means ~ x = x - 1
~ x += 3 // means ~ x = x + 3
~ x -= 5 // means ~ x = x - 5
```

RANDOM(min, max)

ink can generate random integers if required using the RANDOM function. RANDOM is authored to be like a dice (yes, pendants, we said *a dice*), so the min and max values are both inclusive.

```
~ temp dice_roll = RANDOM(1, 6)

~ temp lazy_grading_for_test_paper = RANDOM(30, 75)

~ temp number_of_heads_the_serpent_has = RANDOM(3, 8)
```

Recall that the random number generator can be seeded for testing purposes using SEED_RANDOM() (as detailed in Game Queries and Functions section above).

Advanced: numerical types are implicit

Results of operations – in particular, for division – are typed based on the type of the input. So while floating point division returns floating point results, integer division returns integer results. (This is usually really unexpected and annoying.)

```
~ x = 2 / 3 // => x = 0
~ y = 7 / 3 // => y = 2
~ z = 1.2 / 0.5 // => z = 2.4
```

Advanced: INT(), FLOOR() and FLOAT()

In cases where you don't want implicit types, you want to force a decimal division, or you want to round off a variable, you can cast it directly.

```
{INT(3.2)} is 3.  
{FLOOR(4.8)} is 4.  
{INT(-4.8)} is -4.  
{FLOOR(-4.8)} is -5.  
  
{FLOAT(4)} is, um, still 4.  
{FLOAT(2) / 3} is 0.666667. // nobody's perfect
```

`FLOOR` returns the highest integer less than or equal to the given number. `INT` returns the integer part. `FLOAT` returns the same value, but as a floating point number.

Example: generating random floats

`ink` doesn't have a method for generating a random floating-point number, and since `RANDOM()` returns an integer, the following will always produce 0.

```
RANDOM(1, 10000) / 10000
```

To resolve this, you need to force the division into floating point, either properly:

```
FLOAT(RANDOM(1, 10000)) / 10000
```

or hackily:

```
RANDOM(1, 10000) / 9999.9999
```

String handling

Oddly enough for a text-engine, `ink` doesn't have much in the way of string-handling: it's assumed that any string conversion you need to do will be handled by the game code (and perhaps by external functions – see the chapter in *Running Your Ink* for an example!.)

But you can do some basic string operations and queries.

Concatenation

Strings can be concatenated, either directly:

```
~ name = first_name + " " + second_name
```

Or more powerfully, by using the fact that strings are actually ink:

```
~ name = "{first_name} {second_name}"  
~ magician_name = "the {~marvellous|mysterious}  
{second_name}"
```

The following is valid:

```
~ surname += "Darcy"
```

The following isn't:

```
~ surname -= "Bennett"
```

String queries

ink support four string queries – equality, inequality, substring (which we call ? for reasons that will become clear in a later chapter) and inverse substring (inexplicably called !?).

The following will all return true:

```
{ "Yes please." == "Yes please." } // eggs is eggs  
{ "No." != "Yes, really." }      // no does not mean  
yes  
{ "pirate" ? "irate" }          // pirates are angry  
{ "team" !? "I" }              // there's no I in  
team
```

Example: a or an?

It would be nice to be able to write something like:

```
I put {a("cat")} and {a("ape")} into {a("old box")} with  
{a("elephant")}.
```

... for use in cases where those strings are actually variable: here's an implementation for that.

```
=== function a(x)
  ~ temp stringWithStartMarker = "^" + x
  { stringWithStartMarker ? "^a" or stringWithStartMarker
? "^A" or stringWithStartMarker ? "^e" or
stringWithStartMarker ? "^E" or stringWithStartMarker ?
"^i" or stringWithStartMarker ? "^I" or
stringWithStartMarker ? "^o" or stringWithStartMarker ?
"^O" or stringWithStartMarker ? "^u" or
stringWithStartMarker ? "^U" :
    an {x}
  - else:
    a {x}
}
```

3. Conditional blocks (if/else)

We've seen conditionals used to control options and story content; ink also provides an equivalent of the normal if/else-if/else structure.

A simple if

The if syntax takes its cue from the other conditionals used so far, with the {...} syntax indicating that something is being tested.

```
{ x > 0:  
  ~ y = x - 1  
}
```

Else conditions can be provided:

```
{ x > 0:  
  ~ y = x - 1  
- else:  
  ~ y = x + 1  
}
```

Extended if/else if/else blocks

The above syntax is actually a specific case of a more general structure, something like a “switch” statement of another language:

```
{  
- x == 0:  
  ~ y = 0  
- x > 0:  
  ~ y = x - 1  
- else:  
  ~ y = x + 1  
}
```

(Note, as with everything else, the white-space is purely for readability and has no syntactic meaning.)

Switch blocks

And there's also an actual switch statement, where the value of a variable is used to decide which block to use:

```
{ x:
- 0:    zero
- 1:    one
- 2:    two
- else: lots
}
{ day:
- "Monday": "I hate Mondays."
- "Tuesday": "Tuesdays are okay, I guess."
- "Wednesday": "No one likes Wednesdays, do they?"
- else: "It'll be Monday soon."
}
```

Example: context-relevant content

Note these tests don't have to be variable-based and can use read counts, just as other conditionals can. The following construction is frequent as a way of saying “do some content which is relevant to the current game state”:

```
=== dream ===
{
- visited_snakes && not dream_about_snakes:
  ~ fear++
  -> dream_about_snakes
- visited_poland && not dream_about_polish_beer:
  ~ fear--
  -> dream_about_polish_beer
- else:
  // breakfast-based dreams have no effect
  -> dream_about_marmalade
}
```

The syntax has the advantage of being easy to extend, and prioritise.

Conditional blocks are not limited to logic

Conditional blocks can be used to control story content as well as logic:

```
I stared at Monsieur Fogg.  
{ know_about_wager:  
  <> "But surely you are not serious?" I demanded.  
- else:  
  <> "But there must be a reason for this trip," I  
  observed.  
}  
He said nothing in reply, merely considering his  
newspaper with as much thoroughness as entomologist  
considering his latest pinned addition.
```

You can also put options inside conditional blocks:

```
{ door_open:  
  * I strode out of the compartment[] and I fancied I  
  heard my master quietly tutting to himself.  
  -> go_outside  
- else:  
  * I asked permission to leave[] and Monsieur Fogg  
  looked surprised.  
  -> open_door  
  * I stood and went to open the door[]. Monsieur Fogg  
  seemed untroubled by this small rebellion.  
  -> open_door  
}
```

...but inside a conditional block no gathers are allowed because of the confusion over what `-` might mean, and every option must end in an explicit divert to tell the story flow where to go.

Multiline blocks

There's one other class of multiline block, which expands on the alternatives system from above. The following are all valid and do what you might expect:

```
// Sequence: go through the alternatives, and stick on  
last  
{ stopping:  
  - I entered the casino.
```

```

- I entered the casino again.
- Once more, I went inside.
}

// Shuffle: show one at random
At the table, I drew a card. <>
{ shuffle:
  - Ace of Hearts.
  - King of Spades.
  - 2 of Diamonds.
  'You lose this time!' crowed the croupier.
}

// Cycle: show each in turn, and then cycle
{ cycle:
  - I held my breath.
  - I waited impatiently.
  - I paused.
}

// Once: show each, once, in turn, until all have been
shown
{ once:
  - Would my luck hold?
  - Could I win the hand?
}

```

Advanced: modified shuffles

The shuffle block above is really a “shuffled cycle”; in that it’ll shuffle the content, play through it, then reshuffle and go again. (Imagine dealing a deck of cards, gathering up, shuffling, and dealing again.)

There are two other versions of shuffle:

`shuffle once` which will shuffle the content, play through it, and then do nothing.

```

{ shuffle once:
  - The sun was hot.
  - It was a hot day.
}

```

`shuffle stopping` will shuffle all the content (except the last entry), and once it's been played, it'll stick on the last entry forever.

```
{ shuffle stopping:  
-   A silver BMW roars past.  
-   A bright yellow Mustang takes the turn.  
-   There are like, cars, here.  
}
```

4. Temporary Variables

Temporary variables are for scratch calculations

Sometimes, a global variable is unwieldy. ink provides temporary variables for quick calculations of things.

```
=== near_north_pole ===
~ temp number_of_warm_things = 0
{ blanket:
  ~ number_of_warm_things++
}
{ ear_muffs:
  ~ number_of_warm_things++
}

{ gloves:
  ~ number_of_warm_things++
}

{ number_of_warm_things > 2:
  Despite the snow, I felt incorrigibly snug.
- else:
  That night I was colder than I have ever been.
}
```

The value in a temporary variable is thrown away after the story leaves the stitch in which it was defined.

Knots and stitches can take parameters

A particularly useful form of temporary variable is a parameter. Any knot or stitch can be sent a value as a parameter (or multiple values, separated by commas).

```
*   [Accuse Hasting]
-> accuse("Hastings", false)
*   [Accuse Mrs Black]
-> accuse("Claudia", true)
*   [Accuse myself]
-> accuse("myself", false)
```

```

=== accuse(who, correct) ===
    "I accuse {who}!" Poirot declared.
    {correct:
        "Of course!" Japp replied, smacking himself in the
forehead.
    - else:
        "Really?" Japp replied. "{who == "myself":You did
it?|{who}?}"
        "And why not?" Poirot shot back.
    }

```

You'll need to use parameters if you want to pass a temporary value from one stitch to another.

Advanced: a recursive knot definition

Temporary variables are safe to use in recursion (unlike globals), so the following will work.

```

-> total_one_to_one_hundred(0, 1)

=== total_one_to_one_hundred(total, x) ===
    // add x to our total so far
    ~ total = total + x
    { x == 100:
        -> finished(total)
    - else:
        // if we're not finished yet, add the next number
along
        -> total_one_to_one_hundred(total, x + 1)
    }

=== finished(total) ===
    "The result is {total}!" you announce.
    Gauss stares at you in horror. "Did you honestly do
that the long way?"
    -> END

```

(In fact, this kind of definition is useful enough that ink provides a special kind of knot, called, imaginatively enough, a **function**, which comes with certain restrictions and can return a value. See the section on [functions](#) which is coming up momentarily.)

Advanced: sending divert targets as parameters

Knot/stitch addresses are a type of value, indicated by a `->` character, and can be stored and passed around. The following is therefore legal, and often useful:

```
=== sleeping_in_hut ===  
  You lie down and close your eyes.  
  -> generic_sleep (-> waking_in_the_hut)  
  
=== generic_sleep (-> waking)  
  You sleep perchance to dream etc. etc.  
  -> waking  
  
=== waking_in_the_hut  
  You get back to your feet, ready to continue your  
  journey.
```

...but note the `->` in the `generic_sleep` definition: that's the one case in ink where a parameter needs to be typed: because it's too easy to otherwise accidentally do the following:

```
=== sleeping_in_hut ===  
  You lie down and close your eyes.  
  -> generic_sleep (waking_in_the_hut)
```

... which sends in the read count of `waking_in_the_hut` into the sleeping knot, and then attempts to divert to it, which is catastrophic.

5. Functions

The use of parameters on knots means they are almost functions in the usual sense, but they lack two key concepts – that of a call stack, and the use of return values.

ink includes functions: they are knots, with the following limitations and features:

A function:

- cannot contain stitches
- cannot use diverts or offer choices
- can call other functions
- can include printed content
- can return a value of any type
- can recurse safely

(Some of these may seem quite limiting, but for more story-oriented call-stack-style features, see the section on [tunnels](#).)

Return values are provided via the `~ return` statement.

Defining and calling functions

To define a function, simply declare a knot to be one:

```
=== function say_yes_to_everything ===  
  ~ return true  
  
=== function lerp(a, b, k) ===  
  ~ return ((b - a) * k) + a
```

Functions are called by name, and with brackets, even if they have no parameters:

```
~ x = lerp(2, 8, 0.3)  
  * {say_yes_to_everything()} 'Yes.'
```

As in any other language, a function, once done, returns the flow to wherever it was called from – and despite not being allowed to divert the flow, functions can still call other functions.

```
=== function say_no_to_nothing ===  
  ~ return say_yes_to_everything()
```


Functions end when they hit return

Weirdly, `return` doesn't need to return a value. It can be used simply to leave the function. (This can save you a bit of bracketing (but it's rarely critical).)

```
=== function smoke_them_if_youve_got_them() ===
{ cigars == 0:
  ~ return
}
~ cigars--
~ smellyness++
```

But functions don't need a `return` statement at all, and can simply run out of content and return when that happens. The following is perfectly valid:

```
=== function harm(x) ===
{ stamina < x:
  ~ stamina = 0
- else:
  ~ stamina = stamina - x
}
```

... though remember that since a function cannot divert the flow, while the above prevents a negative `stamina` value, it won't kill a player who hits zero – if you wanted to trigger a death state on losing all your stamina, a tunnel would be better here.

Functions can insert content into the flow

```
~ swear()
[ Please don't swear! ]

=== function swear()
{shuffle:
- Bother!
- Drat!
- Goshdarnit!
}
```

A function like this can be used, for example, to insert text into the story-flow while *also* reporting back something about that content via its return value:

```
~ temp badWord = swear()
[ Please don't swear {badWord:so colourfully}! ]

=== function swear()
{ RANDOM(1, 10) == 1:
  {shuffle:
    - Belgium!
    - Cronkswobble!
  }
  ~ return true
- else:
  {shuffle:
    - Drat!
    - Goshdarnit!
  }
  ~ return false
}
```

Functions can be called inline

Functions can be called on ~ content lines, but can also be called during a piece of content. In this context, any text content produced is glued in, and the return value, if there is one, is also glued in. That said, in this context, it's common not to use a return value at all, since the return value can't be captured by the function call.

```
Monsieur Fogg was looking {describe_health(health)}.
```

```
=== function describe_health(x) ===
{
- x == 100:
  spritely
- x > 75:
  chipper
- x > 45:
  somewhat flagging
- else:
  despondent
}
```

produces:

Monsieur Fogg was looking despondent.

Example: nesting returns and inline functions

The maximum of 2^5 and 3^3 is $\{\max(\exp(2,5), \exp(3,3))\}$.

```
=== function max(a,b) ===  
// does what the inbuilt MAX(a, b) function does  
{ a < b:  
  ~ return b  
- else:  
  ~ return a  
}  
  
=== function exp(x, n) ===  
// does what the inbuilt POW(x, n) function does  
{ n <= 0:  
  ~ return 1  
- else:  
  ~ return x * exp(x, n - 1)  
}
```

produces:

The maximum of 2^5 and 3^3 is 32.

Example: turning numbers into words

The following example is long, but appears in pretty much every inkle game to date. Recall that a hyphenated line inside multiline curly braces indicates either “a condition to test” or, if the curly brace began with a variable, “a value to compare against”; this example uses both.

(This code is available from inky’s “ink” menu, under Useful Functions.)

```
=== function print_num(x) ===  
{
```

```

- x >= 1000:
    {print_num(x / 1000)} thousand { x mod 1000 > 0:
{print_num(x mod 1000)}}
- x >= 100:
    {print_num(x / 100)} hundred { x mod 100 > 0:and
{print_num(x mod 100)}}
- x == 0:
    zero
- else:
    { x >= 20:
        { x / 10:
            - 2: twenty
            - 3: thirty
            - 4: forty
            - 5: fifty
            - 6: sixty
            - 7: seventy
            - 8: eighty
            - 9: ninety
        }
        { x mod 10 > 0:
            <>-<>}
    }
    { x < 10 || x > 20:
        { x mod 10:
            - 1: one
            - 2: two
            - 3: three
            - 4: four
            - 5: five
            - 6: six
            - 7: seven
            - 8: eight
            - 9: nine
        }
    }
- else:
    { x:
        - 10: ten
        - 11: eleven
        - 12: twelve
        - 13: thirteen
    }

```

```
- 14: fourteen
- 15: fifteen
- 16: sixteen
- 17: seventeen
- 18: eighteen
- 19: nineteen
}
}
}
```

which enables us to write things like:

```
~ price = 15
```

I pulled out {print_num(price)} coins from my pocket and slowly counted them.

"Oh, never mind," the trader replied. "I'll take half." And she took {print_num(price / 2)}, and pushed the rest back over to me.

Parameters can be passed by reference

Function parameters can also be passed 'by reference', meaning that the function can actually alter the variable being passed in, instead of creating a temporary variable with that value. For instance, most inkle stories include:

```
=== function alter(ref x, k) ===
  ~ x = x + k
```

Lines such as:

```
~ gold = gold + 7
~ health = health - 4
```

become:

```
~ alter(gold, 7)
~ alter(health, -4)
```

which are (perhaps) slightly easier to read, and (more usefully) can be done inline for maximum compactness.

```
*    I ate a biscuit[] and felt refreshed.  
{alter(health, 2)}  
*    I gave a biscuit to Monsieur Fogg[] and he wolfed  
it down most indecorously. {alter(foggs_health, 1)}  
-    <> Then we continued on our way.
```

Wrapping up simple operations in function can also provide a simple place to put debugging information, if required.

6. Constants

Global constants

Interactive stories often rely on state machines, tracking what stage some higher level process has reached. There are lots of ways to do this, but the most convenient is to use constants.

Sometimes, it's convenient to define constants to be strings, so you can print them out, for gameplay or debugging purposes.

```
CONST HASTINGS = "Hastings"
CONST POIROT = "Poirot"
CONST JAPP = "Japp"

VAR current_chief_suspect = HASTINGS

=== review_evidence ===
{ found_japps_bloodied_glove:
  ~ current_chief_suspect = JAPP
}
Current Suspect: {current_chief_suspect}
```

Sometimes giving them values is useful:

```
CONST PI = 3.14
CONST VALUE_OF_TEN_POUND_NOTE = 10
```

And sometimes the numbers are just placeholders so we can tell things apart in a human-readable way:

```
CONST LOBBY = 1
CONST STAIRCASE = 2
CONST HALLWAY = 3

CONST HELD_BY_AGENT = -1

VAR secret_agent_location = LOBBY
VAR suitcase_location = HALLWAY
```

```

=== report_progress ===
{  secret_agent_location == suitcase_location:
    The secret agent grabs the suitcase!
    ~ suitcase_location = HELD_BY_AGENT

-  secret_agent_location < suitcase_location:
    The secret agent moves forward.
    ~ secret_agent_location++
}

```

However, using constants in this way is a little old-fashioned: it's better to use ink's "list" concept for this kind of tracking, as it provides a system of named constants with useful additional structure. That said, lists are complex enough that they need an entire chapter of their own.

Divert constants

A constant can also contain a divert, but since diverts are themselves constant, this isn't often useful. One could perhaps package up a long address:

```

CONST frodosDescription = -> descriptions.frodo

=== descriptions
= frodo
    Even for a hobbit, Frodo Baggins was short.
->->

```

... but calling descriptions with "Frodo" as a parameter is better form, and using list values for these parameters is even better.

7. Advanced: Game-side logic

ink stories can be self-sufficient, but they're designed to be operated within a wider game context – whether that's a nice UI into a text-based story, or something much more graphical, like an adventure game or a turn-based strategy. In these examples, it becomes necessary to communicate between the ink story and the game-code. There are two core ways to do this communication.

External functions in ink allow you to directly call game-code functions in the game. This can allow ink to branch the story based on wider game-state information. It can also be used, if necessary, to optimise ink functions by providing a much-faster game-code version of the same calculation.

Variable observers are call-backs defined in code that are fired when ink variables are modified. These can be useful for updating the game UI to reflect the value of ink variables, checking for death states, and so on.

Both of these are described in full in the *Running Your Ink* section of this book.

Advanced Flow Control

The systems and syntax covered so far are enough to produce complete, complex branching stories with lots of interactivity, so long as the story runs from beginning to end, in the manner of a choose your path book.

But ink also includes two powerful flow control features – called *threads* and *tunnels* – that allow the story to break out of a tree structure. Threads bring together story content from different places into a single flow, while tunnels allow the same story content to be patched into different points in the main story.

These are features which you might find don't make a lot of sense until you have a story which needs them, so if you want to just get on and write, you can skip this chapter until you find yourself getting stuck!

But should you find yourself writing a dream sequence or a flashback, perhaps; or creating actions which apply in a lot of places – checking what you're carrying, say, or resting to recover – then this chapter will be of use.

1. Tunnels

The default structure for ink stories is a “linear” tree of choices, branching and joining back together, perhaps looping, but with the story always being “at a certain place”.

But this structure makes certain things difficult. For example, imagine a game in which the following interaction can happen:

```
=== crossing_the_date_line ===

*   "Monsieur!"[] I declared with sudden horror. "I have
just realised. We have crossed the international date
line!"

-   Monsieur Fogg barely lifted an eyebrow. "I have
adjusted for it."
*   I mopped the sweat from my brow[]. A relief!
*   I nodded, becalmed[]. Of course he had!
*   I cursed, under my breath[]. Once again, I had been
belittled!
```

The problem is, this can happen at lots of different places in the story. We don't want to have to write copies of this content for each different place, but when the content is finished it needs to know where to return to. We can do this using parameters:

```
=== crossing_the_date_line(-> return_to) ===
    "We have crossed the international date-line,
Monsieur!"
    -> return_to

=== outside_honolulu ===
    We arrived at the large island of Honolulu.
-   (postscript)
    -> crossing_the_date_line(-> done)
-   (done)
    ...

=== outside_pitcairn_island ===
    The boat sailed along the water towards the tiny island.
```

```
- (postscript)
-> crossing_the_date_line(-> done)
- (done)
...
```

Both of these locations now call and execute the same segment of story-flow, but once finished they return to where they need to go next. But what if the section of story being called is more complex – what if it spreads across several knots? Using the above, we'd have to keep passing the `return-to` parameter from knot to knot, to ensure we always knew where to return.

Instead, ink integrates this idea into the language with a new kind of divert, that functions rather like a subroutine, and is called a *tunnel*.

Tunnels run sub-stories

The tunnel syntax looks like a divert, with another divert on the end:

```
-> crossing_the_date_line ->
```

This means “do the `crossing_the_date_line` story, then when you’re done, continue on from here”.

Inside the tunnel itself, the syntax is simplified from the parameterised example: all we do is end the tunnel using the `->->` statement which means, essentially, “off you go!”

```
=== crossing_the_date_line ===
// this is a tunnel!
...
-   ->-> // time to get on with the story
```

Note that unlike functions, tunnel knots aren’t declared as such, so the compiler won’t check that tunnels really do end in `->->` statements, except at run-time. So you will need to write carefully to ensure that all the flows into a tunnel really do come out again.

Tunnels can also be chained together:

```
// this runs one tunnel, then another, then comes back
-> crossing_the_date_line -> check_foggs_health ->
```

or they can finish on a normal divert:

```
// this runs the tunnel, then diverts to 'done'
-> crossing_the_date_line -> done
```

Tunnels can be nested, so the following is valid:

```
=== plains ===
= night_time
  The dark grass is soft under your feet.
  + [Sleep]
    -> sleep_here -> wake_here -> day_time
= day_time
  It is time to move on.

=== wake_here ===
  You wake as the sun rises.
  + [Eat something]
    -> eat_something ->
  + [Make a move]
    - ->->

=== sleep_here ===
  You lie down and try to close your eyes.
  -> monster_attacks ->
  Then it is time to sleep.
  -> dream ->
  ->->
```

... and so on.

Tunnels can return elsewhere

Sometimes, in a story, things happen. So sometimes a tunnel can't guarantee that it will always want to go back to where it came from. `ink` supplies a syntax to allow you to “returning from a tunnel but actually go somewhere else”, but it should be used with caution as the possibility of getting very confused when doing this kind of thing is very high indeed.

Still, there are cases where it's indispensable:

```
=== fall_down_cliff_or_whatever_else
-> hurt(5) ->
```

You're still alive! You pick yourself up and walk on.
...

```
=== hurt(x)
  ~ stamina -= x
  { stamina <= 0:
    ->-> youre_dead
  }
  ->->
```

```
=== youre_dead
  Suddenly, there is a white light all around you. Fingers
  lift an eyepiece from your forehead. 'You lost, buddy.
  Out of the chair.'
```

And even in less drastic situations, we might want to break up the structure:

```
-> talk_to_jim ->

=== talk_to_jim
  - (opts)
    * [ Ask about the warp lacelles ]
      -> warp_lacells ->

    *[ Ask about the shield generators ]
      -> shield_generators ->

    * [ Stop talking ]
      ->->
  - -> opts

= warp_lacells
  { shield_generators : ->-> argue }
  "What do you want to know about the warp lacelles?"
  ...
  ->->

= shield_generators
  { warp_lacells : ->-> argue }
  "I shouldn't really tell you about the shields..."
```

```
->->  
  
= argue  
  "What's with all these questions?" Jim demands,  
  suddenly.  
  ...  
->->
```

Advanced: Tunnels use a call-stack

Tunnels are on a call-stack, so can safely recurse. (ink functions are, internally, tunnels. The `function` type is only used to make them “safer”.) The current depth of the call-stack can be queried by the game using [external functions](#) – see [Running your Ink](#) for more about this.

2. Threads

Tunnels are still quite chunky: the story stops, does a sub-story, then comes back. But it's also possible for a writer to fork a story into different sub-sections, to mix together different possible player actions in one place.

We call this “threading”, though it's not really threading in the sense that computer scientists mean it: it's more like stitching in new content from various places.

Note that this is *definitely* an advanced feature: the engineering of stories becomes markedly more complex once threads are involved!

Threads join multiple sections together

Threads allow you to consolidate sections of content from multiple sources in one place, and is denoted by a “pasting in” arrow, <-. For example:

```
== thread_example ==
  I had a headache; threading is hard to get your head
  around.
  <- conversation
  <- walking

== conversation ==
  It was a tense moment for Monty and me.
  * "What did you have for lunch today?[]" I asked.
    "Spam and eggs," he replied.
  * "Nice weather, we're having,[]" I said.
    "I've seen better," he replied.
  - -> house

== walking ==
  We continued to walk down the dusty road.
  * [Continue walking]
    -> house

== house ==
  Before long, we arrived at his house.
  -> END
```

This allows multiple sections of story to be used simultaneously, leaving it up to the player to choose which to pursue:

I had a headache; threading is hard to get your head around.

It was a tense moment for Monty and me.

We continued to walk down the dusty road.

1: "What did you have for lunch today?"

2: "Nice weather, we're having,"

3: Continue walking

On encountering a thread statement such as `<- conversation`, the compiler will fork the story flow. The first fork considered will run the content at `conversation`, collecting up any options it finds. Once it has run out of flow here it'll then run the other fork.

All the content is collected and shown to the player. But when a choice is chosen, the engine will move to that fork of the story and collapse and discard the others.

Note that global variables are *not* forked, including the read counts of knots and stitches, but local variables including parameters are.

Uses for threads

In a normal story, threads might never be needed. But for games with lots of independent moving parts, threads quickly become essential.

Imagine a game in which characters move independently around a map, and you can talk to whoever is present wherever you are.

```
// Define some constants to represent the rooms in the house
```

```
CONST HALLWAY = 1
```

```
CONST OFFICE = 2
```

```
CONST KITCHEN = 3
```

```
// Track the locations of various characters
```

```
VAR player_location = HALLWAY
```

```
VAR generals_location = HALLWAY
```

```
VAR doctors_location = OFFICE
```

```
-> run_player_location
```

```
== run_player_location
```

```

{ player_location:
- HALLWAY: -> hallway
- OFFICE: -> office
- KITCHEN: -> kitchen
}

== hallway ==
<- characters_present(HALLWAY)
*   [Drawers] -> examine_drawers
*   [Wardrobe] -> examine_wardrobe
*   [Go to Office] -> go_office
-   -> run_player_location

= examine_drawers
// etc...

// And here's the threaded part, which mixes in dialogue
for characters you share the room with at the moment.

== characters_present(room)
{ generals_location == room:
  <- general_conversation
}
{ doctors_location == room:
  <- doctor_conversation
}

== general_conversation
*   [Ask the General about the bloodied knife]
    "It's a bad business, I can tell you."
-   -> run_player_location

== doctor_conversation
*   [Ask the Doctor about the bloodied knife]
    "There's nothing strange about blood, is there?"
-   -> run_player_location

```

Note in particular, that we need an explicit way to return the player who has gone down a side-thread to return to the main flow. In most cases, threads will either need a parameter telling them where to return to, they'll need to end the current story

section, or they'll need some kind of known hub point to return back to (which is what `run_player_location` is doing in the example above).

Mixing threads and weave content

Threads do not take priority over weave content. That means that while

```
<- thread_in_a_choice
  * [ Another choice ]
```

generates two parallel choices,

```
* [ Another choice ]
  <- thread_in_a_choice
```

generates one choice followed by another, because the thread is inside the weave.

In particular, that means if you want to control the order of choices in a game, you might find yourself handicapped by the need to put the threaded choices before the weave-based ones. The solution, sadly, is more threads. If you move your weave content into a knot and thread it, you can order it however you want.

```
<- weave_choices
<- threaded_choices

= weave_choices
  * [ Weave based choices ]
```

When does a side-thread end?

Side-threads end when they run out of flow to process: they collect up options to display alongside any choices found in the main flow, or in other side-threads. (This is unlike tunnels, which are handled first-come-first-served.)

Sometimes a thread has no content to offer – perhaps there is no conversation to have with a character after all, or perhaps we have simply not written it yet. In that case, we must mark the end of the thread explicitly. If we didn't, the end of content might be a story-bug or a hanging story thread, and we want the compiler to tell us about those.

Using `-> DONE`

In cases where we want to mark the end of a thread, we use `-> DONE`: meaning “the flow intentionally ends here”. If we don't, we might end up with a warning

message – we can still play the game, but it's a reminder that we have unfinished business.

The example at the start of this section will generate such a warning; it can be fixed as follows:

```
== thread_example ==  
  I had a headache; threading is hard to get your head  
  around.  
    <- conversation  
    <- walking  
    -> DONE
```

The extra `-> DONE` tells ink that the flow here has ended and it should rely on the threads for the next part of the story.

Note that we don't need a `-> DONE` if the flow ends with options that fail their conditions. The engine treats this as a valid, intentional, end of flow state. It's only there for cases where ink can't be sure that the flow hasn't just been forgotten about.

You do not need a `-> DONE` after an option has been chosen. Once an option is chosen, a thread is no longer a thread – it is simply the normal story flow once more.

```
-> END vs -> DONE
```

In the past, we've used `-> END` to tell ink that we've finished. `-> END` is truly final; it kills the story processing immediately once encountered and if you use it in a side-thread, it would, rather stubbornly, end the story there and then.

`-> DONE` is rather softer, and simply says ends the current thread, allowing others to continue, and any choices gathered so far to be seen. That said, you'll never need to use `-> DONE` outside of a thread.

Longer Example: adding the same choice to several places

Threads can be used to add the same choice into lots of different places. When using them this way, it's normal to pass a divert as a parameter, to tell the story where to go after the choice is done. (Unlike a tunnel, there's no way to tell a thread “come back here when you're done”, because once an option is chosen inside a thread, it *is* the flow. In other words, there's no call-stack involved.)

```
=== outside_the_house  
  The front step. The house smells. Of murder. And  
  lavender.  
- (top)  
  <- review_case_notes(-> top)  
  * [Go through the front door]
```

```

    I stepped inside the house.
-> the_hallway
* [Sniff the air]
    I hate lavender. It makes me think of soap, and soap
makes me think about my marriage.
-> top

=== the_hallway
    The hallway. Front door open to the street. Little
bureau.
- (top)
  <- review_case_notes(-> top)
  * [Go through the front door]
    I stepped out into the cool sunshine.
    -> outside_the_house
  * [Open the bureau]
    Keys. More keys. Even more keys. How many locks do
these people need?
    -> top

=== review_case_notes(-> go_back_to)
  + {not done || TURNS_SINCE(-> done) > 10}
    [Review my case notes]
// the conditional ensures you don't get the option to
check repeatedly
    {I|Once again, I} flicked through the notes I'd made
so far. Still no obvious suspects.
- (done) -> go_back_to

```

Example: organisation of wide choice points

A game which uses ink as a script rather than a literal output might often generate very large numbers of parallel choices, intended to be filtered by the player via some other in-game interaction – such as walking around an environment.

Threads can be useful in these cases simply to organise the choices.

```

=== the_kitchen
- (top)
  <- drawers(-> top)
  <- cupboards(-> top)

```

```
    <- room_exits
= drawers (-> goback)
  // choices about the drawers...
  ...
= cupboards(-> goback)
  // choices about cupboards
  ...
= room_exits
  // exits; doesn't need a "return point" as if you leave,
  you go elsewhere
  ...
```

3. Threaded Tunnels

Threads and tunnels are both independently useful, but can be powerfully combined. Imagine a game in which the player is standing in a room, surrounded by objects to interact with, but also accompanied by a side-character.

We might want to do something like the following:

```
=== kitchen
- (top)
  <- conversation_opts(-> loop)

  * [The cooker]
  ...
  * [The fridge]
  ...
  * [The catflap]
  ...
- (loop) -> top

=== conversation_opts(-> back_to)
  * "So tell me about this soup..."
  * "Are you ever going to cook anything?"
  * "Interested in soufflé at all?"
  - -> back_to
```

The player can explore the objects in their environment, but also at any time they can engage with conversation with their ally – and the same conversation options can be offered in a single line in any other room in the game.

This is a good pattern, but if the conversation options become complex it can get unwieldy: the `-> back_to` parameter has to be passed from knot to knot so the story always knows where to return to once the conversation is over. If those conversations get long and complex, so does the ink involved for ensuring that, once the conversation is over, we go back to where we came from.

A “threaded tunnel” wraps this concept up into a convenient form: but it’s not a built-in feature of the ink language, but rather a function you can include.

```
=== thread_in_tunnel(-> tunnel_to_run, ->
place_to_return_to)
  ~ temp entryTurnChoice = TURNS()
  -> tunnel_to_run ->
```

```
// if the tunnel contained choices which were chosen
// then the turn count will have increased, so we
// use the given return point to continue the flow

{entryTurnChoice != TURNS():
  -> place_to_return_to
}

// otherwise the given tunnel simply ran through, in
// which
// case we should treat this as a side-thread, and stop
-> DONE
```

Once included, it's used as follows:

```
=== kitchen
- (top)
  <- thread_in_tunnel(-> conversation_opts, -> loop)
```

It's very similar to the original thread! But it comes with the advantage that the conversation options now only need to end in the generic `->->` return from tunnel marker.

```
=== conversation_opts
* [Ask about Annabel's budgie] -> annabels_budgie
  ... etc ...

=== annabels_budgie
"Say, have you seen Annabel's budgie?"
"Ah. Ze cat, Douglas. He eat it."
->->
```

The conversation options can be written as entirely separation sections of flow, and they can even be invoked as a tunnel directly from elsewhere if required. The information that they are to be threaded into the room options hub is *kept* in the room options hub; the conversation block itself doesn't need to know.

Advanced State Tracking using LISTS

Games with lots of interaction can get very complex very quickly, and the writer's job is often as much about maintaining continuity as it is about making content.

This becomes particularly important if the game text is intended to model anything – whether it's a game of cards, the player's knowledge of the game-world so far, or the state of the various light-switches in a house.

ink does not provide a full world-modelling system in the manner of a classic parser IF authoring language – there are no “objects”, no concepts of “containment” or being “open” or “locked”. However, it does provide a simple yet powerful system for tracking state-changes in a very flexible way, to enable writers to approximate world models where necessary.

1. Basic Lists

The basic unit of state-tracking is a list of states, defined using the `LIST` keyword. Note that a list is really nothing like a C# list (which is an array).

```
LIST kettleState = cold, boiling, recently_boiled
```

This line defines two things: firstly three new meaningful values – `cold`, `boiling` and `recently_boiled` – and secondly, a variable, called `kettleState`, to hold these states.

We can tell the list what value to take:

```
~ kettleState = cold
```

We can change and query the value:

```
*   [Turn on kettle]
    The kettle begins to bubble and boil.
    ~ kettleState = boiling

-
*   [Touch the kettle]
    { kettleState == cold:
      The kettle is cool to the touch.
    - else:
      The outside of the kettle is very warm!
    }
```

For convenience, we can give a list a value when it's defined using a bracket:

```
LIST kettleState = cold, (boiling), recently_boiled
// From the start, this kettle is switched on. Edgy, huh?
```

2. Reusing Lists

The above example is fine for the kettle, but what if we have a pot on the stove as well? We can then define a list of states, but put them into variables – and have as many of these variables as we want.

```
LIST weekdays = Monday, Tuesday, Wednesday, Thursday, Friday
```

```
VAR today = Monday  
VAR tomorrow = Tuesday  
VAR lieInDay = Sunday
```

States can be used repeatedly

This allows us to use the same state machine in multiple places.

```
LIST heatedWaterStates = cold, boiling, recently_boiled
```

```
VAR kettleState = cold  
VAR potState = cold
```

```
* {kettleState == cold} [Turn on kettle]  
  The kettle begins to boil and bubble.  
  ~ kettleState = boiling  
* {potState == cold} [Light stove]  
  The water in the pot begins to boil and bubble.  
  ~ potState = boiling
```

But what if we add a microwave as well? We might want start generalising our functionality a bit:

```
LIST heatedWaterStates = cold, boiling, recently_boiled
```

```
VAR kettleState = cold  
VAR potState = cold  
VAR microwaveState = cold
```

```
=== function boilSomething(ref thingToBoil, nameOfThing)
```

```
The {nameOfThing} begins to heat up.  
~ thingToBoil = boiling
```

```
=== do_cooking  
* {kettleState == cold} [Turn on kettle]  
  {boilSomething(kettleState, "kettle")}  
* {potState == cold} [Light stove]  
  {boilSomething(potState, "pot")}  
* {microwaveState == cold} [Turn on microwave]  
  {boilSomething(microwaveState, "microwave")}
```

or even...

```
LIST heatedWaterStates = cold, boiling, recently_boiled  
VAR kettleState = cold  
VAR potState = cold  
VAR microwaveState = cold
```

```
=== cook_with(nameOfThing, ref thingToBoil)  
+ {thingToBoil == cold} [Turn on {nameOfThing}]  
  The {nameOfThing} begins to heat up.  
  ~ thingToBoil = boiling  
  -> do_cooking.done
```

```
=== do_cooking  
  <- cook_with("kettle", kettleState)  
  <- cook_with("pot", potState)  
  <- cook_with("microwave", microwaveState)  
- (done)
```

(Note that the `heatedWaterStates` list is still available as well, and can still be tested, and take a value, though it might start to get confusing if you did use it.)

Advanced: list values can share names

Reusing lists brings with it ambiguity. If we have:

```
LIST colours = red, green, blue, purple  
LIST moods = mad, happy, blue
```

```
VAR status = blue
```

... how can the compiler know which blue you meant?

We resolve these using a `.` syntax similar to that used for knots and stitches.

```
VAR status = colours.blue
```

...and the compiler will issue an error until you specify.

Note the “family name” of the state, and the variable containing a state, are totally separate things. So

```
{ statesOfGrace == statesOfGrace.fallen:  
  // is the current state "fallen"  
}
```

... is correct.

Advanced: a LIST is actually a variable

One surprising feature that bears repeating is that the statement

```
LIST statesOfGrace = ambiguous, saintly, fallen
```

actually does two things simultaneously: it creates three values, `ambiguous`, `saintly` and `fallen`, and gives them the name-parent `statesOfGrace` if needed; and it creates a variable called `statesOfGrace`. And that variable can be used like a normal variable! So the following is valid, if horribly confusing.

```
LIST statesOfGrace = ambiguous, saintly, fallen  
~ statesOfGrace = 3.1415 // set the variable to a number  
instead
```

...and it wouldn't preclude the following from being fine. Yikes!

```
~ temp anotherStateOfGrace = statesOfGrace.saintly
```

3. List Values

When a list is defined, the values are listed in an order, and that order is considered to be significant and immutable. In fact, we can treat these values as if they *were* numbers. (That is to say, they're enums.)

```
LIST volumeLevel = off, quiet, medium, loud, deafening
VAR lecturersVolume = quiet
VAR murmurersVolume = quiet
{ lecturersVolume < deafening:
  ~ lecturersVolume++
  { lecturersVolume > murmurersVolume:
    ~ murmurersVolume++
    The murmuring gets louder.
  }
}
```

The values themselves can be printed using the usual {...} syntax.

The lecturer's voice becomes {lecturersVolume}.

Converting values to numbers

The numerical value, if needed, can be got explicitly using the `LIST_VALUE` function. Note the first value in a list has the value 1, and not the value 0.

The lecturer has {LIST_VALUE(deafening) - LIST_VALUE(lecturersVolume)} notches still available to him.

Converting numbers to values

You can go the other way by using the list's name as a “creator” function:

```
LIST Numbers = one, two, three
~ temp score = Numbers(2) // score will be "two"
```

Advanced: defining your own numerical values

By default, the values in a list start at 1 and go up by one each time, but you can specify your own values if you need to.

```
LIST primeNumbers = two = 2, three = 3, five = 5
```

If you specify one value, but not the next value, ink will assume an increment of 1. So the following is the same:

```
LIST primeNumbers = two = 2, three, five = 5
```

Note in particular that lists are 1-indexed, and if you want a zero-indexed list, you have to specify that.

```
LIST integers = zero = 0, one, two, three, four, five,  
loads
```

4. Multivalued Lists

The following examples have all included one deliberate simplification, which we'll now remove. Lists – and variables containing list values – do not have to contain only *one* value.

Lists are Boolean sets

A list variable is *not* a variable containing a number. Rather, a list is like the in/out nameboard in an doctor's surgery, that contains a list of names, each with a slider to say “in” or “out”.

Maybe it's the weekend, which is when small children always get ill, and so no one is in:

```
LIST DoctorsInSurgery = Adams, Bernard, Cartwright,  
Denver, Eamonn
```

Or maybe it's cake-day and everyone's there:

```
LIST DoctorsInSurgery = (Adams), (Bernard), (Cartwright),  
(Denver), (Eamonn)
```

Or maybe it's just, like, Tuesday, and some are and some aren't:

```
LIST DoctorsInSurgery = (Adams), Bernard, (Cartwright),  
Denver, Eamonn
```

Names in brackets are those which are included in the initial state of the list. Note that if you're defining your own values, you can place the brackets around the whole term or just the name:

```
LIST primeNumbers = (two = 2), (three) = 3, (five = 5)
```

Assigning multiple values

We can assign all the values of the list at once as follows:

```
~ DoctorsInSurgery = (Adams, Bernard)~ DoctorsInSurgery =  
(Adams, Bernard, Eamonn)
```


We can assign the empty list to clear a list out:

```
~ DoctorsInSurgery = ()
```

Adding and removing entries

List entries can be added and removed, singly or collectively.

```
~ DoctorsInSurgery = DoctorsInSurgery + Adams  
~ DoctorsInSurgery += Adams // this is the same as the  
above  
~ DoctorsInSurgery -= Eamonn  
~ DoctorsInSurgery += (Eamonn, Denver)  
~ DoctorsInSurgery -= (Adams, Eamonn, Denver)
```

Trying to add an entry that's already in the list does nothing. Trying to remove an entry that's not there also does nothing. Neither produces an error.

Printing out a list

The ink `{listName}` produces a comma-delimited print of what's in the list.

```
VAR DoctorsInSurgery = (Adams), Bernard, (Cartwright)  
{DoctorsInSurgery}
```

produces

```
Adams, Cartwright
```

This is sometimes good enough to use in game (especially when the list only has one value), but looks artificial in most contexts – it's included more for debugging purposes.

5. Querying Lists

Basic queries

We have a few basic ways of getting information about what's in a list:

```
LIST DoctorsInSurgery = (Adams), Bernard, (Cartwright),  
Denver, Eamonn
```

```
{LIST_COUNT(DoctorsInSurgery)}      // "2"  
{LIST_MIN(DoctorsInSurgery)}        // "Adams"  
{LIST_MAX(DoctorsInSurgery)}        // "Cartwright"  
{LIST_RANDOM(DoctorsInSurgery)}     // "Adams"?  
"Cartwright"?
```

Note that `LIST_RANDOM`, `LIST_MIN` and `LIST_MAX` will all produce the empty list, `()`, if and only if the list is currently empty.

Testing for emptiness

Like most values in ink, a list can be tested “as it is”, and will return true, unless it's empty.

```
{ DoctorsInSurgery: The surgery is open today. | Everyone  
has gone home. }
```

Testing for exact equality

Testing multi-valued lists is slightly more complex than single-valued ones. Equality (`==`) now means ‘set equality’ – that is, all entries must be present, and no other entries must be present.

So one might say:

```
{ DoctorsInSurgery == (Adams, Bernard):  
  Dr Adams and Dr Bernard are having a loud argument in  
  one corner.  
}
```

If Dr Eamonn is there as well, the two won't argue, as the lists being compared won't be equal – `DoctorsInSurgery` will have an `Eamonn` that the list `(Adams, Bernard)` doesn't have.

Not equals works as expected:

```
{ DoctorsInSurgery != (Adams, Bernard):  
    At least Adams and Bernard aren't arguing.  
}
```

Testing for containment

What if we just want to simply ask if *at least* Adams and Bernard are present? For that we use a new operator, `has`, otherwise known as `?`.

```
{ DoctorsInSurgery ? (Adams, Bernard):  
    Dr Adams and Dr Bernard are having a hushed argument  
    in one corner.  
}
```

And `?` can apply to single values too:

```
{ DoctorsInSurgery has Eamonn:  
    Dr Eamonn is polishing his glasses.  
}
```

We can also negate it, with `hasnt` or `!?` (not `?`). Note this starts to get a little complicated as

```
DoctorsInSurgery !? (Adams, Bernard)
```

does not mean that neither Adams nor Bernard is present, only that they are not *both* present (and arguing).

Note this is the same syntax we used for inclusion and not-inclusion in the [section on string queries](#).

Warning: every list does not contain the empty list

Be warned, the following is *false*:

```
{ some_list ? () }
```

This used to be the other way around, but that proved foolish, as it meant that tests like:

```

~ temp weapon = getWieldedWeapon()

{ silverWeapons ? weapon:
  The Werewolf shrinks at the sight of the {weapon}.
}

```

... would allow you to frighten off a werewolf simply by dropping all your weapons entirely. As of ink version 1.1, this was changed since we found lots of cases where the old behaviour was unhelpful, and none where it was good.

Example: basic knowledge tracking

The simplest use of a multi-valued list is for tracking “game flags” tidily.

```

LIST Facts = (Fogg_is_fairly_odd), first_name_phileas,
(Fogg_is_English)

{Facts ? Fogg_is_fairly_odd:I smiled politely.|I frowned.
Was he a lunatic?}
'{Facts ? first_name_phileas:Phileas|Monsieur}, really!'
I cried.

```

In particular, it allows us to test for multiple game flags in a single line.

```

{ Facts ? (Fogg_is_English, Fogg_is_fairly_odd):
  <> 'I know Englishmen are strange, but this is
*incredible*!'
}

```

Example: a doctor's surgery

We’re overdue a fuller example, so here’s one.

```

LIST DoctorsInSurgery = (Adams), Bernard, Cartwright,
(Denver), Eamonn

* [Time passes...]
  {doctorLeaves(Adams)}
  {doctorEnters(Cartwright)}
  {doctorEnters(Eamonn)}

```

```

    { whos_in_today() }

=== function whos_in_today()
    In the surgery today are {DoctorsInSurgery}.

=== function doctorEnters(who)
    { DoctorsInSurgery !? who:
      ~ DoctorsInSurgery += who
      Dr {who} arrives in a fluster.
    }

=== function doctorLeaves(who)
    { DoctorsInSurgery ? who:
      ~ DoctorsInSurgery -= who
      Dr {who} leaves for lunch.
    }

```

This produces:

In the surgery today are Adams, Denver.

> Time passes...

Dr Adams leaves for lunch. Dr Cartwright arrives in a fluster. Dr Eamonn arrives in a fluster.

In the surgery today are Cartwright, Denver, Eamonn.

Advanced: nicer list printing

The basic list print is not especially attractive for use in-game. The following is better:

```

=== function listWithCommas(list, if_empty)
    {LIST_COUNT(list):
      - 2:
        {LIST_MIN(list)} and {listWithCommas(list -
LIST_MIN(list), if_empty)}
      - 1:
        {list}
    }

```

```

- 0:
  {if_empty}
- else:
  {LIST_MIN(list)}, {listWithCommas(list -
LIST_MIN(list), if_empty)}
}

```

```

LIST favouriteDinosaurs = (stegosaurus), brachiosaur,
(anklyosaurus), (pleiosaur)

```

```

My favourite dinosaurs are
{listWithCommas(favouriteDinosaurs, "all extinct")}.

```

It's probably also useful to have an is/are function to hand:

```

=== function isAre(list)
  {LIST_COUNT(list) == 1:is|are}

```

```

My favourite dinosaurs {isAre(favouriteDinosaurs)}
{listWithCommas(favouriteDinosaurs, "all extinct")}.

```

And to be pedantic:

```

My favourite dinosaur{LIST_COUNT(favouriteDinosaurs) !=
1:s} {isAre(favouriteDinosaurs)}
{listWithCommas(favouriteDinosaurs, "all extinct")}.

```

Lists don't need to have multiple entries

Lists don't *have* to contain multiple values. If you want to use a list as a state-machine, the examples above will all work – set values using `=`, `++` and `--` and test them using `==`, `<`, `<=`, `>` and `>=`. These will all work as expected.

Intersecting lists

The `has` or `?` operator is, more formally, the “are you a subset of me” operator $\supseteq$ which includes the case of the sets being equal, but which requires that the larger set does entirely contain the smaller set.

To test for “some overlap” between lists, we use the overlap operator, `^`, to get the *intersection*.

```
LIST CoreValues = strength, courage, compassion, greed,  
nepotism, self_belief, delusions_of_godhood
```

```
VAR desiredValues = (strength, courage, compassion,  
self_belief )
```

```
VAR actualValues = ( greed, nepotism, self_belief,  
delusions_of_godhood )
```

```
{desiredValues ^ actualValues} // prints "self_belief"
```

The result is a new list, so you can test it:

```
{LIST_COUNT(desiredValues ^ actualValues) == 1:  
  The new president has exactly only one desirable  
quality.  
  {desiredValues ^ actualValues == self_belief:  
    It's the scary one.  
  }  
}
```

6. Advanced List Operations

The above section covers basic comparisons. There are a few more powerful features as well, but – as anyone familiar with mathematical sets will know – things begin to get a bit fiddly. So this section comes with an ‘advanced’ warning.

A lot of the features in this section won't be necessary for most games.

The “full” list

Note that `LIST_COUNT`, `LIST_MIN` and `LIST_MAX` are referring to who's in/out of the list, not the full set of *possible* doctors. We can access that using

```
LIST_ALL(element of list)
```

or

```
LIST_ALL(list variable associated with a list type)

{LIST_ALL(DoctorsInSurgery)}    // Adams, Bernard,
Cartwright, Denver, Eamonn
{LIST_COUNT(LIST_ALL(DoctorsInSurgery))} // "5"
{LIST_MIN(LIST_ALL(Eamonn))}      // "Adams"
```

Advanced: “refreshing” a list's type

“Associated with” is a solid concept that's largely unexplained here. But if you need to, you can make an empty list that knows what type of list it is.

```
LIST ValueList = first_val, second_val, third_val
VAR myList = ()

~ myList = ValueList()

{ LIST_ALL(myList) } // produces first_val, second_val,
third_val
```

Advanced: a portion of the "full" list

You can also retrieve just a “slice” of the full list, using the `LIST_RANGE` function. There are two formulations, both valid:


```
LIST_RANGE(list_name, min_integer_value,  
max_integer_value)
```

and

```
LIST_RANGE(list_name, min_value, max_value)
```

Min and max values here are inclusive. If the game can't find the values, it'll get as close as it can, but never go outside the range. So for example:

```
{LIST_RANGE(LIST_ALL(primeNumbers), 10, 20)}
```

will produce

```
11, 13, 17, 19
```

Comparing lists

We can compare lists less than exactly using `>`, `<`, `>=` and `<=`. But be warned! The definitions we use are not exactly standard fare. They are based on comparing the numerical value of the elements in the lists being tested.

“Distinctly bigger than”

`LIST_A > LIST_B` means “the smallest value in A is bigger than the largest values in B”: in other words, if put on a number line, the entirety of A is to the right of the entirety of B. `<` does the same in reverse.

“Definitely never smaller than”

`LIST_A >= LIST_B` means – take a deep breath now – “the smallest value in A is at least the smallest value in B, and the largest value in A is at least the largest value in B”. That is, if drawn on a number line, the entirety of A is either above B or overlaps with it, but B does not extend higher than A.

Note that `LIST_A > LIST_B` implies `LIST_A != LIST_B`, and `LIST_A >= LIST_B` allows `LIST_A == LIST_B` but precludes `LIST_A < LIST_B`, as you might hope. Or else you didn't read any of that.

Health warning!

`LIST_A >= LIST_B` is *not* the same as `LIST_A > LIST_B` or `LIST_A == LIST_B`.

The moral is, don't use these unless you have a clear picture in your mind.

Inverting lists

A list can be inverted, which is the equivalent of going through the accommodation in/out name-board and flipping every switch to its opposite:

```
LIST GuardsOnDuty = (Smith), (Jones), Carter, Braithwaite

=== function changingOfTheGuard
  ~ GuardsOnDuty = LIST_INVERT(GuardsOnDuty)
```

Note that `LIST_INVERT` on an empty list will return a null value, if the game doesn't have enough context to know *what* to invert, and the full list if it does. If you need to handle that case, it's safest to do it by hand:

```
=== function changingOfTheGuard
  {!GuardsOnDuty: // "is GuardsOnDuty empty right now?"
    ~ GuardsOnDuty = LIST_ALL(Smith)
  - else:
    ~ GuardsOnDuty = LIST_INVERT(GuardsOnDuty)
  }
```

Footnote

The syntax for inversion was originally `~ list` but we changed it because:

```
~ list = ~ list
```

was not only functional, but rather perversely caused a list to invert itself.

Example: Tower of Hanoi

To demonstrate a few of these ideas, here's a functional Tower of Hanoi example, written so no one else has to write it.

Please don't type it in by hand: the last person to do that was never seen again.

```
LIST Discs = one, two, three, four, five, six, seven

VAR post1 = ()
VAR post2 = ()
VAR post3 = ()
```

```
~ post1 = LIST_ALL(Discs)
```

```
-> gameloop
```

```
=== function can_move(from_list, to_list) ===
```

```
{
-   LIST_COUNT(from_list) == 0:    // no discs to move
  ~ return false
-   LIST_COUNT(to_list) > 0 && LIST_MIN(from_list) >
LIST_MIN(to_list):
// the moving disc is bigger than the smallest of the
discs on the new tower
  ~ return false
-   else:    // nothing stands in your way!
  ~ return true
}
```

```
== function getListForTower(towerNum)
```

```
{ towerNum:
- 1:    ~ return post1
- 2:    ~ return post2
- 3:    ~ return post3
}
```

```
=== gameloop
```

Staring down from the heavens you watch your followers finish construction of the last great temple, ready to begin.

```
- (top)
```

```
+ [ Regard the temples]
```

On each temple are stacked the rings of stone.

```
{describe_pillar(1)} {describe_pillar(2)}
```

```
{describe_pillar(3)}
```

```
<- move_post(1, 2, post1, post2)
```

```
<- move_post(2, 1, post2, post1)
```

```
<- move_post(1, 3, post1, post3)
```

```
<- move_post(3, 1, post3, post1)
```

```
<- move_post(3, 2, post3, post2)
```

```
<- move_post(2, 3, post2, post3)
```

-> DONE

```
= move_post(from_post, to_post, ref from_post_list, ref
to_post_list)
+ { can_move(from_post_list, to_post_list) }
  [ Move ring from the {name(from_post)} to the
{name(to_post)}]
  { move_ring(from_post_list, to_post_list) }
  { stopping:
```

- The priests far below construct a great harness, and after many years, the great stone ring is lifted up into the air, and swung over to the next of the temples. The ropes are slashed, and it falls once more.

- Your next decree is met with a great feast and many sacrifices. After the funeary smoke has cleared, work to shift the great stone ring begins in earnest. A generation grows and falls, and the ring falls into its ordained place.

```
- {cycle:
  - Years pass as the ring is slowly moved.
  - The priests below fight a war over what colour
  robes to wear, but while they die, the work is completed.
}
- > top
```

```
=== function describe_pillar(listNum) ==
~ temp list = getListForTower(listNum)
{
- LIST_COUNT(list) == 0:
  The {name(listNum)} is empty.

- LIST_COUNT(list) == 1:
  The {list} ring lies on the {name(listNum)}.

- else:
```

```
    On the {name(listNum)}, are the discs numbered  
{list}.  
}
```

```
=== function move_ring( ref from, ref to ) ===  
  ~ temp whichRingToMove = LIST_MIN(from)  
  ~ from -= whichRingToMove  
  ~ to += whichRingToMove
```

```
=== function name(postNum)  
  { postNum:  
    - 1: first  
    - 2: second  
    - 3: third  
  }  
<> temple
```

7. Multi-list Lists

So far, all of our examples have included one large simplification, again – that the values in a list variable have to all be from the same list family. But they don't.

This allows us to use lists – which have so far played the role of state-machines and flag-trackers – to also act as general properties, which is useful for world modelling.

This is our inception moment. The results are powerful, but also more like “real code” than anything that’s come before.

Lists to track objects

For instance, we might define:

```
LIST Characters = Alfred, Batman, Robin
LIST Props = champagne_glass, newspaper

VAR BallroomContents = (Alfred, Batman, newspaper)
VAR HallwayContents = (Robin, champagne_glass)
VAR BathroomContents = ()

=== function describe_room(roomState)
{ roomState ? Alfred:
  <> Alfred is here, standing quietly in a corner.
}
{ roomState ? Batman:
  <> Batman's presence dominates all.
}
{ roomState ? Robin:
  <> Robin is all but forgotten.
}
{ roomState ? champagne_glass:
  <> A champagne glass lies discarded.
}
{ roomState ? newspaper:
  On one table, a headline blares out WHO IS THE BATMAN?
  AND *WHO* IS HIS BARELY-REMEMBERED ASSISTANT?
}
```

This gives us one function which can described the littered items and people within any room of the game, updating as necessary. We have

```
{ describe_room(HallwayContents) }
```

giving:

Robin is all but forgotten. A champagne glass lies discarded.

While

```
{ describe_room(BallroomContents) }
```

produces:

Alfred is here, standing quietly in a corner. Batman's presence dominates all.
On one table, a headline blares out WHO IS THE BATMAN? AND *WHO* IS HIS BARELY-REMEMBERED ASSISTANT?

And

```
{ describe_room(BathroomContents) }
```

produces nothing.

We can also have options based on combinations of things:

```
* { currentState ? (Batman, Alfred) }  
  [Talk to Alfred and Batman]  
  'Say, do you two know each other?'
```

Lists to track multiple states

We can model devices with multiple states by creating lists for each state chain, and a single variable representing the object and what's true about it right now. Back to the kettle again...

```
LIST OnOff = on, off  
LIST HotCold = cold, warm, hot
```

```

VAR kettleState = (off, cold)

=== function turnOnKettle() ===
  { kettleState ? hot:
    You turn on the kettle, but it immediately flips off
    again.
    - else:
      The water in the kettle begins to heat up.
      ~ kettleState -= off // not "=", as that would
      ~ kettleState += on  // remove all existing states
    }

=== function can_make_tea() ===
  ~ return kettleState ? (hot, off)

```

These mixed states can make changing state a bit trickier, as the off/on above demonstrates, so the following helper function can be useful.

```

=== function changeStateTo(ref stateVariable,
stateToReach)
  // remove all states of this type
  ~ stateVariable -= LIST_ALL(stateToReach)

  // put back the state we want
  ~ stateVariable += stateToReach

```

which enables code like:

```

~ changeState(kettleState, on)

~ changeState(kettleState, warm)

```

so the kettle can be turned on without, say, forgetting it's already boiled.

How does this affect queries?

The list queries given above generalise to multi-valued lists:

```

LIST Letters = a,b,c
LIST Numbers = one, two, three

```



```

VAR mixedList = (a, three, c)

{LIST_ALL(mixedList)}    // a, one, b, two, c, three
{LIST_COUNT(mixedList)} // 3
{LIST_MIN(mixedList)}   // a
{LIST_MAX(mixedList)}   // three or c, albeit
unpredictably

{ LIST_INVERT(mixedList) } // inverts both: one, b, two

```

List content queries extend too:

```

{mixedList ? (a,b) }      // false
{mixedList ^ LIST_ALL(a)} // a, c

{ mixedList >= (one, a) } // true
{ mixedList < (three) }   // false

```

Lists to track knowledge

One of the most powerful applications of ink lists is to track “knowledge” in the game – what has the player learned, or discovered, about the game’s various mysteries? What tasks have they completed? What plots have they moved forward?

We tend to use a particular design we call a “knowledge chain” – a collection of small state machines, each built with the assumption that every state includes, implicitly, all the states that come before it. So we might have something like

```

LIST WhoMurderedTheButler = KNOW_BUTLER_DEAD,
BELIEVE_BUTLER_MURDERED, KNOW_BUTLER_MURDERED,
KNOW_BUTLER_STABBED, ACCUSED_THE_OTHER_BUTLER,
SECURED_OTHER_BUTLER_CONFESION

```

When an event occurs or a piece of knowledge discovered, we record it (and all the preceding states in that chain) into a game-wide knowledge variable as follows:

```

VAR AllTrueStates = ()

=== function reach(statesToSet)
  ~ temp x = pop(statesToSet)

```

```

{
- not x:
~ return false

- not reached(x):
~ temp chain = LIST_ALL(x)
~ temp statesGained = LIST_RANGE(chain,
LIST_MIN(chain), x)
~ AllTrueStates += statesGained
~ reach (statesToSet) // set any other states left
~ return true // and we set this state, so true

- else:
~ return false || reach(statesToSet)
}

```

The function recurses through the list of states provided to it, and returns true if any are being set for the first time.

We test what the player has so far encountered, we provide two functions, `reached` and `between` – asking “have we reached this state”, and “are we between having reached this state and having not reached *any* of this other state” respectively. (There are other queries you might need, but these two are surprisingly applicable.)

```

=== function reached(x)
~ return AllTrueStates ? x

=== function between(x, y)
~ return AllTrueStates ? x && not (AllTrueStates ^ y)

```

So far, so straightforward. The elegance of this system begins emerge once starts considered events across multiple chains of knowledge, because we can start to ask broad, unspecific questions, like “does the player know the butler is dead, but not how it happened?”

```

{ between(KNOW_BUTLER_DEAD, ( KNOW_BUTLER_STABBED ,
SEEN_BLOOD_ON_KNIFE)):
  There are bloodstains on this knife.
  { reach(KNOW_BUTLER_MURDERED):
    The butler must have been stabbed!
  - else:
    The butler must have been murdered!

```

```
    }  
    ~ reach((KNOW_BUTLER_STABBED, SEEN_BLOOD_ON_KNIFE))  
}
```

And we can jump forward in the plot:

```
~ reach(SECURED_OTHER_BUTLER_CONFESION)  
OTHER BUTLER:    All right! I confess! It was me! I  
stabbed him!
```

But best of all, we can add detail retrospectively: if we later decide we need to add a state that fits within the chain (say, `SUSPECT THE OTHER_BUTLER`, between `KNOW_BUTLER_STABBED` and `ACCUSED_THE_OTHER_BUTLER`) we can simply add it to the knowledge chain and all of the tests in the game will still work exactly as expected. We can go the other way as well; if we decide there should be two separate chains – one for having discovered the murder, and one for suspecting the other butler (and perhaps others for suspecting other characters too!), then we can split the one chain into two:

```
LIST WhatHappenedToTheButler = KNOW_BUTLER_DEAD,  
BELIEVE_BUTLER_MURDERED, KNOW_BUTLER_MURDERED,  
KNOW_BUTLER_STABBED  
  
LIST SuspectingTheOtherButler = ACCUSED_THE_OTHER_BUTLER,  
SECURED_OTHER_BUTLER_CONFESION
```

... and again, all the tests in the game continue to work without any need to update them.

8. Long example: crime scene

Finally, here's a long example, demonstrating a lot of ideas from this section in action. You might want to try playing it before reading through to better understand the various moving parts. (It's included in the docs inside inky, and you can paste the ink directly from there.)

```
-> murder_scene

// Helper function: popping elements from lists
=== function pop(ref list)
  ~ temp x = LIST_MIN(list)
  ~ list -= x
  ~ return x

//
// System: items can have various states
// Some are general, some specific to particular items
//

LIST OffOn = off, on
LIST SeenUnseen = unseen, seen

LIST GlassState = (none), steamed, steam_gone
LIST BedState = (made_up), covers_shifted, covers_off,
bloodstain_visible

//
// System: inventory
//

LIST Inventory = (none), cane, knife

=== function get(x)
  ~ Inventory += x

//
// System: positioning things
// Items can be put in and on places
```

```

//

LIST Supporters = on_desk, on_floor, on_bed, under_bed,
held, with_joe

=== function move_to_supporter(ref item_state,
new_supporter) ===
~ item_state -= LIST_ALL(Supporters)
~ item_state += new_supporter

// System: Incremental knowledge.
// Each list is a chain of facts. Each fact supersedes
the fact before
//

VAR knowledgeState = ()

=== function reached (x)
~ return knowledgeState ? x

=== function between(x, y)
~ return knowledgeState? x && not (knowledgeState ^ y)

=== function reach(statesToSet)
~ temp x = pop(statesToSet)
{
- not x:
~ return false

- not reached(x):
~ temp chain = LIST_ALL(x)
~ temp statesGained = LIST_RANGE(chain,
LIST_MIN(chain), x)
~ knowledgeState += statesGained
~ reach (statesToSet) // set any other states left to
set
~ return true          // and we set this state, so true

- else:
~ return false || reach(statesToSet)

```

```

}

//
// Set up the game
//

VAR bedroomLightState = (off, on_desk)

VAR knifeState = (under_bed)


//
// Knowledge chains
//


LIST BedKnowledge = neatly_made, crumpled_duvet,
hastily_remade, body_on_bed, murdered_in_bed,
murdered_while_asleep

LIST KnifeKnowledge = prints_on_knife,
joe_seen_prints_on_knife, joe_wants_better_prints,
joe_got_better_prints

LIST WindowKnowledge = steam_on_glass,
fingerprints_on_glass, fingerprints_on_glass_match_knife


//
// Content
//

=== murder_scene ===

The bedroom. This is where it happened. Now to look for
clues.

- (top)
  { bedroomLightState ? seen:      <- seen_light  }
  <- compare_prints(-> top)

```

```

*   (dobed) [The bed...]
    The bed was low to the ground, but not so low
    something might not roll underneath. It was still neatly
    made.
    ~ reach (neatly_made)
    - - (bedhub)
    * *   [Lift the bedcover]
          I lifted back the bedcover. The duvet
    underneath was crumpled.
          ~ reach (crumpled_duvet)
          ~ BedState = covers_shifted
    * *   (uncover) {reached(crumpled_duvet)}
          [Remove the cover]
          Careful not to disturb anything beneath, I
    removed the cover entirely. The duvet below was rumped.
          Not the work of the maid, who was conscientious
    to a point. Clearly this had been thrown on in a hurry.
          ~ reach (hastily_remade)
          ~ BedState = covers_off
    * *   (duvet) {BedState == covers_off} [ Pull back the
    duvet ]
          I pulled back the duvet. Beneath it was a sheet,
    sticky with blood.
          ~ BedState = bloodstain_visible
          ~ reach (body_on_bed)
          Either the body had been moved here before being
    dragged to the floor - or this is was where the murder
    had taken place.
    * *   {BedState != made_up} [ Remake the bed ]
          Carefully, I pulled the bedsheets back into
    place, trying to make it seem undisturbed.
          ~ BedState = made_up
    * *   [Test the bed]
          I pushed the bed with spread fingers. It creaked
    a little, but not so much as to be obnoxious.
    * *   (darkunder) [Look under the bed]
          Lying down, I peered under the bed, but could
    make nothing out.

    * *   {TURNS_SINCE(-> dobod) > 1} [Something else?]
          I took a step back from the bed and looked

```

around.

-> top

- - -> bedhub

* {darkunder && bedroomLightState ? on_floor &&
bedroomLightState ? on}

[Look under the bed]

I peered under the bed. Something glinted back at
me.

- - (reaching)

* * [Reach for it]

I fished with one arm under the bed, but
whatever it was, it had been kicked far enough back that
I couldn't get my fingers on it.

-> reaching

* * {Inventory ? cane} [Knock it with the cane]

-> knock_with_cane

* * {reaching > 1 } [Stand up]

I stood up once more, and brushed my coat
down.

-> top

* (knock_with_cane) {reaching && TURNS_SINCE(->
reaching) >= 4 && Inventory ? cane } [Use the cane to
reach under the bed]

Positioning the cane above the carpet, I gave the
glinting thing a sharp tap. It slid out from the under
the foot of the bed.

~ move_to_supporter(knifeState, on_floor)

* * (standup) [Stand up]

Satisfied, I stood up, and saw I had knocked
free a bloodied knife.

-> top

* * [Look under the bed once more]

Moving the cane aside, I looked under the bed
once more, but there was nothing more there.

-> standup

* {knifeState ? on_floor} [Pick up the knife]


```

    Careful not to touch the handle, I lifted the blade
    from the carpet.
    ~ get(knife)

    *    {Inventory ? knife} [Look at the knife]
        The blood was dry enough. Dry enough to show up
        partial prints on the hilt!
        ~ reach (prints_on_knife)

    *    [    The desk... ]
        I turned my attention to the desk. A lamp sat in
        one corner, a neat, empty in-tray in the other. There was
        nothing else out.
        Leaning against the desk was a wooden cane.
        ~ bedroomLightState += seen

        - - (deskstate)
        * *    (pickup_cane) {Inventory !? cane} [Pick up
the cane ]
            ~ get(cane)
            I picked up the wooden cane. It was heavy,
            and unmarked.

        * *    { bedroomLightState !? on } [Turn on the
lamp]
            -> operate_lamp ->

        * *    [Look at the in-tray ]
            I regarded the in-tray, but there was nothing
            to be seen. Either the victim's papers were taken, or his
            line of work had seriously dried up. Or the in-tray was
            all for show.

        + +    (open) {open < 3} [Open a drawer]
            I tried {a drawer at random|another drawer|a
            third drawer}. {Locked|Also locked|Unsurprisingly, locked
            as well}.

        * *    {deskstate >= 2} [Something else?]
            I took a step away from the desk once more.
            -> top

```

- - -> deskstate

* {(Inventory ? cane) && TURNS_SINCE(-> deskstate)
<= 2} [Swoosh the cane]

I was still holding the cane: I gave it an experimental swoosh. It was heavy indeed, though not heavy enough to be used as a bludgeon.

But it might have been useful in self-defence. Why hadn't the victim reached for it? Knocked it over?

* [The window...]

I went over to the window and peered out. A dismal view of the little brook that ran down beside the house.

- - (window_opts)

<- compare_prints(-> window_opts)

* * (downy) [Look down at the brook]
{ GlassState ? steamed:

Through the steamed glass I couldn't see the brook. -> see_prints_on_glass -> window_opts
}

I watched the little stream rush past for a while. The house probably had damp but otherwise, it told me nothing.

* * (greasy) [Look at the glass]
{ GlassState ? steamed: -> downy }

The glass in the window was greasy. No one had cleaned it in a while, inside or out.

* * { GlassState ? steamed && not
see_prints_on_glass && downy && greasy }
[Look at the steam]

A cold day outside. Natural my breath should steam. -> see_prints_on_glass ->

+ + {GlassState ? steam_gone} [Breathe on the
glass]

I breathed gently on the glass once more. {
reached (fingerprints_on_glass): The fingerprints
reappeared. }

~ GlassState = steamed

```

+ +   [Something else?]
      { window_opts < 2 || reached
(fingerprints_on_glass) || GlassState ? steamed:
      I looked away from the dreary glass.
      {GlassState ? steamed:
        ~ GlassState = steam_gone
        <> The steam from my breath faded.
      }
      -> top
    }
    I leant back from the glass. My breath had
steamed up the pane a little.
    ~ GlassState = steamed

- -   -> window_opts

*   {top >= 5} [Leave the room]
    I'd seen enough. I {bedroomLightState ?
on:switched off the lamp, then} turned and left the room.
    -> joe_in_hall

-   -> top

= operate_lamp
  I flicked the light switch.
  { bedroomLightState ? on:
    <> The bulb fell dark.
    ~ bedroomLightState += off
    ~ bedroomLightState -= on
  - else:
    { bedroomLightState ? on_floor: <> A little light
spilled under the bed.} { bedroomLightState ? on_desk :
<> The light gleamed on the polished tabletop. }
    ~ bedroomLightState -= off
    ~ bedroomLightState += on
  }
  ->->

```

```
= compare_prints (-> backto)
  * { between ((fingerprints_on_glass, prints_on_knife),
fingerprints_on_glass_match_knife) }
    [Compare the prints on the knife and the window ]
    Holding the bloodied knife near the window, I
breathed to bring out the prints once more, and compared
them as best I could.
    Hardly scientific, but they seemed very similar -
very similiar indeed.
    ~ reach (fingerprints_on_glass_match_knife)
    -> backto
```

```
= see_prints_on_glass
  ~ reach (fingerprints_on_glass)
  {But I could see a few fingerprints, as though someone
hadpressed their palm against it.|The fingerprints were
quite clear and well-formed.} They faded as I watched.
  ~ GlassState = steam_gone
  ->->
```

```
= seen_light
  * {bedroomLightState !? on} [ Turn on lamp ]
    -> operate_lamp ->

  * { bedroomLightState !? on_bed  && BedState ?
bloodstain_visible }
    [ Move the light to the bed ]
    ~ move_to_supporter(bedroomLightState, on_bed)
```

I moved the light over to the bloodstain and peered closely at it. It had soaked deeply into the fibres of the cotton sheet.

There was no doubt about it. This was where the blow had been struck.

```
    ~ reach (murdered_in_bed)
```

```
  * { bedroomLightState !? on_desk } {TURNS_SINCE(->
floorit) >= 2 }
    [ Move the light back to the desk ]
    ~ move_to_supporter(bedroomLightState, on_desk)
    I moved the light back to the desk, setting it down
```

where it had originally been.

```
* (floorit) { bedroomLightState !? on_floor &&
darkunder }
  [Move the light to the floor ]
  ~ move_to_supporter(bedroomLightState, on_floor)
  I picked the light up and set it down on the floor.
```

```
- -> top
```

=== joe_in_hall

My police contact, Joe, was waiting in the hall. 'So?' he demanded. 'Did you find anything interesting?'

```
- (found)
  * {found == 1} 'Nothing.'
    He shrugged. 'Shame.'
    -> done
  * { Inventory ? knife } 'I found the murder weapon.'
    'Good going!' Joe replied with a grin. 'We thought
the murderer had gotten rid of it. I'll bag that for you
now.'
    ~ move_to_supporter(knifeState, with_joe)

  * {reached(prints_on_knife)} { knifeState ? with_joe }
    'There are prints on the blade[.'],' I told him.
    He regarded them carefully.
    'Hrm. Not very complete. It'll be hard to get a
match from these.'
    ~ reach (joe_seen_prints_on_knife)
  * { reached((fingerprints_on_glass_match_knife,
joe_seen_prints_on_knife)) }
    'They match a set of prints on the window, too.'
    'Anyone could have touched the window,' Joe replied
thoughtfully. 'But if they're more complete, they should
help us get a decent match!'
    ~ reach (joe_wants_better_prints)
  * { between(body_on_bed, murdered_in_bed)}
    'The body was moved to the bed at some point[.'],' I
told him. 'And then moved back to the floor.'
    'Why?'
    * *      'I don't know.'
```

```

Joe nods. 'All right.'
* *      'Perhaps to get something from the floor?'
          'You wouldn't move a whole body for that.'
* *      'Perhaps he was killed in bed.'
          'It's just speculation at this point,' Joe
remarks.
* { reached(murdered_in_bed) }
  'The victim was murdered in bed, and then the body
was moved to the floor.'
  'Why?'
* *      'I don't know.'
          Joe nods. 'All right, then.'
* *      'Perhaps the murderer wanted to mislead us.'
          'How so?'
* * *    'They wanted us to think the victim was
awake[.'], I replied thoughtfully. 'That they were
meeting their attacker, rather than being stabbed in
their sleep.'
* * *    'They wanted us to think there was some
kind of struggle[.'],' I replied. 'That the victim wasn't
simply stabbed in their sleep.'
- - -    'But if they were killed in bed,
that's most likely what happened. Stabbed, while
sleeping.'
          ~ reach (murdered_while_asleep)
* *      'Perhaps the murderer hoped to clean up the
scene.'
          'But they were disturbed? It's possible.'

* { found > 1} 'That's it.'
  'All right. It's a start,' Joe replied.
  -> done
- -> found
- (done)
{
- between(joe_wants_better_prints,
joe_got_better_prints):
  ~ reach (joe_got_better_prints)
  <> 'I'll get those prints from the window now.'
- reached(joe_seen_prints_on_knife):
  <> 'I'll run those prints as best I can.'

```

```
- else:  
    <> 'Not much to go on.'  
}  
-> END
```

9. Summary

To summarise a difficult section, ink's list construction provides for:

Flags

- Each list entry is an event
- Use `+=` to mark an event as having occurred
- Test using `?` and `!?`

```
LIST GameEvents = foundSword, openedCasket, metGorgon
```

```
{ GameEvents ? openedCasket }  
{ GameEvents ? (foundSword, metGorgon) }  
~ GameEvents += metGorgon
```

State machines

- Each list entry is a state
- Use `=` to set the state; `++` and `--` to step forward or backward
- Test using `==`, `>` etc

```
LIST PancakeState = ingredients_gathered, batter_mix,  
pan_hot, pancakes_tossed, ready_to_eat
```

```
{ PancakeState == batter_mix }  
{ PancakeState < ready_to_eat }  
~ PancakeState++
```

Properties

- Each list is a different property, with values for the states that property can take (on or off, lit or unlit, etc)
- Change state by removing the old state, then adding in the new
- Test using `?` and `!?`


```

LIST OnOffState = on, off
LIST ChargeState = uncharged, charging, charged

VAR PhoneState = (off, uncharged)

*   {PhoneState !? uncharged } [Plug in phone]
    ~ PhoneState -= LIST_ALL(ChargeState)
    ~ PhoneState += charging
    You plug the phone into charge.
*   { PhoneState ? (on, charged) } [ Call my mother ]

```

Knowledge chains

- Each list entry represents an idea or event that builds on the entry before
- Use a function to set the state and all states below
- Test using functions (reached, between)

```

LIST Harry = HEARD_OF_HARRY_LIME, SPOTTED_HARRY_LIME,
MET_HARRY_LIME
LIST Travelling = GONE_TO_PRAGUE, GONE_TO_APARTMENT

*   {between(HEARD_OF_HARRY_LIME, GONE_TO_PRAGUE)}
    [ Leave for Warsaw ] -> warsaw

*   {between(GONE_TO_APARTMENT, HEARD_OF_HARRY_LIME)}
    [ Read the name plate ]
    "Harry Lime"
    ~ reach (HEARD_OF_HARRY_LIME )

```

International Character Support

By default, ink has no limitations on the use of non-ASCII characters inside the story content (including emojis; knock yourself out face). However, a limitation currently exists on the characters that can be used for names of constants, variables, stitches, diverts and other named flow elements (a.k.a. the language's *identifiers*).

Sometimes it is inconvenient for a writer using a non-ASCII language to write a story because they have to constantly switch to naming identifiers in ASCII and then switching back to whatever language they are using for the story. In addition, naming identifiers in the author's own language could improve the overall readability of the raw story format.

In an effort to assist with this, ink automatically supports a list of pre-defined non-ASCII character ranges that can be used as identifiers. In general, those ranges have been selected to include the alpha-numeric subset of the official Unicode character range, which would suffice for naming identifiers. The below section lists the non-ASCII characters ink automatically supports.

Supported identifier characters

The support for the additional character ranges in ink is currently limited to a predefined set of character ranges.

Below is a listing of the currently supported identifier ranges.

Arabic

Enables characters for languages of the Arabic family and is a subset of the official *Arabic* unicode range `\u0600-\u06FF`.

Armenian

Enables characters for the Armenian language and is a subset of the official *Armenian* unicode range `\u0530-\u058F`.

Cyrillic

Enables characters for languages using the Cyrillic alphabet and is a subset of the official *Cyrillic* unicode range `\u0400-\u04FF`.

Greek

Enables characters for languages using the Greek alphabet and is a subset of the official *Greek and Coptic* unicode range `\u0370-\u03FF`.

Hebrew

Enables characters in Hebrew using the Hebrew alphabet and is a subset of the official *Hebrew* unicode range `\u0590-\u05FF`.

Latin Extended A

Enables an extended character range subset of the Latin alphabet - completely represented by the official *Latin Extended-A* unicode range \u0100-\u017F.

Latin Extended B

Enables an extended character range subset of the Latin alphabet - completely represented by the official *Latin Extended-B* unicode range \u0180-\u024F.

Latin 1 Supplement

Enables an extended character range subset of the Latin alphabet - completely represented by the official *Latin 1 Supplement* unicode range \u0080 - \u00FF.

NOTE! ink files should be saved in UTF-8 format, which ensures that the above character ranges are supported.

If a particular character range that you would like to use within identifiers isn't supported, feel free to open an issue or pull request on the main ink repo.

*Running
your Ink*

Getting Started

This chapter is written with a focus on using ink with Unity, but it's possible (and straightforward) to run your ink in a non-Unity C# environment. It's also easy to create ink games in JavaScript using inky's “export as web” feature, and the JavaScript framework, inkjs, supports most of the features detailed here.

There are also libraries for the Godot engine and a work-in-progress Unreal library.

1. Setting Up

Software

You can [download the latest version of the ink-unity-integration Unity package](#), and add to your project.

Select your `.ink` file in Unity, and you should see a *Play* button in the file's inspector. Click it, and you'll get an Editor window that lets you play (preview) your story.

Meanwhile, in the Window menu, you'll find the ink Player Window, a useful viewer for the ink state as the story runs. It can display variables, allow you to call ink functions, and watch the content as it is produced by the engine.

Loading in a story

ink uses an intermediate `.json` format, which is compiled from the original `.ink` files. This is treated by Unity as a `TextAsset`, that you can then load up in your game. ink's Unity integration package automatically compiles ink files for you.

The main runtime code is included in the `ink-engine.dll`.

We recommend that you create a wrapper `MonoBehaviour` component for the ink `Story`. Here, we'll call the component "Script" – in the "film script" sense, rather than the "Unity script" sense!

```
using Ink.Runtime;

public class Script : MonoBehaviour {
    // Set this file to your compiled json asset

    public TextAsset inkAsset;

    // The ink story that we're wrapping
    Story _inkStory;
```

The API for loading and running your story is very straightforward. Construct a new `Story` object, passing in the JSON string from the `TextAsset`.

```
using Ink.Runtime;

...

void Awake()
```

```
{  
  _inkStory = new Story(inkAsset.text);  
}
```

2. Running the content

Once your story object is created, you make calls to the story in a loop to make it progress. There are two stages – presenting content, and making choices.

Presenting content

To draw content from the engine you repeatedly call `Continue()`, which returns individual lines of string content, until the `canContinue` property becomes false. For example:

```
while (_inkStory.canContinue) {  
    Debug.Log (_inkStory.Continue ());  
}
```

A simpler way to achieve the above is through one call to `_inkStory.ContinueMaximally()`. However, in many stories it's useful to pause the story at each line, for example when stepping through dialogue. Also, in such games, there may be state changes that should be reflected in the UI, such as resource counters.

Making choices

When there isn't any more content, you should check to see whether there any choices to present to the player. To do so, use something like:

```
if( _inkStory.currentChoices.Count > 0 )  
{  
    for (int i = 0; i < _inkStory.currentChoices.Count;  
    ++i) {  
        Choice choice = _inkStory.currentChoices [i];  
        Debug.Log("Choice " + (i + 1) + ". " + choice.text);  
    }  
}
```

...and when the player provides input:

```
_inkStory.ChooseChoiceIndex (index);
```

And now you're ready to return to step 1, and present content again.

3. Content tags

Tags exist to add metadata to your game's content. They aren't visible to the player but are read in by the engine at the same time as the content itself is generated. Within ink, add a # character followed by any string content you want to pass over to the game. There are three main places where you can add these hash tags:

Line by line tags

One use case is for a graphic adventure that has different art for characters depending on their facial expression. So, you could do:

```
Passepartout: Really, Monsieur. #surly
```

To add more than one tag to a line, simply delimit them with more # characters:

```
Passepartout: Really, Monsieur. #surly  
#really_monsieur.ogg
```

On the game side, as you get content with `_inkStory.Continue()`, tags will be collected up in `_inkStory.currentTags`, a `List<string>`. In the above case with two elements: "surly", and "really_monsieur.ogg".

Tags for a line can be written above it, or on the end of the line:

```
# the first tag  
# the second tag  
This is the line of content. # the third tag
```

All of the above tags will be included in the `currentTags` list.

Knot tags

Any tags that you include at the very top of a knot:

```
=== Munich ==  
  # location: Germany  
  # overview: munich.ogg  
  # require: Train ticket  
  First line of content in the knot.
```

... are accessible by calling

```
_inkStory.TagsForContentAtPath("your_knot")
```

... which is useful for getting metadata from a knot before you actually want the game to go there.

Note that these tags will also appear in the `currentTags` list for the first line of content in the knot.

Global tags

Any tags provided at the very top of the main ink file are accessible via the `Story's globalTags` property, which also returns a `List<string>`. Any top level story metadata can be included there.

We suggest the following by convention, if you wish to share your ink stories publicly:

```
# author: Joseph Humfrey  
# title: My Wonderful Ink Story
```

Note that inky will use the title tag in this format as the `<h1>` tag in a story exported for web.

Advanced: Choice tags

Note that the syntax

```
* [A choice!] #spoken  
  A response.
```

... looks rather like it ought to create a choice with the `spoken` tag. In fact, it doesn't – the tag isn't applied to the choice at all, but rather to the content produced once the choice is taken. In practice, when the choice text is suppressed, that means the tag will be applied to a blank line of content that appears *after* the choice and before the response.

Choices can be tagged, but the tag has to go in the “choice part”:

```
* “We leave today!” #exciting [] I declared with  
surprise.
```

... which will apply the tag to both the choice and the output line, or else:

```
* “We leave today!”[ #exciting ] I declared with  
surprise. #surprising
```

... which, by putting the tag into the “suppressed part” of the choice text, indicates that the tag is choice-only.

For games where choice text is always suppressed, this makes tagging choices hopefully quite natural:

```
* [ Scream! #scream.jpg ]  
"Yikes!" cries one of the humans. "Is there a ghost in  
this room?"
```

On the code side the tags are stored in a `List<string>` on each choice object.

Advanced: Dynamic tags

In older versions of ink, tags were hard-coded strings, but with the release of version 1.1 they can be dynamic, which is to say, they are also ink, and can contain any inline features, such as variable substitution, shuffles, cycles, and function calls.

For instance, to create a generic way to start a conversation with a character in a game where tags are used to tell the game what art to show, perhaps we give every character 3 variants of a “not paying attention” graphic, which changes to looking up when the player engages them in conversation:

```
=== talkTo(character, -> conversation)  
* [ Talk to {character} #  
{character}_looking_away{~1|2|3}.jpg ]  
  {character}: Yes? #{character}_looking_up.jpg  
  -> conversation
```

Or perhaps we want to provide an audio sting on the first time we learn a certain fact, but not if we hear it again:

```
+ [ Look at the murder weapon ]  
  It's covered in blood! #{once: shocking_sting.mp3}
```

But then again, maybe we can learn the same fact in two different places, and only want the sting on the one we find first:

+ [Look at the murder weapon]
It's covered in blood! #{ playShockSting() }

+ [Peek at the murder weapon]
Blood? #{ playShockSting() }

```
=== function playShockSting()  
{once: shocking_sting.mp3}
```

4. Other API features

Saving and loading

To save the state of your story within your game, call:

```
string savedJson = _inkStory.state.ToJson();
```

...and then to load it again:

```
_inkStory.state.LoadJson(savedJson);
```

For more detail on saving and loading, see the section below.

Error handling

If you made a mistake in your ink that the compiler can't catch, then the story will throw an exception. To avoid this and get standard Unity errors instead, you can use an error handler that you should assign when you create your story:

```
_inkStory = new Story(inkAsset.text);

_inkStory.onError += (msg, type) => {
    if( type == Ink.ErrorType.Warning )
        Debug.LogWarning(msg);
    else
        Debug.LogError(msg);
};
```

Advanced: Using the compiler at runtime

Precompiling your stories is more efficient than loading `.ink` at runtime. That said, it's a useful approach in some situations and can be done with the following code:

```
// inkFileContents: linked TextAsset, or Resources.Load,
// or even StreamingAssets
var compiler = new Ink.Compiler(inkFileContents);
Ink.Runtime.Story story = compiler.Compile();
Debug.Log(story.Continue());
```

Note that if your story is broken up into several ink files using `INCLUDE`, you'll need to add a file handler to enable Unity to find the other files, as follows:

```
var compiler = new Ink.Compiler(inkFileContents, new
Compiler.Options
{
    countAllVisits = true,
    fileHandler = new
UnityInkFileHandler(Path.GetDirectoryName(inkAbsolutePath))
});

Ink.Runtime.Story story = compiler.Compile();
Debug.Log(story.Continue());
```

5. Engine usage and philosophy

In Unity, we recommend using your own component class to wrap `Ink.Runtime.Story`. The runtime ink engine has been designed to be reasonably general purpose and have a simple API. We also recommend wrapping rather than inheriting from `Story`, so that you can expose to your game only the functionality that you need.

Often when designing the flow for your game, the sequence of interactions between the player and the story may not precisely match the way the ink is evaluated. For example, with a classic choose-your-own-adventure type story, you may want to show multiple lines (paragraphs) of text and choices all at once. For a visual novel, you may want to display one line per screen.

Additionally, since the ink engine outputs lines of plain text, it can be effectively used for your own simple sub-formats. For example, for a dialog based game, you could write:

```
*   Lisa: Where did he go?
    Joe:  I think he jumped over the garden fence.
* *   Lisa: Let's take a look.
* *   Lisa: So he's gone for good?
```

As far as the ink engine is concerned, the `:` characters are just text. But as the lines of text and choices are produced by the game, you can do some simple text parsing of your own to turn the string `Joe: What's up?` into a game-specific dialog object that references the speaker and the text (or even audio).

This approach can be taken even further to text that flexibly indicates non-content directives. Again, these directives come out of the engine as text, but can be parsed by your game for a specific purpose:

```
PROPLIST table, chair, apple, orange
```

The above approach might be used for the writer to declare the props they expect to be in the scene. These could be picked up in the game editor in order to automatically fill a scene with placeholder objects, or just to validate that the level designer has populated the scene correctly.

To mark up content more explicitly, you may want to use *tags* or *external functions* – see below. At inkle, we find that we use a mixture, but we actually find the above approach useful for a very large portion of our interaction with the game – it's very flexible.

Controlling the Story

The wrapper around an ink story can be very simple, chunking line by line through the content and offering choices as appropriate. (The default web export from `inky` does this.)

However, if you want to add any additional features to the game's UI – a clock showing the current time, perhaps, or an inventory window representing what the player is holding – then you'll need ways to more tightly integrate the ink state and the game state.

The runtime contains a lot of different ways to manage where the story is, and what the current game state is.

1. Controlling the state

Setting/getting ink variables

The state of the variables in the ink engine is, appropriately enough, stored within the `variablesState` object within the `story`. You can both set:

```
_inkStory.variablesState["player_health"] = 100
```

and get variables this way, but you'll need to cast correctly on getting:

```
int health = (int) _inkStory.variablesState["player_health"]
```

Read and visit counts

To find out the number of times that a knot or stitch has been visited by the ink engine, you can use this API:

```
_inkStory.state.VisitCountAtPathString("...");
```

The path string is in the form `"yourKnot"` for knots, and `"yourKnot.yourStitch"` for stitches.

Variable observers

You can register a delegate function to be called whenever a particular variable changes. This can be useful to reflect the state of certain ink variables directly in the UI, so the game can reflect story changes immediately.

```
_inkStory.ObserveVariable ("health", (string varName,  
object newValue) => {  
    SetHealthInUI((int)newValue);  
});
```

The reason that the variable name is passed in is so that you can have a single observer function that observes multiple different variables, and if you want to do that, you might want to use a faster binding too. The API provides `ObserveVariables`, which takes a list of variable names.

```
ObserveVariables(IList<string> variableNames,  
VariableObserver observer)
```

Finally, you can deregister a variable observer should you need to:

```
RemoveVariableObserver(VariableObserver observer = null,  
string specificVariableName = null)
```

If the `observer` is `null`, then all observers are removed for the named variable; if the `specificVariableName` is `null`, then all instances of the `observer` are removed, and if both are `null`, then all the variable observers in the game are removed.

2. External functions

In general, we try to store as much of the game's core variables inside the ink story and use variable observers like this one to drive the game's UI systems, because that way saving and loading of state is handled automatically, and the story itself can query and respond to the state of gameplay level easily.

However, a story will often require the ability to query some element of the wider game state – or complete some kind of calculation or piece of string handling – that is beyond the reach of normal ink syntax.

In these cases, you can define game-side function in C# that can be called directly from ink. This is called an *external* function.

Defining an external function

Firstly, you declare an external function using something like this at the top of one of your ink files. (Like all ink functions, it will have a global scope.)

```
EXTERNAL playSound(soundName)
```

You'll also need an ink version of this function, so the compiler can understand what's going on (this is called a *fallback function*, more on them below.)

```
=== function playSound(soundName)
[ SOUND: {soundName}! ]
```

You then create a C# version of the function, and bind it to the ink-side version. For example:

```
_inkStory.BindExternalFunction ("playSound", (string
name) => {
    _audioController.Play(name);
});
```

You can then call that function within the ink:

```
~ playSound("whack")
```

The types you can use as parameters and return values are int, float, bool (automatically converted from ink's internal ints) and string.

Fallbacks for external functions

When testing your story, either in inky or in the ink-unity integration player window, you don't get an opportunity to bind a game function before running the story.

This is *fallback functions* are for: they're used if the `EXTERNAL` function can't be found. There's no special syntax: the fallback is simply the ink function with the same name and parameters. It might return a default value for a calculation, or a random value to simulate a gameplay condition, or print a text line to describe what would have happened had the function been encountered in game.

Binding overloads

There are convenience overloads for `BindExternalFunction`, to allow for up to four parameters, for both generic `System.Func` and `System.Action`. There is also a general purpose `BindExternalFunctionGeneral` that takes an object array for more than 4 parameters.

Example: getting the player's name

A common usage for external functions is when you want the player to input some text which the game can then use. ink has no native typing input, but thanks to external functions, it doesn't need one. Instead we use game code to collect the information, and request it from ink via a function.

```
VAR playerName = ""
~ playerName = getName()
Hello, {playerName}!EXTERNAL getName()

=== function getName()
    // this is a fallback used in inky
    ~ return "Seth, Destroyer of Worlds"
```

... and in C# we can do the actual work of letting the player type a name in, or pick it from a list, or whatever else we'd like to do.

```
_inkStory.BindExternalFunction ("getName", () => {
    var name = AskPlayerForNameAndGetResponse(); // some UI
    code
    return name;
}, false);
```

Alternatives to external functions

Remember that in addition to external functions, there are other good ways to communicate between your ink and your game:

You can set up a variable observer if you just want the game to know when some state has changed. This is perfect for say, changing the score in the UI.

You can use tags to add invisible metadata to a line in ink.

In inkle's games such as *Heaven's Vault*, we use the text itself to write instructions to the game, and then have a game-specific text parser decide what to do with it. This is a very flexible approach, and allows us to have a different style of writing on each project. For example, we use the following syntax to ask the game to set up a particular camera shot:

```
>>> SHOT: view_over_bridge
```

Advanced: Actions vs. pure functions

Warning: The following section is subtly complex! However, don't worry – you can probably ignore it and use default behaviour. If you find a situation where glue isn't working the way you expect and there's an external function in there somewhere, or if you're just plain curious, read on...

There are actually two kinds of external functions:

- **Actions** - for example, to play sounds, show images, etc. Generally, these may change game state in some way.
- **Pure functions** - those that don't cause side effects. Specifically, it should be harmless to call them more than once and they shouldn't affect game state. For example, a mathematical calculation, or pure inspection of game state.

By default, external functions are treated as Actions, since we think this is the primary use-case for most people. Both can return values! However, the distinction can be important for subtle reasons to do with the way that glue works.

When the engine looks at content, it may look ahead further than you would expect *just in case* there is glue in future content that would turn two separate lines into one.

However, external functions that are intended to be run as actions, you don't want them to be run prospectively, since the player is likely to notice, so for this kind we cancel any attempt to glue content together. If it was in the middle of prospectively looking ahead and it sees an action, it'll stop before running it. Conversely, if all you're doing is a mathematical calculation for example, you don't want your glue to break. For example:

```
The square root of 9  
~ temp x = sqrt(9)  
<> is {x}.
```

You can define how you want your function to behave when you bind it, using the `lookaheadSafe` parameter:

```
public void BindExternalFunction(string funcName,
Func<object> func, bool lookaheadSafe=false)
```

Actions should have `lookaheadSafe = false`
Pure functions should have `lookaheadSafe = true`

Example: UPPERCASE

ink doesn't have much in the way of string handling functions because we can instead rely on code. For instance, to transform a string to upper case:

```
EXTERNAL UPPERCASE()
=== function UPPERCASE (x)
    // this is a fallback used in inky
    ~ return "[{x}]"
```

On the C# side we write a simple function to perform the uppercase.

```
_inkStory.BindExternalFunction ("getName", (string x) =>
{
    return name.ToUpper();
}, true);
```

Note the `, true` – this is a lookahead-safe, pure function and it's useful to mark it as such because then ink can use it in lines of text involving glue: and for a function that transforms the text itself, that's very useful.

3. Working with LISTS

Lists are the most complex type used in the ink engine, so interacting with them is a bit more involved than with integers, floats and strings.

Lists always need to know the origin of their items. For example, in ink you can do:

```
~ myList = (Orange, House)
```

...even though `Orange` may have come from a list called `fruit` and `House` may have come from a list called `places`. In ink these *origin* lists are automatically resolved for you when writing. However when working in game code, you have to be more explicit, and tell the engine which origin lists your items belong to.

Creating and modifying lists

To create a list with items from a single origin, and assign it to a variable in the game:

```
var newList = new Ink.Runtime.InkList("fruit", story);  
newList.AddItem("Orange");  
newList.AddItem("Apple");  
  
story.variablesState["myList"] = newList;
```

If you're modifying a list, and you know that it has/had elements from a particular origin already:

```
var fruit = story.variablesState["fruit"] as  
Ink.Runtime.InkList;  
fruit.AddItem("Apple");
```

Note that single list items in ink, such as:

```
VAR lunch = Apple
```

...are actually just lists with single items in them rather than a different type. So to create them on the game side, just use the techniques above to create a list with just one item.

You can also create lists from items if you explicitly know all the metadata for the items - i.e. the origin name as well as the int value assigned to it. This is useful if you're building a list out of existing lists. Note that `InkLists` actually derive from `Dictionary`, where the key is an `InkListItem` (which in turn has `originName` and `itemName` strings), and the value is the int value:

```
var newList = new Ink.Runtime.InkList();
var fruit = story.variablesState["fruit"] as
Ink.Runtime.InkList;
var places = story.variablesState["places"] as
Ink.Runtime.InkList;
foreach(var item in fruit) {
    newList.Add(item.Key, item.Value);
}
foreach (var item in places) {
    newList.Add(item.Key, item.Value);
}
story.variablesState["myList"] = newList;
```

Querying lists from code

To test if your list contains a particular item:

```
fruit = story.variablesState["fruit"] as
Ink.Runtime.InkList;

if( fruit.ContainsItemNamed("Apple") ) {
    // We're eating apples tonight!
}
```

The list API also exposes many of the operations you can do in ink:

```
list.minItem // equivalent to calling LIST_MIN(list) in
ink
list.maxItem // equivalent to calling LIST_MAX(list) in
ink

list.inverse // equivalent to calling LIST_INVERT(list)
list.all     // equivalent to calling LIST_ALL(list) in
ink
```



```
list.Union(otherList)      // equivalent to (list +  
otherList)  
list.Intersect(otherList) // equivalent to (list ^  
otherList)  
list.Without(otherList)   // equivalent to (list -  
otherList)  
list.Contains(otherList)  // equivalent to (list ?  
otherList)
```

Controlling the Flow

By default an ink story starts from the top of the main file and then follows ink diverts until the story reaches an `-> END`. But there are plenty of other ways to want to use ink – as a database of content, as a collection of different scenes or episodes or locations, or even as a way of driving menu text.

By controlling the flow and executing functions from the game side we can do both these things.

1. Controlling the flow

The most basic control is to cause ink to begin at a named knot. This can be good for debugging, but it can also be the way ink is used – each location in *Heaven's Vault* begins at a knot which the game jumps to on landing.

(Note that we recommend putting all these knots into one big ink file, not making a new ink story for each location – so that one part of the story can query and affect another!)

Starting from a named knot

Top level named sections in ink can be jumped to directly from the engine.

```
_inkStory.ChoosePathString("myKnotName");
```

You then call `Continue()` to begin processing the story from this point.

To jump directly to a stitch within a knot, use a `.` as a separator:

```
_inkStory.ChoosePathString("myKnotName.theStitchWithin");
```

Note that this path string is a *runtime* path rather than the path as used within the ink format. It's just been designed so that for the basics of knots and stitches, the format works out the same. Unfortunately, however, you can't reference gather or choice labels this way.

2. Executing ink functions

ink functions can be invoked from the game-side without interrupting the story flow or losing the current story position. The result returned by the function needs to be cast to the right type to be used in C#.

Evaluate function

```
(int)
inkStory.EvaluateFunction("calculateInventoryWeight");
```

There are various overloads, allowing you to send in parameters (the types must match those expected by ink, or there will be a runtime error); and allowing you to receive text output from the function into an `out` parameter. (Note that the parameter will need to be casted before it is used in C#.)

```
var weightDescription = "";

var totalWeight = (int)
_inkStory.EvaluateFunction("calculateInventoryWeight",
out weightDescription, true);
```

Note this means than an ink function called in this way can generate *both* a return value and a line of text: in the example above, the ink calculates the weight of the player's inventory, but also returns a text string to use as a line of dialogue for the player to say about this weight they're lugging around with them.

Callstack depth

You can query the depth of the current call-stack (which tunnels and function calls contribute to, but not diverts) using `story.state.callstackDepth` – in particular, this is useful for knowing how many tunnel levels deep the story currently is.

Example: tunnelling out of all tunnels

In ink:

```
EXTERNAL tunnelDepth()
```

```
=== function tunnelDepth()  
  [ Tunnel Depth not supported in inky ]  
  ~ return 1
```

In C#:

```
inkStory.BindExternalFunction("tunnelDepth", () => {  
  return inkStory.state.callstackDepth;  
});
```

In particular, you can use this to allow ink to “tunnel out of all tunnels”, which can be a sensible thing to do before saving the game. (See the upcoming section on [Migrating Saves Between Versions](#) for a note on *why* it’s sensible.)

```
=== tunnelOutThenDone  
{ tunnelDepth() > 1:  
  [ Tunnelling out! ]  
  ->-> tunnelOutThenDone  
}  
-> DONE
```

... or alternatively, “tunnel out of all tunnels then divert to here”, which can be useful to jump out of a deep conversation, say, and get on with the story.

```
=== tunnelOut(-> thenGoTo)  
{ tunnelDepth() > 1:  
  [ Tunnelling out! ]  
  ->-> tunnelOut(thenGoTo)  
}  
-> thenGoTo
```

3. Multiple story flows

It is possible to have multiple parallel “flows” running at once. They share the same global variable state, but have different story locations. The story processes whichever flow is currently selected by the runtime, but you can switch from one flow to another in code seamlessly.

This allows for the following examples:

- A background conversation between two characters while the protagonist talks to someone else. The protagonist could then leave and interject in the background conversation.
- Non-blocking interactions: you could interact with an object which generates a bunch of choices, but “pause” them, and go and interact with something else. The original object's choices won't block you from interacting with something new, and you can resume them later.
- Multiple parallel timelines which the player switches between using an UI layer command.

Flow API

The API calls are as follows:

`story.SwitchFlow("Your flow name")` – create a new Flow context, or switch to an existing one. The name can be anything you like, though you may choose to use the same name as an entry knot that you would go on to choose with `story.ChoosePathString("knotName")`.

`story.SwitchToDefaultFlow()` – before you start switching Flows there's an implicit default Flow. To return to it, call this method.

`story.RemoveFlow("Your flow name")` – destroy a previously created Flow. If the Flow is already active, it returns to the default flow.

`story.aliveFlowNames` – the names of currently alive flows. A flow is alive if it's previously been switched to and hasn't been destroyed. Does not include default flow.

`story.currentFlowIsDefaultFlow` – true if the default flow is currently active. By definition, will also return true if not using flow functionality.

`story.currentFlowName` – a string containing the name of the currently active flow. May contain internal identifier for default flow, so use `currentFlowIsDefault` to check first.

Production Features

Once your game is made, you'll want to polish it up and release it! The ink runtime contains some features to help you ensure your story script is running efficiently and not causing frame drops in your game.

But no game is ever quite finished – there's always another bug to fix – and interactive stories are notorious for needing a lot of post-release fixes, for typos, continuity errors, and, most seriously of all, missed opportunities for jokes. ink contains features to ensure that saves made in one version can be loaded in the next, but these features aren't magic and there are cases where they'll fail, so it's important to understand how they work and what they can and can't cope with.

Finally, many developers will want to localise their game into multiple languages. ink doesn't have any built-in tools for this but there are a few strategies that developers have successfully used.

1. Profiling and Optimising

In the normal course of things ink computation is fast and computationally light. ink can evaluate story logic and compile large numbers of choices without an impact of frame-rate.

As stories get larger and more complex – involving large amounts of ink list processing, for example, or evaluating several hundred choices per turn, or looping and recursing thousands of times – the frame rate can start to be affected, usually as a result of garbage collection processes generated by ink's call-stack and parameter passing.

Profiling ink

The ink-unity integration includes a profiler for ink, that captures timing information which can help to identify slow ink functions and knots.

Optimising ink

- Expensive ink functions are best optimized by reimplementing them with external functions. This is particularly valuable for recursive functions dealing with large list parameters as it can save on multiple creations, allocations and removals of ink lists.
- Switch blocks with ink list variables get expensive when they have a lot of cases to test. It can be faster to convert the variable to a string and test that instead, as strings generate less garbage.

```
== function roomName(roomNameAsListItem)
  "{roomNameAsListItem}":
    "LOUNGE": Lounge
    "KITCHEN": Kitchen
  }
```

- ink often evaluates the same content repeatedly before finalising the results of the pass. Caching the results of expensive functions can speed up computation, but checking the cache has to occur on the game side. In practice that means an ink function that calls an external function that checks the cache, and if the cache doesn't exist, calls an ink function to do the computation.

2. Migrating Saves Between Versions

The story state can be saved and restored as detailed above:

```
string savedJson = _inkStory.state.ToJson();
_inkStory.state.LoadJson(savedJson);
```

But what happens if the story file has changed between the saved version, and the version of the game being loaded into? This will happen all the time during development, as the game is edited, but once a game has gone live we need the ability to update the story file to fix typos, story logic bugs and add extra content.

ink is designed to be able to patch the save file from version to version automatically without error, loading what it can, discarding data in the save file it can't interpret, and allowing new variables to be created as required. However, there are limitations to what it can manage – after all, if you deleted the entire contents of a story file and replaced it with another story, it wouldn't be reasonable to expect the game to cope!

So here's a little more detail on how saving and loading handles mismatches between versions.

Text

- Text changes that don't add additional gathers or choices have no impact. Fixing typos, adding new words, paragraphs, inline variations and conditional blocks is entirely safe.

Variables

- Variables in the old data which no longer exist are discarded. Since they're not in the new data, this loss should never be noticed.
- Missing variables new to the new story file are created as required, and take their default value – that's the value defined in the game's `VAR` statement.

Read counts

The basic logic for loading read counts is the same as for variables:

- Read counts for knots and stitches which no longer exist are silently discarded.
- Read counts for new knots and stitches are created with a value of zero.

Advanced: Not all read counts are saved!

However, there's a detail here. `ink` doesn't actually save the read counts of all the stitches and knots in the story; only the ones that are actually used by the story. That covers all once-only choices and any gathers or choices which are used in logical conditions in the story-file. This keeps the save data small and is invisible to the player – unless the new version of a story tests for something that the old story didn't. If this happens, then when the save file is loaded the read count for that knot will be zero even though the player might actually have seen it. So data loss is possible!

Advanced: Content addresses can and do change!

The second detail to know is that while stitches and knots with hand-authored names are reliable, most choices in the story are auto-named internally and the auto-naming system is very basic: it uses the name of the knot and an index number. This can mean that if you insert a new choice into a knot, it can change the indexes of the choices that follow, which can cause the save data to be mismatched with the story content. This is very difficult to avoid so it's important to ensure your content can cope with having a few choices being arbitrarily unavailable: make sure there's always a choice available, include a fallback choice if all the choices are conditional.

You can reduce the renumbering between versions as much as possible by trying to edit existing knots as little as possible:

- Instead of removing a choice you no longer want, add a `{false}` conditional to prevent it appearing.
- Break wide weaves into a series of threaded-in weaves, to reduce the number of choices impacted by a change.
- Instead of adding a new choice at the top of a weave, thread the new choice in.

Note that it's always safe to add a new `(name)` to a choice, because the name is actually applied to the output content of that choice, not the choice itself.

Current story position

The last loaded element in the save file is the current story position. If the story address no longer exists, `ink` will do its best to find the nearest matching address. That can get unpredictable.

The best strategy for truly reliable saving is to control where the game saves. If your game is structured using hub knots, you could save the game only at those hub points. If you want to save in middle of long conversations, you could divert to a “save” sub-stitch with a robust, not-going-to-change address name, then divert again to continue the story into more volatile waters.

Advanced: Saving inside tunnels

Another important note: saving inside tunnels is particularly dangerous. This is because `ink` saves the call-stack addresses but doesn't parse them until it needs to.

That means potentially the story could load successfully, continue for some time until hitting a tunnel out marker (->->), and then tunnel back to an address that doesn't exist. The ink flow would then die, and if the game has overwritten the initial save there's no way to recover the flow.

You can either avoid saving inside tunnels entirely – or be extra careful not to edit the flow *above* the start of a tunnel.

Don't panic!

A final note: we've used this system in three big and complicated games – *Heaven's Vault*, *Pendragon* and *Overboard!*, and, like, it's been fine.

3. Localisation

ink has no built-in support for localisation, and by its nature it's hard to set one up – there is no core database of the strings in an ink file, since the logic is embedded around the text.

Localisations have been done, however, usually by one of two core strategies.

Tag-based localisation

The core issue of localising the ink is providing a way to “map” strings in the source language to strings in the output language. The easiest, if most time-consuming, way to do this is by hand using the tagging system. By giving each line in the game a unique tag, it can then be associated to translated versions via a standard database format.

```
* [ Talk to Watson    #talk_to_watson ]  
  SHERLOCK: Well now, Watson?    #well_now_watson  
  WATSON:   Yes, Holmes?        #yes_holmes
```

The run-time can then ignore the content produced by the engine during gameplay, and simply use the tags.

This approach has advantages – as lines are moved around, they keep their tags with them, and so edits to other areas of the file won't cause knock-on consequences. Typos can be fixed safely as they don't alter the tags, and typos in other languages are simply updated in their database. It also allows for language switching mid-game: since all that needs to happen is a different output is chosen for each tag generated by the game.

However, this approach does make some core assumptions, in particular that every line of content *is* a line; it does not provide any solution for inline content variation or for procedural printing via functions. Further, recall that tags are generated per line of content, so glued content accumulates tags., so when gluing lines together thus:

```
This line    #this_line  
<> glues to this one    #glues_to_this_one
```

the output will require some deciphering by the game.

```
This line glues to this one    # this_line,  
glues_to_this_one
```

ink-file localisation

The second approach is to localise the entire ink file itself – literally, to copy the file and replace the source language words with words in the target language, and do this for each of the languages required. This allows the localiser – if they have sufficient skill! – to resolve issues with inline substitutions, to account for different grammatical rules in different languages, and to use the full feature set of ink. (Here at inkle we've not done much localisation, but this is the approach we've taken when we have.)

For example, here's a moment in the English version of *Overboard!*

```
= found_earring
VO:      Sure enough, there by the rail is one of my
earrings.
~ reach(KNOW_EARRING_ON_DECK)
~ show_npc(EarringOnDeck)

* [ Saunter over ]
~ stepTime(2)
V:  Easy does it now...

* [ Grab it now! ]
V:  Before anyone sees!
~ getOneEarring()
~ reach(GOT_EARRING_BACK)
~ remove(EarringOnDeck)
V:  Got it.

-      ->->
```

And here's the same moment in the Spanish version:

```
= found_earring
VO:      Efectivamente, el pendiente que me falta está
allí, cerca de la barandilla.
~ reach(KNOW_EARRING_ON_DECK)
~ show_npc(EarringOnDeck)

* [ Acercarse ]
~ stepTime(2)
V:  Con cuidado...
```

```
* [ ¡Cogerlo! ]  
V: ¡Antes de que nadie lo vea!  
~ getOneEarring()  
~ reach(GOT_EARRING_BACK)  
~ remove(EarringOnDeck)  
V: Te pillé.  
  
- ->->
```

The game then ships with one compiled story json per language and internally, it loads whichever one the player wants into the game's story object and processes it as normal. This does mean that swapping language mid-game is a harder thing to implement (as you have to copy across the variable state precisely), but swapping language while resetting the game is very fast and easy to implement.

The main advantage is power and flexibility – for example, in *Overboard!* we needed to rewrite the “number to words” function entirely because the Spanish rules for describing a number – while not especially complex – are very different from those in English (“two hundred and four” becomes “doscientos cuatros”, while “eighty two” is “ochenta y dos”). This presented no problem, because the structure of the English ink file didn't constrain the structure of the Spanish file.

The main disadvantage is maintainability: every fix or addition made to the ink after localisation has been done must be replicated across every language. This is more a production issue than anything else – it best places localisation after beta-testing, and even better after *release*.

Ink
Patterns

List Patterns

As the size of the [chapter on lists](#) indicates, they're one of ink's most powerful features, and they're definitely worth a bit more time. Here are some useful ways of working with lists.

1. Basic list tricks

Filtering a list

Lists have lots of uses: one of which is as a *list of things*, such as inventory or a collection of attributes a person has. Sometimes that's more information than we want, so here's a pattern for filtering the list to the parts we're interested in.

```
// a list of things we might or might not have with us
LIST Inventory = knife, fork, spoon, apple, banana, hat, gloves

// Categorise our items. These act as our filters.
VAR Cutlery = (knife, fork, spoon)
VAR Fruit = (apple, banana)
VAR Clothes = (hat, gloves)
VAR PointyThings = (knife, fork, spoon, banana)

// Do we have any fruit?
// Get all of what we have~ temp fruitInHand = Inventory ^
Fruit

// What should we hold under our coats as a pretend gun?
// Don't get all items; pick the "best" one
~ temp falsePistol = LIST_MAX(Inventory ^ PointyThings)

// What can we eat?
{ Inventory ^ Cutlery:
  - Cutlery: anything!
  - (spoon, fork): baked beans
  - (spoon, knife): Nutella
  - (knife, fork): roast beef
  - spoon: soup
  - fork: rice
  - knife: bread and butter
  - (): doughnuts
}
```

Printing a list as a sentence

The default printout for a list is its current values, comma-delimited, which looks horrible, to be honest.

To print a list as a comma-delimited list with an “and” no Oxford comma:

```
=== function listPrint(list)
  ~ temp element = LIST_MAX(list)
  ~ list -= element
  { list: {list} and} {last_element}
```

This produces output like

```
I like blue, purple and aquamarine.
```

This is the simplest list printing routine, but it's limited, and relies on ink's internal list-printing system which is really intended for debugging rather than player-facing output.

To do better, we need to develop a new technique.

2. List recursion

The most powerful way to working through a list is to use recursion – that is a function which calls itself, until it has run out of data to work on.

To recurse through a list, it's a good idea to first define a **pop** function:

```
=== function pop(ref list)
  ~ temp element = LIST_MIN(list)
  ~ list -= element
  ~ return element
```

This takes the lowest value from the list, altering it in the process, and returns that lowest element. If the list is empty, the empty list is returned.

Using recursion for a calculation

Here's a contrived example that uses the pop function to turn a list of binary flags into a total.

```
LIST BinaryNumbers = (one = 1), two = 2, (four = 4),
eight = 8
```

The binary number 0101 is {total(BinaryNumbers)}.

```
== function total(list)
  ~ temp el = pop(list)
  {el:
    ~ return LIST_VALUE(el) + total(list)
  }
  ~ return 0
```

Here's another, to count the number of entries in the list that contain a certain substring:

```
LIST DaysOfTheWeek = Monday, Tuesday, Wednesday,
Thursday, Friday, Saturday, Sunday

{ CountContaining (LIST_ALL(DaysOfTheWeek), "y")} days
contain a 'y'.
```

```

=== function CountContaining(list, substring)
  ~ temp el = pop(list)
  { el:
    ~ temp containsSubString = 0

    { "{el}" ? substring:
      ~ containsSubString = 1
    }
    ~ return containsSubString + CountContaining(list,
substring)
  }
  ~ return 0

```

Having had two silly examples, we can now use recursion to produce some really useful effects.

Printing a list using names

Our previous list printing routine works fine if all the list elements are named in proper English. But often they won't be – for instance, if any elements are more than a single word long!

To print out “nice” names we need to record them somewhere, and the easiest way to do this is to make a “database function”: a function which acts as a database and converts the list element into its screen-ready name. This also requires us to recurse through the list as we print it element by element, so our printing routine becomes a bit more complex. However, once written, the list printer is generic and will work on any list we throw at it.

```

LIST People = (ElizabethBennett), (DavidDarcy),
(JimBroadbent)

```

```

In the room stands {listPrint(People, -> nameOfPerson)}.

```

```

=== function listPrint(list, -> nameFunction)
  ~ temp person = pop (list)
  { nameFunction(person)}
  { list:
    { LIST_COUNT(list):
      - 1: <> and <>
      - else: <>, <>
    }
    {listPrint(list, nameFunction)}
  }

```

```

}

=== function nameOfPerson (who)
{ who:
- ElizabethBennett: Elizabeth Bennett
- DavidDarcy: David Darcy // he's probably called David
- JimBroadbent: Jim Broadbent // he's in everything
- else: {who} // by default, print the raw key
}

```

One other advantage of this system is that the printed name of an ink list element can change as the game progresses, by putting logic into the database function itself. (Mr Darcy might become Mr Bennett-Darcy, for example.)

Printing a list in a random order

To print in a random order, we need to recurse through it, pulling out a random element each time.

The easiest way is to define a `pop_random` function, which works the same way as the `pop` function only picks elements using `LIST_RANDOM`:

```

LIST Integers = One, Two, Three, Four, Five, Six, Seven,
Eight

```

```

One to eight? I prefer
{print_random(LIST_ALL(Integers))}.

```

```

=== function pop_random(ref list)
~ temp el = LIST_RANDOM(list)
~ list -= el
~ return el

```

```

=== function print_random(list)
~ temp el = pop_random(list)
{ el:
{el}
{ list:
{ LIST_COUNT(list):
- 1: <> and <>
- else: <>, <>

```

```

    }
    {print_random(list)}
  }
}

```

Printing the closest value of a list

We can print list entries by their associated value quite easily.

```

LIST Integers = One, Two, Three, Four, Five, Six, Seven,
Eight
The number {Integers(7)} is 7.

```

But what if not every value is represented in the list? Here's an example that uses a list to encode a set of adjectives for describing a game quantity in a friendly way. It uses recursion to search through a list for the element that's the best match – the same idea could be applied to any other kind of test condition.

```

LIST Adjectives = (no = 0), (one = 1), (two = 2), (some =
4), (several = 8), (many = 12)

```

```

VAR goldCoins = 0
~ goldCoins = RANDOM(1, 15)

```

```

I have {describe(goldCoins, Adjectives)} gold
coin{goldCoins != 1:s} ({goldCoins}).

```

```

=== function describe(value, adjList)
  ~ temp el = ()
  ~ describeIfBetterThan(value, adjList, el)
  ~ return el

```

```

=== function describeIfBetterThan(value, adjList, ref
bestAdjective)
  ~ temp newAdj = LIST_MIN(adjList)
  ~ adjList -= newAdj
  { newAdj:
    { abs(LIST_VALUE(newAdj) - value) <
abs(LIST_VALUE(bestAdjective) - value):

```

```
    ~ bestAdjective = newAdj
  }
  ~ describeIfBetterThan(value, adjList, bestAdjective)
}

=== function abs(x) // the "absolute" value, without a
sign
{ x < 0:
  ~ return -1 * x
}
~ return x
```

The pattern here – recursing through a list, and returning the best fit of some logical condition – is quite a common one, and can be adapted to any “find the best” test for LIST keys. (The game *Pendragon* does this to select which story-template level will be next placed on the map, scoring that decision with a wide range of metrics stored on both the ink and game side.)

3. Converting strings to list entries

There's no inbuilt conversion between a string and a list item, but by recursing through a list you can find a matching element. The code to do this can be slow, but it can be useful!

```
=== function stringToList(stringName, sourceList)
  ~ temp minElement = LIST_MIN(sourceList)
  {minElement:
    { stringName == "{minElement}":
      ~ return minElement
    }
    ~ return stringToList (stringName, sourceList -
minElement)
  }
  ~ return ()
```

Note this is very easy to do in game-code – there's a function in the API for it, and we can wrap it into an external function.

```
story.BindExternalFunction("STRING_TO_LIST", (string
itemKey) => {
  try
  {
    return InkList.FromString(itemKey, story);
  }
  catch
  {
    return new InkList();
  }
}, true);
```

However, if your game relies on this and you want to test in inky, you'll need the purely ink method as a fallback.

Loop Patterns

ink is an usual programming language in a lot of ways, one of which is that it doesn't have any native loop structures. But since it's a language built from gotos, it doesn't really need them, as they can be quickly (and flexibly built.)

1. Basic loops

A counting loop

A basic for-next loop looks like this:

```
~ temp count = 0
- (top_of_loop)
  Give me a {count}!
- (bottom_of_loop)
  ~ count++
  { count < 5:
    -> top_of_loop
  }
```

A recursive loop

We can achieve the same effect using functions as well. (Though it's worth noting this way is less efficient – every function call duplicates the call-stack and results in additional garbage collection by the runtime environment.)

```
=== function count(n)
  Give me a {count}!
  { count < 5:
    ~ count(n + 1)
  }
```

2. Loops of choices

Every time a choice is created, it takes a snapshot of the local variable state at the time of the choice's creation. That means you can use loops to efficiently create sets of choices – the same choice line can create multiple choices with their correct state attached.

```
Pick a number between 1 and 5?
```

```
~ temp count = 0
- (top_of_loop)
  <- choice(count)
- (bottom_of_loop)
  ~ count++
  { count < 5:
    -> top_of_loop
  }
  -> DONE

= choice(count)
+ [Give me a {count}!]
  Okay, you chose {count}!
  -> END
```

Example: a combination lock

Say we have a 4-digit safe to open. In a normal programming language we might make an array, and let the player fill it in element by element. But ink doesn't have any arrays!

So we might do this by using a loop of choices, and storing the results as a string:

```
VAR correctCode = "1066"

~ temp codeEntered = ""
~ temp codeLength = 0

- (top)
  {codeLength == 4:
    -> check_code
  }
```

```

~ temp i = 0
- (loop_top)
  { i <= 9:
    <- enter(i)
    ~ i++
    -> loop_top
  }
-> DONE

= enter(i)
+ [ Type {i} ]
  ~ codeLength++
  ~ codeEntered += "{i}"
  The safe display now reads: {codeEntered}.
  -> top

= check_code
{ correctCode == codeEntered:
  -> safe_opens
- else:
  ~ codeEntered = ""
  ~ codeLength = 0
  The safe clicks, and the display clears.
  -> top
}

= safe_opens
  Inside the safe is a pile of stuffed animals. Bizarre.
  -> END

```

Using a list to create choices

Here's how we might offer choices based on what the player is holding:

```

LIST Inventory = (knife), mirror, (axe)
-> secret_santa

=== secret_santa
  I've got to put something in the secret Santa machine.
  But what?

```

```

~ temp items = Inventory

- (top)
~ temp el = pop(Inventory)

{ el:
  <- item_choice(el)
  -> top
}

- (bottom_of_loop)
// a fallback in case you want to keep your stuff

*   [ Trim a fingernail; it'll do. ]
    I never liked Steve anyway.
    -> santa_done (())

= item_choice(item)
+   [ Put in the {item} ]
    That's a great idea!
    -> santa_done (item)

= santa_done(item)
  The Secret Santa machine wraps up the {item:
{item}|fingernail} and ties a bow on it. Too late to
change my mind now.
-> DONE

```

Timing Patterns

For advanced responsiveness in our story, it's useful to know not just what the player has seen – which `ink` tracks quite happily – but also *when* they saw it.

For this, we use the `TURNS_SINCE` function which tells us how many turns have elapsed since the content was seen (and -1 if it's never been seen.)

1. The timing of events

What have we seen?

In the normal course of things we can test if any knot has been seen by just using its name, but it can be useful to generalise this idea to any divert target.

```
=== function seen_ever (-> x)
  ~ return TURNS_SINCE(x) >= 0
```

(So we can, for example, store the knot which contains a character's conversation options and then ask, have we spoken to them?)

What did we just see?

We've already encountered the `came_from` function:

```
=== function came_from(-> x)
  ~ return TURNS_SINCE(x) == 0
```

... which can be used to determine if we reached the current point as the result of a particular choice or intro piece of content.

What did we see recently?

Also useful are `came_from`'s sister functions: `seen_very_recently` and `seen_last_turn`.

```
=== function seen_last_turn (-> x)
  ~ return TURNS_SINCE(x) <= 1 && seen_ever(x)
=== function seen_very_recently(-> x)
  ~ return TURNS_SINCE(x) <= 3 && seen_ever(x)
```

`seen_ever` is useful inside functions like these, giving us the equivalent of the usual `{knot_name}` conditional. `last turn` tests if the player saw a given piece of content just a moment ago., and `seen_very_recently` if it was within the last few turns.

We can also reverse that idea, to offer things only if they happened a while ago:

```
=== function seen_but_not_recently(-> x)
  ~ return seen_ever(x) && TURNS_SINCE(x) >= 8
```


2. The order of events

What did we seen when?

We can use the `TURNS()` function to give us a time-stamp at any moment in the game, which can be useful for storing up information about when things happened.

```
VAR turnedOnKettle = -1

- (opts)

+ {turnedOnKettle == -1} [ Turn on the kettle ]
  ~ turnedOnKettle = TURNS()
  You turn on the kettle.

+ [ Wait ]
  Ho de hum...

- { turnedOnKettle > 0 && TURNS() >= turnedOnKettle + 5:
  The kettle flicks off.
  -> boiled_kettle
  }

-> opts
```

What did we seen most recently?

```
=== function seen_more_recently_than(-> link, -> marker)
{ TURNS_SINCE(link) >= 0:
  { TURNS_SINCE(marker) == -1:
    ~ return true // you've never seen "marker"
  }
// did you see link fewer turns ago than marker?
  ~ return TURNS_SINCE(link) < TURNS_SINCE(marker)
}
~ return false // you've never seen "link"
```

Which can then be used to make decisions based on the *order* things have been seen in.

```
- (start_of_scene)
"Welcome!"

- (opts)
*   (cough) [Cough politely]
    I clear my throat.
    -> opts

*   { seen_more_recently_than(-> cough, ->
start_of_scene) }
    "Hello!"

+   { not_seen_more_recently_than(-> cough, ->
start_of_scene) }
    ["Hello!"]
    I try to speak, but I can't get the words out!
    -> opts
```

This allows for things like, options you can only perform once per scene, and options that only make sense if you tried a particular thing this scene.

Varying Choice Patterns

Choices are the core of the player's interaction with the world, and there's nowhere in a game where that's more painfully obvious than conversation. Conversations in games can be all too often robotic, repetitive and transactional, and it can require some solid trickery to shake them up so they can feel varied and surprising.

Here are some patterns for producing varied choices from a pool of possibilities, offering various trade-offs between extensibility, author control and flexibility. They can be used in any context, but they really shine in dialogue.

1. Varying choice text

We saw earlier that the simple construction

```
* "{~Hi|Howdy|Wotcha}!"
```

doesn't do what we want it to, because the shuffle block is evaluated twice: once when the choice is offered and again when it's chosen.

Stable, shuffled choice text

If we want to make variable choice text like this, the easiest way is to use a temporary variable:

```
~ temp greeting = "{~Hi|Howdy|Wotcha}!"  
* "{greeting}"
```

The value of the temporary variable is stored inside the choice when it's generated, so you can even do this safely inside a loop. (And if you're a programmer, this should be surprising!) So the following ink will generate three options, with different greetings, all of which will be correctly reflected in the content when chosen.

```
- (top)  
  ~ temp greeting = "{~Hi|Howdy|Wotcha}!"  
  { greeting:  
    * "{greeting}"  
    -> reply  
  }  
  {CHOICE_COUNT() < 3:  
    -> top  
  }  
  -> DONE
```

(Note that the choice is inside a conditional block to prevent ink going into weave mode, otherwise the loop would never happen. A block like this effectively creates an “inline thread”.)

Prevent choice repetition

Here's another version of that example, with more complex choice generation and a little extra code to prevent the same choice text being offered twice in the same turn (by using a text cache to store which options we've seen so far):

```
// create a cache, of what we've used so far
~ temp greetingsSoFar = ""

- (top)
// Our greetings can be more complex, and full of nested
shuffles
~ temp greeting = "{~Hi|Hello|{~Howdy|Wotcha|Pip-pip}
{~chum|mate|partner|friend|compadre}}!"

// One rule: offer the choice if we haven't offered it
yet
{ greetingsSoFar != greeting:
// record the choice
~ greetingsSoFar += greeting

// offer the choice
*   "{greeting}"
  -> reply
}

// loop a lot to test the system
{CHOICE_COUNT() < 10:
  -> top
}
-> DONE
```

Add a limited number of choices

Sometimes, if you're using threads to collect up choices from various places into the flow, you might want to control how many choices each thread can generate.

Say you're building a conversation out of serious and silly topics:

```
=== conversation
<- one_serious_topic
<- one_silly_topic
```

```
+ "Goodbye!"  
-> leave_chat
```

The easiest way to ensure a single topic is to use `CHOICE_COUNT`, and on each choice ask: have we added a choice yet, or not?

```
=== one_serious_topic  
~ temp x = CHOICE_COUNT()  
* { x == CHOICE_COUNT() } { other conditions? } Option  
A  
* { x == CHOICE_COUNT() } { other conditions? } Option  
B  
...  
  
=== one_silly_topic  
~ temp x = CHOICE_COUNT()  
* { x == CHOICE_COUNT() } { other conditions? } Option  
Bananas  
* { x == CHOICE_COUNT() } { other conditions? } Option  
Asparagus  
...
```

The above is a little over-engineered, since in the current flow the value of `x` in `one_serious_topic` can't be anything but zero. But putting the same check there allows us to vary the order of serious and silly topics, should we want to. Note that if you want to allow each thread to offer more than 1 choice, the logic gets confusing enough that it's worth separating it into a function.

```
=== function only(startingChoices, choiceLimit)  
// How many choices have you added so far; is there room  
for another?  
~ return CHOICE_COUNT() - startingChoices < choiceLimit  
  
=== two_silly_topics  
~ temp x = CHOICE_COUNT()  
* { only(x, 2) } { other conditions? } Option Bananas  
...
```

Add a varying number of choices

Building on the above, we can move the parameters that determine how many choices are used to the main conversation hub itself:

```
=== conversation
  <- serious_topics(CHOICE_COUNT(), 3)
  <- silly_topics(CHOICE_COUNT(), 1)
  + "Goodbye!"
  -> leave_chat

= serious_topics(x, limit)
* {only(x, limit)} {other conditions?} Choice A
* {only(x, limit)} {other conditions?} Choice B
...

= silly_topics(x, limit)
* {only(x, limit)} {other conditions?} Choice Pedoodle
...
```

... which removes repetitive code when you have a lot of topic blocks, but also lets you “balance” the conversation based on varying factors. For example, perhaps the evening starts off as fun and games and turns rather more serious...

```
=== conversation
  ~ temp numSerious = RANDOM( 1 , 2 )
  {
    - totalNumberOfBodiesFound >= 2:
      ~ numSerious = 3
    - totalNumberOfBodiesFound >= 1:
      ~ numSerious++
  }

  <- serious_topics(CHOICE_COUNT(), numSerious)
  <- silly_topics(CHOICE_COUNT(), 4 - numSerious)

  + "Goodbye!"
  -> leave_chat
```

2. Shuffling choices

Offering lots of shuffled choices without repetition is a common problem, and usually the options aren't all just the same one in disguise. One approach is to use a LIST to track which options have appeared this turn.

Varying options using a list

```
LIST CluesKnown = (Key), BloodyKnife, ScrapOfWhiteCloth

+   [Ask Ernie a question]
-   (askquestion)

    ~ temp timesLooped = 0

-   (loop)
    { shuffle:
      -   { offer_answer(Key): <- ask_about_key}
      -   { offer_answer(BloodyKnife): <- ask_about_knife}
      -   { offer_answer(ScrapOfWhiteCloth): <-
ask_about_cloth}
    }

    ~ timesLooped++

// don't loop forever; and max out at two choices.
{timesLooped < 10 && CHOICE_COUNT() < 2: -> loop }

// closing choice offered after the others
+   {came_from(-> next)} "I've asked enough for now."
    Ernie tugs his collar. "Glad to 'ear it."
-> done

// fallback, in case there's nothing to say
+   ->
    "Are you well?"
    "Perfick."
-> done
```


- (next) -> askquestion
- (done) -> END

The choices are only offered if the `offer_answer` function allows them to be; and this function uses a list to keep track of what's already gone up.

```
==== function offer_answer(answerKey)
  VAR answersOffered = ()

  { not came_from(-> note_answer):
    ~ answersOffered = ()
  }
  { answersOffered ? answerKey:
    ~ return false
  }
  ~ return note_answer(answerKey)

=== function note_answer(answerKey)
  ~ answersOffered += answerKey
  ~ return true

=== function came_from(-> target)
// has the flow been through "target" this turn already?
  ~ return (TURNS_SINCE(target) == 0)
```

Each time the `offer_answer` function is invoked it checks to see if it's a new turn, and if it is, it clears out the `answersOffered` list. Note that by using a shuffle we encourage different options to appear from one turn to the next, as shuffles exhaust themselves before beginning again.

Varying options without tracking

The solution above works well when every question is tied to an identifiable key variable. But it can become unwieldy to have to define a matching key for every line of dialogue you want to include, simply to prevent the same option being offered twice.

A somewhat hacky but more extensible pattern is to split the conversation into multiple “buckets”, and pick one from each.

We'll loop each bucket for a while to maximise our chance of getting a valid line (because the lines might have been used already, or might fail other conditions on

them.)

```
- (opts)
  <- shuffler(-> big_topics, -> opts)
  <- shuffler(-> little_topics , -> opts )
  *   [ Stop talking ]
  "I've had enough of this!"
  -> END

=== big_topics(-> done)
{ shuffle:
  -   *   "Something massive!" -> done
  -   *   "Something serious!" -> done
  -   *   "Something huge!" -> done
}
-> DONE

=== little_topics(-> done)
{ shuffle:
  -   *   "Something minor!" -> done
  -   *   "Something trivial!" -> done
  -   *   "Something insignificant!" -> done
}
-> DONE

=== shuffler(-> choice_block, -> done)
~ temp loopsTaken = 0
~ temp x = CHOICE_COUNT()
- (picker)
~ loopsTaken ++
<- choice_block( done )
{ x == CHOICE_COUNT() && loopsTaken < 10:
  -> picker
}
-> DONE
```

We use this pattern in the game *Overboard!*, generally splitting the buckets into “minor” topics and “major” topics, so the player is offered one of each kind each time they can choose what to. (In that game, we also randomise the order in which two topic blocks are invoked, to avoid the big and little distinction becoming a fixed pattern.)

Note that the above can be made a little lighter on the writer by using the “threaded tunnel” concept inside the shuffler.

Topics blocks using weave

The above pattern has a drawback: inside a shuffle block, like the one we used in `big_topics`, you can use weave. So it's usually better to break topic blocks out using threads. The following is a direct refactor of the `big_topics` block above, that turns the top level of that knot into an “index” of topics, and gives each topic its own stitch for the actual writing.

Note this pattern makes it easier to author follow-up dialogue choices, as demonstrated in the ‘massive’ block below. The second line cannot appear until the first has been used up by the player.

```
=== big_topics(-> done)
{ shuffle:
-   <- massive_topic ( done)
-   <- serious_topic ( done)
-   <- huge_topic   ( done)
}
-> DONE

= massive_topic  (-> done)
*   (massive) "Something massive!"
*   {massive} "Something massive. Definitely."
-   -> done

= serious_topic  (-> done)
*   "Something serious!"    -> done

= huge_topic     (-> done)
*   "Something huge!"       -> done
```

Faster topic blocks

Sometimes the above pattern can seem like too much boilerplate code. Since each topic block is called repeatedly as ink hunts for a valid choice, we can also write something which is *almost* functionally the same, but much faster, by simply adding more randomness via conditionals – but if we do it this way, we'll need to keep checking that we're not bringing in more than one choice. (Or, since we're not in a loop anymore... we could just allow that to happen.)

```
=== function maybe()
~ return RANDOM(1, 3) == 1
```

```
=== function noChoiceYet(x)
  ~ return CHOICE_COUNT() == x

=== big_topics(-> done)
  ~ temp x = CHOICE_COUNT()
  *   {maybe()} {noChoiceYet(x)} "Something massive!"
  *   {maybe()} {noChoiceYet(x)} "Something serious!"
  *   {maybe()} {noChoiceYet(x)} "Something huge!"
  - -> done
```

3. Unchooseable choices

A common pattern in games is to display a choice that would be available should the player meet a condition they don't meet. By default ink doesn't support "unclickable choices", as any choice that fails its preconditions is simply not rendered to the player.

If we rethink this requirement as a UI design question, we see that what we really want here is not a third kind of choice, but rather a mark-up on the choice to show that it's unchooseable, for the code to recognise; and we can do this using choice tags.

```
VAR charm = 0
VAR strength = 0
VAR brains = 0
```

```
~ charm = RANDOM(0, 10)
~ strength = RANDOM(0, 10 - charm)
~ brains = 10 - charm - strength
```

The guard blocks your way. 'Not comin' in. Don't even traí.'

```
* [Attack the guard #{strength < 6 : locked} ]
  You charge the guard, roaring! He cowers.
  -> run_inside
```

```
* [Charm the guard #{charm < 6 : locked} ]
  You smile sweetly, and approach, blinking. The guard swoons.
  -> run_inside
```

```
* [Outwit the guard #{brains < 6 : locked} ]
  You point up. 'Look at that!'
  -> run_inside
```

```
* [Walk away ]
  'Oh, okay,' you grumble. 'But I might come back later.'
```

The code can then look for the `#locked` tag and ensure that choice is displayed, but can't be selected – and the example could be easily expanded to provide the player with data on why the choice was locked.

Database Patterns

When making a game in ink it's common to want to store up numerical values associated with things like inventory items. But ink doesn't, by default, have any traditional object structures.

You can cover this on the code side quite easily, but you can also get quite far in ink itself by using the concept of database functions – and this has the advantage of allowing you to still test your game content in inky.

1. Basic database functions

The most basic database function is the one we saw before, that converted a list element into a nice-looking name:

```
=== function nameOfPerson (who)
{ who:
- ElizabethBennett: Elizabeth Bennett
- DavidDarcy: David Darcy // he's probably called David
- JimBroadbent: Jim Broadbent // he's in everything
- else: {who} // by default, print the raw key
}
```

This idea can be used for any property the game requires, with the advantage that it's evaluated at run-time so can vary:

```
VAR yearsOfPlay = 0

=== function ageOfPerson (who)
{ who:
- ElizabethBennett: ~ return 18 + yearsOfPlay
- DavidDarcy: ~ return 36 + yearsOfPlay
- OldFather: ~ return 73 + MIN(yearsOfPlay, 4) // the
character dies
- else: [ Error: no age for {who} ]
}
```

Database of objects and properties

But what if each person in the game has more than one useful piece of data associated with them? Here's a solution that allows us to link data fields in the game.

```
LIST People = ElizabethBennett, DavidDarcy, JimBroadbent

LIST Titles = Mr, Miss, Sir

LIST Data = Name, Age, Title

~ temp who = LIST_RANDOM(LIST_ALL(People))
```


This is {PersonData(who, Title)} {PersonData(who, Name)},
who is {PersonData(who, Age)} years old.

```
=== function PersonData (who, what )
{ who:
- ElizabethBennett:
  ~ return data(what, "Elizabeth Bennett", 22, "Miss")
- DavidDarcy:
  ~ return data(what, "David Darcy", 37, "Mr")
- JimBroadbent:
  ~ return data(what, "Jim Broadbent", 79, "Sir")
- else: [ Error: no data associated with {who}. ]
}

=== function data(what, nameData, ageData, titleData)
{what:
- Name: ~ return nameData
- Age: ~ return ageData
- Title: ~ return titleData
}
```

The **PersonData** function is effectively a database, though it is called every time the values are used, which means that the values it provides can be dynamic and vary as the game continues.

2. Dynamic databases

The above works for computable values – but if values need to be tracked and set freely during the course of the game, you’ll need a little more engineering to store those values. The following pattern builds on the previous one, by passing in a parameter called **delta** to the core database: if it’s zero, it does nothing and returns the current value. If it’s not zero, it alters the value – if it can! – and then returns the new value.

```
LIST People = ElizabethBennett, DavidDarcy, JimBroadbent
LIST Titles = Mr, Miss, Sir
LIST Data = Name, Age, Title, LovePoints

VAR ElizabethLovePoints = 0
VAR DarcyLovePoints = 0
VAR JimLovePoints = 0

- (pick_person)
  ~ temp who = LIST_RANDOM(LIST_ALL(People))
  This is {PersonData(who, Title)} {PersonData(who, Name)}.
  They are {PersonData(who, Age)} years old.
  I love them {PersonData(who, LovePoints)}.

- (loop)
  + Love them more!
    ~ alter(who, LovePoints, 5)
  + Love them less!
    ~ alter(who, LovePoints, -5)
  + Someone else
    -> pick_person
- Now I love them {PersonData(who, LovePoints)}.
  -> loop

=== function alter(who, what, alterBy)
  ~ return mainPersonDatabase (who, what, alterBy)

=== function PersonData (who, what )
  ~ return mainPersonDatabase (who, what, 0)

== function mainPersonDatabase(who, what, delta)
  { who:
    - ElizabethBennett:
      ~ return data(what, "Elizabeth Bennett", 22, "Miss",
        ElizabethLovePoints, delta)

    - DavidDarcy:
      ~ return data(what, "David Darcy", 37, "Mr",
        DarcyLovePoints, delta)
```

```

- JimBroadbent:
  ~ return data(what, "Jim Broadbent", 79, "Sir",
JimLovePoints, delta)
- else:
  [ Error: no data associated with {who}. ]
}

=== function data(what, nameData, ageData, titleData, ref
lovePoints, delta)
{what:
- Name:
  ~ return nameData
- Age:
  ~ return ageData
- Title:
  ~ return titleData
- LovePoints:
  { delta != 0:
    ~ lovePoints += delta
  }
  ~ return lovePoints
}

```

While the above example is not a good dating sim, it is a good example of what makes a useful ink pattern. While a bit fiddly to set up, it's now very easy to extend. Adding a new stat to each character is straight-forward; and adding an additional character just means adding an additional line in the database, and providing a variable to track their `lovePoints`.

Also, because the data is evaluated “live”, you can alter it based on game events by adding logic to the `mainPersonDatabase` function. Should Sir Jim Broadbent be stripped of his hypothetical knighthood, for instance, you might have:

```

- JimBroadbent:
  ~ temp title = "{disgraced:Merely|Sir}"
  ~ return data(what, "Jim Broadbent", 79, title,
JimLovePoints, delta)

```

... and the logic here can be as involved as you require it to be.

Decision-Making Patterns

ink is designed for creating branching conversation trees, but the same structures can be used for decision-making processes, so long as they're reliant on a basically priority-based structure rather than something more stochastic.

The follow section collects up patterns based on making decisions, first at random, and then using a “first, best match” approach.

1. Making random decisions

In games with a strong random component it's often useful to be able to choose a knot in the game at random from some kind of index.

```
~ temp goToKnot = getKnot()  
-> goToKnot
```

```
=== function getKnot()  
  { shuffle:  
    - ~ return -> knot_A  
    - ~ return -> knot_B  
    ...  
  }
```

But what if the list of what's available is dynamic? The `shuffle` might happen to choose something which can't be used this time. We can cover this by providing a fallback – and using a “recurse on failure” pattern to ensure we *find* it.

```
=== function getKnot()  
  {shuffle:  
    // some options guarded by conditionals  
    - { condition: ~ return -> knot_A }  
    - { condition: ~ return -> knot_B }  
  
    // at least one option that can definitely be used  
    - ~ return -> knot_C  
  }  
  
  // if we picked one we weren't allowed, pick again!  
  ~ return getKnot()
```

So long as there's something this function is allowed to choose every time it's called, it'll always get to a return value eventually and there won't be an infinite loop. (This isn't an efficient pattern, but in reality, it's rarely going to slow ink down by much.)

2. AI decisions

It's common for games to have opponents. Less so, for story games to have them. But should a story require an NPC who can act against the player intelligently, this requires a decision making pattern – an analysis of the situation, followed by a choice of appropriate response.

ink can do this nicely using fallback choices – so long as the flow finds no “real” choices, it will work through the valid fallbacks and pick the first one to follow.

Here's a simple example. An NPC character is playing a game of a rock, paper, scissors. We could choose at random but people don't: they often base their decision on what the other player did last turn.

```
+ { playersLastPlay == Rock } {RANDOM(1, 3) <= 2 } ->
  ~ choice = Scissors
+ { playersLastPlay == Scissors } {RANDOM(1, 3) <= 2 }
->
  ~ choice = Paper
+ { playersLastPlay == Paper } {RANDOM(1, 3) <= 2 } ->
  ~ choice = Rock
+ { playersLastPlay == Rock } ->
  ~ choice = Paper
+ ->
  ~ choice = Rock
```

3. Reaction dialogue

In games it's frequent for the narrative content to be short and reactive – a “bark” – and triggered in response to some in-game action.

```
if player is shot at => shout 'Ow!'
```

But in some games a more heuristic approach is used, with the line delivered being the “best-fit” for the circumstance.

```
if the player is shot at by a catapult => shout 'Rascal!'
otherwise if the player is shot at => shout 'Ow!'
```

This is a decision-making pattern too; the game works through some kind of prioritised list looking for the first match, and delivers it.

```
VAR health = 100
```

```
VAR damage = 20
```

```
- (top)
```

```
~ damage = RANDOM(0 , 20)
```

```
~ health -= damage
```

```
You have been hit for [{damage}] bringing your health to
[{health}].
```

```
*   { health < 0} ->
    "You killed me!"
```

```
-> END
```

```
*   { damage > health }->
    "I can't take another hit like that."
```

```
*   {damage == 0} ->
    "Ha! Missed me!"
```

```
*   { damage <= 3} ->
    "Just a flesh-wound!"
```

```
*   { health < 10} ->
    "I can't take much more of this!"
```

```
*   { health < 50 } ->
    "This isn't going as well as I'd hoped it would."

*   { health > 80 } { damage < 5 } ->
    "What? Are you going to tickle me to death?"
*   { damage > 15 } ->
    "Ow, that really hurt!"

+   -> // fallback that does nothing

-   -> top
```

ink's system of one-time choices is great here for preventing repetition of content, but in a longer game you might want to consider using `{not seen_recently(-> name_of_choice)}` on a sticky choice instead, to allow repetition after some time has elapsed.

A pattern like this is really a database, and can be extended with new lines as and when they're needed, simply by adding the line with the necessary conditionals at the right point in the list. And of course, you have access to the rest of the story state, so you can vary the dialogue itself based on other factors, should you need to.

A Game of Pontoon (a long example)

It's traditional for books about computer programming to close on a ludicrously long and hard to type in example, so here's one from us: a simplified version of the playable Pontoon game from inkle's reverse-murder-mystery *Overboard!*

This example uses a lot of the tricks listed in this chapter and employs a lot of advanced features. A deck of cards is simulated using a list, with variables to hold what's in each player's hand. There's an AI routine for working out what the NPC (Carstairs, an English gentleman and card shark) will do each turn.

In the real game, conversation options are added between hands, and there a few other more nefarious strategies for tilting the game in your favour!

```
-> play_game -> END

/* -----

    Functions and Definitions

----- */

VAR myCards = ()    // my hand
VAR hisCards = ()   // his hand
VAR faceUpCards = () // the face up cards (from both
hands)

VAR money = 400
VAR CstrsBank = 1000

// The deck is stored as a list, but note the values we
assign:
// Spades take the values 1 through 13
// Diamonds 101 through 113
// Hearts 201 through 213
// Clubs 301 through 313
// This allows us to convert a list item into its face
value and suit

LIST PackOfCards =
```

```
A_Spades = 1, 2_Spades, 3_Spades, 4_Spades,  
5_Spades, 6_Spades, 7_Spades, 8_Spades,  
9_Spades, 10_Spades, J_Spades, Q_Spades, K_Spades,
```

```
A_Diamonds = 101 , 2_Diamonds, 3_Diamonds, 4_Diamonds,  
5_Diamonds, 6_Diamonds, 7_Diamonds, 8_Diamonds,  
9_Diamonds, 10_Diamonds, J_Diamonds, Q_Diamonds,  
K_Diamonds,
```

```
A_Hearts = 201, 2_Hearts, 3_Hearts, 4_Hearts,  
5_Hearts, 6_Hearts, 7_Hearts, 8_Hearts,  
9_Hearts, 10_Hearts, J_Hearts, Q_Hearts, K_Hearts,
```

```
A_Clubs = 301, 2_Clubs, 3_Clubs, 4_Clubs,  
5_Clubs, 6_Clubs, 7_Clubs, 8_Clubs,  
9_Clubs, 10_Clubs, J_Clubs, Q_Clubs, K_Clubs
```

```
LIST Suits = Spades = 0, Diamonds, Hearts, Clubs
```

```
LIST Values = Ace = 1, Two, Three, Four, Five, Six,  
Seven, Eight, Nine, Ten, Jack, Queen, King
```

```
=== function suit(x)  
  // Suit is derived from the integer part of the card  
  value / 100  
  ~ return Suits(INT(FLOOR(LIST_VALUE(x) / 100)))
```

```
=== function number(x)  
  // Face value is the tens and units part of the card  
  value  
  ~ return Values(LIST_VALUE(x) mod 100)
```

```
=== function value(x)  
  // in Pontoon, all face cards (King, etc) are worth 10.  
  ~ return MIN(LIST_VALUE(x) mod 100, 10)
```

```
=== function shuffle()  
  ~ PackOfCards = LIST_ALL(PackOfCards)
```

```
=== function addCard(ref toHand, faceUp)  
  ~ temp x = pullCardOfValue(LIST_ALL(Values)) // choose a
```

```

card
  ~ temp retVal = addSpecificCard( toHand, x, faceUp) //
deal it
  ~ return retVal

=== function pullCardOfValue(valuesAllowed)
  ~ temp card = pop_random(PackOfCards)
  { card:
    { valuesAllowed != number(card):
      ~ return pullCardOfValue(valuesAllowed)
    }
    ~ return card
  }
[ Error: couldn't find a card of value {valuesAllowed}!
]
  ~ shuffle()
  ~ return pullCardOfValue(valuesAllowed) // try again

=== function addSpecificCard(ref toHand, x, faceUp)
  ~ toHand += x
  {faceUp:
    ~ faceUpCards += x
  }
  ~ return x

/* -----

Querying Hands

----- */

=== function handContains(x, card)
  ~ temp y = pop(x)
  { y:
    { number(y) == card:
      ~ return true
    - else:
      ~ return handContains(x, card)
    }
  }
}

```

```

~ return false

=== function isPontoon(x)
~ return handContains(x, Ace) && (handContains(x, King)
|| handContains(x, Queen) || handContains(x, Jack)) &&
LIST_COUNT(x) == 2

=== function minTotalOfHand(x)
~ temp y = pop(x)
{y:
~ return minTotalOfHand(x) + value(y)
}
~ return 0

=== function maxTotalOfHand(x)
~ temp minTot = minTotalOfHand(x)
{handContains(x, Ace) && minTot <= 11:
~ return minTot + 10
- else:
~ return minTot
}

/* -----

```

Printing Cards and Hands

```

----- */

=== function nameCard(x)
{ _nameCard(x, true) }

== function listMyCards()
~ _listOfCards(myCards)

=== function printHand(x)
~ _printHand(x)

=== function _nameCard(x, allowVariants)
~ temp num = number(x)
{allowVariants:
{ RANDOM(1, 3) == 1:

```

```

    a{(Eight, Ace) ? num :<>n} {num} in {suit(x)}
- else:
    the {num} of {suit(x)}
}
- else:
    {num} of {suit(x)}
}

```

```

== function _listOfCards(hand)
~ temp y = pop(hand)
{ y:
  <>{_nameCard(y, false)}
  {hand:
    <><br>
    ~ _listOfCards(hand)
  }
}

```

```

=== function _printHand(x)
~ temp y = pop(x)
{y:
  {nameCard(y)}
  {LIST_COUNT(x):
    - 0:
      ~ return
    - 1:
      <> and {_printHand(x)}
    - else:
      <>, {_printHand(x)}
  }
}

```

```

=== function printHandDescriptively(x, mine)
{printHand(faceUpCards ^ x)} face up
~ temp faceDownCards = x - faceUpCards
{faceDownCards:
  <>, and <>
  { mine:
    {printHand(faceDownCards)}
  - else:

```

```

    {print_number(LIST_COUNT(faceDownCards))} <> more
  }
  <> {~{mine:hidden|}|face down|blind}
}

/* -----

Other Printing Functions

----- */

=== function finalTotalOfHand(x)
{ isPontoon(x):
  pontoon
- else:
  {print_number(maxTotalOfHand(x))}
}

=== function sayTotalOfHand(x)
~ temp minTot = minTotalOfHand(x)
{ shuffle:
-   for a total of
-   total of
-   giving
-   making
}
<> {print_number(minTot)}
{ handContains(x, Ace) && minTot <= 11:
  ~ temp max = maxTotalOfHand(x)
  <>, or {print_number(maxTotalOfHand(x))}
}

=== function describeMyCards()
{ shuffle:
-   V:      ... {printHandDescriptively(myCards, true)}.
#thought
- { shuffle:
-   CARSTAIRS: {~First {~card|out|up} {!for you} is|}

```

```

- CARSTAIRS: The lady {~has|gets}
}
<> {nameCard(faceUpCards ^ myCards)}
V: ... and face down, {nameCard(myCards -
faceUpCards)} ... #thought
}
V: ... {sayTotalOfHand(myCards)} ... #thought

== function describePot(bet)
{ shuffle:
- CARSTAIRS: The {~bet|stake|pot} is
{print_number(bet)} pounds.
- CARSTAIRS: {~That makes|{~There|That}'s}
{print_number(bet)} pounds {~in the pot|on the table}.
}

/*-----

GAMEPLAY CONTENT LOOP

-----*/

=== play_game
- (top_of_game)
~ temp startingMoney = money
~ myCards = ()
~ hisCards = ()
~ faceUpCards = ()
~ temp bet = 20
{ once:
- V0: I throw two ten-pound notes onto the table.
- V: Twenty pounds.
CARSTAIRS: The pot stands at twenty pounds.
- V0: I toss in my ante.
}
{
- LIST_COUNT(PackOfCards) < 10:
~ shuffle()
~ temp plural = RANDOM(1,2)
V0: Carstairs {~collects together|gathers up}
{plural:{~all|} the cards|the deck}, and

```

```

{~riffles|shuffles} {plural:them|it}
{~thoroughly|expertly|quickly|carelessly||} before
dealing the first two cards.
- else:
  VO:      Carstairs {~passes me|spins me|tosses
over|deals out} {~{~an opening|a new} card|my first card}
{~face up|} {~from the {~top of the|} deck|}.
  }

~ temp myNewCard = addCard(myCards, true)

{ shuffle:
- CARSTAIRS: {~First {~card|out} is|}
{nameCard(myNewCard)}.
- CARSTAIRS: The lady {~has|gets}
{nameCard(myNewCard)}.
}
~ temp hisNewCard = addCard(hisCards, true)
{ stopping:
- CARSTAIRS: And the dealer... gets
{nameCard(hisNewCard)}.
- { shuffle:
- CARSTAIRS: And it's {nameCard(hisNewCard)} for me.
- CARSTAIRS: {~Dealer {~gets...|has}|And I have}
{nameCard(hisNewCard)}.
}
}

{once:
- CARSTAIRS:      You can fold, or make a bet to stay
in.
}
~ temp incr = 0

- (bet_opts)
+ [ Fold ]
V: {~Pass|Fold}.
-> i_lost

+ [ Bet 50 ]
~ incr = 50

```



```

+ {money - bet < 200} [ Bet 100 ]
~ incr = 100

+ {money - bet >= 200} [ Bet higher... ]
+ + {CHOICE_COUNT() < 2 } {money - bet <= 300} [ Bet
100 ]
~ incr = 100
+ + {CHOICE_COUNT() < 2 } {money - bet <= 250} [ Bet
150 ]
~ incr = 150
+ + {CHOICE_COUNT() < 2 } [ Bet 200 ]
~ incr = 200
+ + {CHOICE_COUNT() < 2 } {money - bet >= 300} [ Bet
300 ]
~ incr = 300
+ + [ Bet lower... ]
-> bet_opts
- { shuffle:
- V: I put in {print_number(incr)} pounds {incr >
50:more}.
- V: I raise {print_number(incr)} pounds.
}
{ incr >= 200:
{ shuffle once:
- VO: Carstairs raises an eyebrow.
- CARSTAIRS: Crikey.
- CARSTAIRS: Well, now.
- CARSTAIRS: Someone's feeling lucky.
}
}
- ~ bet += incr
{ describePot(bet) }
VO: He {~hands|deals} {~me|out} a second card, face
down.

{once:
- CARSTAIRS: Take a look, don't let me see.
}
~ myNewCard = addCard(myCards, false)

```

```

V: {nameCard(myNewCard)}: {sayTotalOfHand(myCards)}
#thought

~ addCard(hisCards , false)
{ shuffle:
- VO: He deals one more for himself, face down.
- CARSTAIRS: One more blind for me, too.
}

- (myplay)
{ minTotalOfHand(myCards) > 21:
{ shuffle:
- V: I'm bust.
- V: Damn.
- VO: I {~toss|throw} my cards down.
}
{ i_lost mod 3 == 2:
{ shuffle:
- V: You're rigging this.
- V: How are you doing this?
- V: This can't be fair.
}
{ shuffle:
- CARSTAIRS: I assure you I'm not!
- CARSTAIRS: I play the odds, Ma'am, not the
player.
- CARSTAIRS: I promise you, I'm as square as they
come!
}
}
-> i_lost
}
{ LIST_COUNT(myCards) == 5:
CARSTAIRS: A five card trick!
CARSTAIRS: That beats the same value on fewer cards.
}

- (check_for_burn)
{ LIST_COUNT(myCards) == 2 && minTotalOfHand(myCards) ==
13 && money - bet >= 20:
+ {came_from(-> burny)}

```

```

[ Burn again ]
-> burny

+ (burny) {not came_from(-> burny)}
[ Burn for twenty more ]
~ bet += 20
V: Burn.
>>> AUDIO CardCollectAndDealTwoCards
VO: Carstairs collects the cards and deals two
more.
~ faceUpCards -= myCards
~ myCards = ()
~ addCard(myCards, true)
~ addCard(myCards, false)
V: {printHandDescriptively(myCards, true)}
#thought
V: {sayTotalOfHand(myCards)} #thought
-> check_for_burn

* [ Keep them ]
-> bid_loop
- else:
-> bid_loop
}
-> DONE

- (bid_loop)
{ not seen_very_recently(-> describePot):
{ describePot(bet) }
}
~ temp gotTwentyOne = (maxTotalOfHand(myCards) == 21)
{gotTwentyOne:
{isPontoon(myCards):
V: ... It's a pontoon..! #thought
- else:
V: ... Twenty-one! #thought
}
}
}

+ [ Stick {not gotTwentyOne: on
{finalTotalOfHand(myCards)}} ]

```

```

CARSTAIRS: Final bet is {print_number(bet)} pounds.
-> hisplay_begins

* (gloat) {gotTwentyOne} [ Gloat ]
>>> AUDIO: V Chuckle 1
V: You're in trouble now, Mr Carstairs...
CARSTAIRS: Is that so?
-> hisplay_begins

* {gotTwentyOne} [ Give nothing away ]
>>> AUDIO: V Clear Throat 1
V: Your turn, then.
CARSTAIRS: I take it you're sticking, then?
-> hisplay_begins

+ {not gotTwentyOne} [ Twist ]
{ shuffle:
- V: Twist.
- V: Another card.
- V: Give me another.
- V: One more, face up.
}
~ temp newUpCard = addCard(myCards, true)

CARSTAIRS: {nameCard(newUpCard)}.

V: ... {sayTotalOfHand(myCards)}. #thought
-> myplay

+ { (money - bet) >= 50 } {not gotTwentyOne}
[ Buy for fifty ]

~ bet += 50
~ temp newDownCard = addCard(myCards, false)
{shuffle:
- V: Buy.
- V: I'll buy one.
- V: One more, face down.
}
{shuffle:
- CARSTAIRS: The stake is now {print_number(bet)}.

```

```

- CARSTAIRS: {print_number(bet)} in the pot.
}

{ shuffle:
- VO: Carstairs passes me another card, face-
down.
- CARSTAIRS: Here's your card.
}

V: ... {nameCard(newDownCard)}. #thought
V: ... {sayTotalOfHand(myCards)}. #thought
-> myplay

- (hisplay_begins)

~ faceUpCards += hisCards

{ shuffle:
- CARSTAIRS: Let's see what I have...
CARSTAIRS: {printHandDescriptively(hisCards, false)}.
- CARSTAIRS: Dealer has...
{printHandDescriptively(hisCards, false)}.
}

CARSTAIRS: {sayTotalOfHand(hisCards)}.

- (hisplay_main)

// AI plays

~ temp hes_scare = seen_more_recently_than(-> gloat, ->
top_of_game)
~ temp hisTotal = minTotalOfHand(hisCards)

{ hisTotal > 21:
{ shuffle:
- CARSTAIRS: I'm bust!
- CARSTAIRS: Too high!
- CARSTAIRS: No luck there!
}
-> i_won

```

```

}

~ temp hisMaxTotal = maxTotalOfHand(hisCards)
~ temp yourVisibleTotal = maxTotalOfHand(myCards ^
faceUpCards)
~ temp yourBestTotal = 21

// edge case. You have ? - 3 - 5 => your best is 19.
{ LIST_COUNT(myCards - faceUpCards) == 1 &&
yourVisibleTotal < 10:
  ~ yourBestTotal = 11 + yourVisibleTotal
}

// AI uses fallback choices to pick a strategy

+ {hisMaxTotal > yourBestTotal || (hisMaxTotal ==
yourBestTotal && LIST_COUNT(myCards) < 5)} ->
- - (he_sticks)
  CARSTAIRS: Dealer sticks on
{finalTotalOfHand(hisCards)}.
  -> hisplayover
+ { hisMaxTotal >= 18 && !handContains(hisCards, Ace)}
-> he_sticks

+ { hisTotal == 10 || hisTotal == 11 } -> he_twists

+ { hisMaxTotal <= 15 || (hisMaxTotal <= 17 &&
handContains(hisCards, Ace)) || (hisMaxTotal <= 18 &&
hes_scared) } ->
- - (he_twists)
  { shuffle:
    - CARSTAIRS: I'll take another.
    - CARSTAIRS: Dealer twists.
    - CARSTAIRS: One more...
  }

~ temp newHisCard = addCard(hisCards, true)
CARSTAIRS: {nameCard(newHisCard)},
{sayTotalOfHand(hisCards)}.
-> hisplay_main

```

```

+   {RANDOM(1, 3) == 1} ->
    -> he_sticks

+   -> he_twists

-   (hisplayover)
    ~ temp facedownCards = myCards - faceUpCards

-   (dealoutcards)
    { pop(facedownCards):
      -> dealoutcards
    }
    ~ temp scoreDiff = maxTotalOfHand(myCards) -
maxTotalOfHand(hisCards)

    { cycle:
      -   V0:      I lay my cards down.
      -   V0:      I {~turn|flip} my cards {~face-up|over}.
      -
    }

    { cycle:
      -   V:  I've got {scoreDiff < 0:only}
{finalTotalOfHand(myCards)}{scoreDiff==0:<> too}.
      -   V:      {finalTotalOfHand(myCards)}.
    }

    {
      - scoreDiff > 0 && maxTotalOfHand(myCards) < 21:
        V:  I won{~.|!!|?}
        -> i_won

      - scoreDiff < 0:
        CARSTAIRS:  Dealer wins!
        -> i_lost

      - scoreDiff == 0:
        {LIST_COUNT(myCards) >= 5 && LIST_COUNT(hisCards) < 5:
          CARSTAIRS:  Five card trick wins!
          -> i_won
        }
    }

```

```

CARSTAIRS: It's a draw. Dealer wins, I'm afraid.
-> i_lost
}

- (i_won)
~ money += bet
~ CstrsBank -= bet
VO:      I collect up the money from the table.
{
- isPontoon(myCards):
CARSTAIRS: And pontoon earns double.
~ money += bet
~ CstrsBank -= bet
VO:      He counts out another {print_number(bet)}
pounds.

- maxTotalOfHand(myCards)==21 && LIST_COUNT(myCards)==2:
{ once:
- CARSTAIRS: But it's not a pontoon, I'm afraid.
CARSTAIRS: Need a face card for that.
}
}
{ shuffle:
- VO:      I've now got {print_number(money)} pounds.
- V:      ... I've now got {print_number(money)}
pounds.
}
-> done

- (i_lost)
~ money -= bet
~ CstrsBank += bet
VO:      Carstairs {~takes|{~collects|scoops} {~up|}} the
{~pot|stake|money {~{~off|from} the table|}} and gathers
up the cards.
{ money < 50:
V: You've cleaned me out!
CARSTAIRS: I'm sorry to hear that, Mrs V.
VO:      He tucks his winnings into his waistcoat pocket
and grins like an idiot.
-> finished

```



```

}
{ money >= startingMoney:
  { shuffle:
    - V0:      I've still got {print_number(money)}
pounds.
    }
    - else:
      { shuffle:
        - V:      ... I'm down to {print_number(money)}
pounds ... #thought
        - V:      ... {print_number(money)} pounds left ...
#thought
      }
    }
  -> done

- (done)
~ temp wasPontoon = isPontoon(myCards)
~ myCards = ()

{ CstrsBank <= 50:
  CARSTAIRS: Well, you've cleaned me out of spending
money, Mrs Villensey!
  CARSTAIRS: I must say; a much better show than your
husband achieved.
  -> finished
}

{
- came_from(-> i_lost):

  {shuffle:
    - CARSTAIRS:      Have you had enough?
    - CARSTAIRS:      Keep going?
    - CARSTAIRS:      Again?
  }
- came_from(-> i_won):

  { shuffle:
    - CARSTAIRS:      Another round?
    - CARSTAIRS:      Again?
  }
}

```

```

- CARSTAIRS:      Another?
}
- else:

{ cycle:
- VO:      Carstairs {~has been squaring up|is
fiddling with} the {~pack|deck}.
- VO:      Carstairs is shuffling idly.
  ~ shuffle()
}
{ shuffle:
- CARSTAIRS:      Are we still playing?
- CARSTAIRS:      Another hand, Mrs Villensey?
}
}
- (replay_opts)
+ [ Play another round ]
{
- money >= 250:
{ shuffle:
- V: Deal.
- V: Another!
}
- money >= 100:
{ shuffle:
- V: I'll play another round.
- V: I'm not finished yet.
}
- money >= 70:
{ shuffle:
- V: I can afford one more round.
- V: I'd better be lucky this time!
}
}
-> top_of_game

+ [ Stop playing ]
V: Perhaps later.

- (finished)
~ myCards = ()

```

```

->->

/*-----

STOCK FUNCTIONS

-----*/

=== function came_from(-> x)
~ return TURNS_SINCE(x) == 0

=== function seen_very_recently(-> x)
~ return TURNS_SINCE(x) >= 0 && TURNS_SINCE(x) <= 3

=== function seen_more_recently_than(-> link, -> marker)
{ TURNS_SINCE(link) >= 0:
  { TURNS_SINCE(marker) == -1:
    ~ return true
  }
  ~ return TURNS_SINCE(link) < TURNS_SINCE(marker)
}
~ return false

=== function pop(ref _list)
~ temp e1 = LIST_MIN(_list)
~ _list -= e1
~ return e1
=== function pop_random(ref _list)
~ temp e1 = LIST_RANDOM(_list)
~ _list -= e1
~ return e1

=== function print_number(x)
{
- x >= 100:
  ~ temp z = x mod 100
  {print_number((x - z) / 100)} hundred
  {z > 0: <> and {print_number(z)} }
- x == 0:      zero
- else:
  { x >= 20:

```

```

{ x / 10:
- 2: twenty
- 3: thirty
- 4: forty
- 5: fifty
- 6: sixty
- 7: seventy
- 8: eighty
- 9: ninety
}
{ x mod 10 > 0: <>-<> }
}
{ x < 10 || x > 20:
{ x mod 10:
- 1: one
- 2: two
- 3: three
- 4: four
- 5: five
- 6: six
- 7: seven
- 8: eight
- 9: nine
}
- else:
{ x:
- 10: ten
- 11: eleven
- 12: twelve
- 13: thirteen
- 14: fourteen
- 15: fifteen
- 16: sixteen
- 17: seventeen
- 18: eighteen
- 19: nineteen
}
}
}

```

www.inklestudios.com